



UNIVERSITÉ DE FIANARANTSOA
ÉCOLE NATIONALE D'INFORMATIQUE

MÉMOIRE DE FIN D'ÉTUDES POUR L'OBTENTION DU DIPLÔME DE MASTER PROFESSIONNEL

Mention : Informatique

Parcours : Informatique Générale

Intitulé :

ÉTUDE DE LA MISE EN PLACE D'UN PIPELINE CI/CD ET ORCHESTRATION POUR UNE APPLICATION EN ARCHITECTURE MICROSERVICES

Présenté le 26 novembre 2025

Par Monsieur RAVELOMAHARAVO Rotsy Maminiaina Thierry Michaël

Membres du jury :

Président :	Monsieur RAFAMANTANANTSOA Fontaine, Professeur
Examinateur :	Monsieur RAZAFINDRAMONJA Clément Aubert, Assistant d'Enseignement Supérieur et de Recherche
Rapporteur :	Monsieur RANDRIANOMENJANAHARY Lala Ferdinand, Maître de Conférences

Année Universitaire : 2024-2025

CURRICULUM VITAE

RAVELOMAHARAVO Rotsy Maminiaina Thierry Michaël

Âge : 24 ans

Situation matrimoniale : Célibataire

Adresse : 240 Bis | 3611 Tanambao, Fianarantsoa 301

Nationalité : Malagasy

Téléphone : +261 34 88 359 57

E-mail : thierrymichael2001@gmail.com



PARCOURS ACADÉMIQUE

- **2024-2025 :** Deuxième année de formation en master professionnel à l'École Nationale d'Informatique. Parcours : Informatique Générale.
- **2023-2024 :** Première année de formation en master professionnel à l'École Nationale d'Informatique. Parcours : Informatique Générale.
- **Octobre 2024 :** Certification Google Cloud Platform – Associate Cloud Engineer (ACE).
- **2022-2023 :** Troisième année de licence professionnelle à l'École Nationale d'Informatique, parcours Informatique Générale, et obtention du diplôme de licence (mention Très bien pour le mémoire portant sur la mise en place d'un CI/CD pour une API Node.js avec Jenkins).
- **2021-2022 :** Deuxième année de licence professionnelle à l'École Nationale d'Informatique. Parcours : Informatique Générale.
- **2019-2021 :** Première année de licence professionnelle à l'École Nationale d'Informatique. Parcours : Informatique Générale.

STAGES ET EXPÉRIENCES PROFESSIONNELLES

- **Décembre 2024 – présent :** DevOps chez TETIKA – SIDINA
  - Automatisation des tests de non-régression avec Cypress
  - Déploiements sur AWS, CI/CD GitLab, gestion de Kubernetes et Docker
- **Août 2024 – Novembre 2024 :** Développeur FullStack JavaScript chez MAR IT Consulting
  - Développement d'applications Vue 3 / Nuxt.js / Node.js / Supabase
  - OCR de factures, UI/UX avancée, gestion des droits (RBAC)
- **Janvier 2024 :** Développeur Back-End Node.js chez OnlyTravaux
  - Backend Node.js, GraphQL, intégration d'API

- **Octobre 2023 – Décembre 2023 :** Stagiaire DevOps chez OpenData Madagascar
  - CI/CD avec Jenkins, automatisation du déploiement d'une API Node.js
- **Octobre 2022 – Décembre 2022 :** Administrateur réseaux stagiaire à l'ESD Nanisana, Antananarivo
  - Mise en place d'un portail captif, développement d'une application dédiée, gestion de l'infrastructure réseau

PROJETS ACADÉMIQUES

- Portail captif (Mikrotik, PfSense)
- Système VoIP avec Asterisk
- Répartition de charge et basculement (HAProxy)
- Clusters Docker Swarm et Kubernetes (Vagrant, VirtualBox)
- Portfolio en ligne, outils SanityCheck, MyAgenda, FileShare, Faker Data Generator
- API de ressources humaines, gestion d'emprunts et d'employés, réservation de chambres, gestion d'appartements

COMPÉTENCES INFORMATIQUES

- **Langages de programmation :** PHP, Python, Node.js, Java, TypeScript, GraphQL, SQL
- **Frameworks et bibliothèques :** React, React Native, Vue.js, Nuxt.js, Quasar, Tailwind CSS
- **SGBD :** MySQL, PostgreSQL, Supabase
- **Système d'exploitation :** Linux (Debian)
- **Cloud et DevOps :** GCP, AWS, GitLab CI/CD, GitHub Actions, Jenkins, Ansible, Terraform, Docker, Docker Compose, Kubernetes, Vagrant
- **Logiciels :** VirtualBox, EoN, GNS3, Jenkins, Asterisk, Git, Docker, Vagrant, Kubernetes, Supabase, Mikrotik, PfSense, HAProxy
- **Services :** SSH, Apache2
- **Tests :** Cypress

CONNAISSANCES LINGUISTIQUES

Langues	Parlé	Écrit	Compréhension
Français	Très bien	Très bien	Très bien
Anglais	Très bien	Très bien	Très bien

LOISIRS

Lecture, volley-ball.

SOMMAIRE

CURRICULUM VITAE I

SOMMAIRE III

REMERCIEMENTS VI

LISTE DES FIGURES VII

LISTE DES TABLEAUX IX

LISTE DES ABRÉVIATIONS X

INTRODUCTION GÉNÉRALE 1

PARTIE I. PRÉSENTATIONS 2

Chapitre 1. Présentation de l'ENI 3

1.1. Information d'ordre général 3

1.2. Missions et historique 3

1.3. Organigramme institutionnel 5

1.4. Domaines de spécialisation 5

1.5. Architecture des formations pédagogiques 6

1.6. Relation de l'ENI avec les organismes externes 7

1.7. Débouchés professionnels des diplômés 8

1.8. Ressources humaines 11

Chapitre 2. Présentation de TETIKA 12

2.1. Information générale sur la société 12

2.2. Missions et historique 12

2.3. Organigramme fonctionnel et hiérarchique de TETIKA 13

2.4. Domaines d'expertise 15

2.5. Métiers 16

Chapitre 3. Description du projet 17

3.1. Formulation 17
3.2. Objectif du projet 17
3.3. Moyens nécessaires à la réalisation du projet 17
3.4. Résultats attendus 18
3.5. Chronogramme de travail 19

PARTIE II. ANALYSE PRÉALABLE ET CHOIX DES OUTILS 21

Chapitre 4. Analyse de l'existant et problématique 22
4.1. Analyse de l'existant 22
4.2. Problématique 29
4.3. Proposition de solution 30

Chapitre 5. Études théoriques et choix des outils 31
5.1. Notions de microservices 31
5.2. Gestionnaire de versions 31
5.3. Conteneurisation 33
5.4. Notions d'intégration continue et de déploiement continu 34
5.5. Notions d'orchestration 36
5.6. Choix de la solution de virtualisation 37
5.7. Scripts YAML 38
5.8. Plateformes d'orchestration de conteneurs Docker 39

PARTIE III. MISE EN ŒUVRE 41

Chapitre 6. Installations et configurations 42
6.1. Infrastructure 42
6.2. Automatisation CI/CD 45
6.3. Configuration de l'orchestration : manifests Kubernetes 58
6.4. Application et services à déployer 69
6.5. Vérification des jobs 87

Chapitre 7. Validations et perspectives 91
7.1. Validation du cluster Kubernetes 91

7.2. Validation des manifests des microservices 93
7.3. Validation de la mise à l'échelle automatique 95
7.4. Validation d'un déploiement de bout en bout 97
7.5. Perspectives d'amélioration 103

CONCLUSION GÉNÉRALE XII

BIBLIOGRAPHIE XIII

WEBOGRAPHIE XIV

GLOSSAIRE XV

TABLE DES MATIÈRES XVII

RÉSUMÉ XXI

REMERCIEMENTS

Je tiens à exprimer ma sincère gratitude à tous ceux qui m'ont aidé dans la réalisation de ce mémoire de master, ainsi qu'à toutes les personnes qui ont contribué de près ou de loin à l'aboutissement de ce travail. Je tiens à remercier :

- Monsieur HAJALALAINA Aimé Richard, Professeur, Président de l'Université de Fianarantsoa, pour avoir accepté mon inscription au sein de l'université ;
- Monsieur MAHATODY Thomas, Professeur, Directeur de l'École Nationale d'Informatique, qui a bien voulu m'admettre dans son établissement et suivre ma formation ;
- Monsieur RAFAMANTANANTSOA Fontaine, Professeur, qui me fait l'honneur de présider le jury de cette soutenance ;
- Monsieur RAZAFINDRAMONJA Clément Aubert, Assistant d'Enseignement Supérieur et de Recherche, Responsable du parcours Informatique Générale, qui a accepté d'examiner ce travail ;
- Monsieur RANDRIANOMENJANAHARY Lala Ferdinand, Maître de Conférences, qui a bien voulu m'encadrer pédagogiquement et être le rapporteur de ce mémoire, pour sa disponibilité et ses précieux conseils ;
- Monsieur Pascal AMET et Monsieur Guillaume VIGOUROUX, dirigeants de la société TETIKA, de m'avoir accueilli au sein de leur entreprise ;
- Monsieur ANDRIAMAHATRADRAIBE Marjean Claude, Directeur du développement de TETIKA, pour sa confiance et son accompagnement tout au long de ce travail ;
- Tous les membres du corps professoral de l'École Nationale d'Informatique, qui nous ont transmis les connaissances nécessaires ;
- Tous les collaborateurs de TETIKA, en particulier l'équipe des tests automatisés, pour leur accueil et leur soutien ;
- Ma famille, qui m'a toujours soutenu tant moralement que financièrement.

LISTE DES FIGURES

Figure 1 : Organigramme de l'École Nationale d'Informatique 5
Figure 2 : Architecture des études correspondant au système LMD 7
Figure 3 : Organigramme fonctionnel et hiérarchique de l'entreprise TETIKA 14
Figure 4 : Organigramme fonctionnel de la direction du développement 15
Figure 5 : Diagramme de Gantt du projet 19
Figure 6 : Architecture logicielle actuelle de l'application GESCOM 22
Figure 7 : Processus de déploiement actuel de GESCOM 24
Figure 8 : Topologie de l'environnement de développement local 26
Figure 9 : Topologie globale de l'infrastructure de l'entreprise 29
Figure 10 : Topologie du cluster Kubernetes 45
Figure 11 : Accès aux paramètres des runners du dépôt GitHub 46
Figure 12 : Instructions d'installation d'un runner auto-hébergé 47
Figure 13 : Enregistrement du runner auprès du dépôt GitHub 48
Figure 14 : Configuration du dossier de travail du runner 48
Figure 15 : Runner démarré et en attente de tâches 49
Figure 16 : Exemple d'exécution d'une tâche réussie par le runner 49
Figure 17 : Exemple d'exécution d'une tâche en échec 50
Figure 18 : Emplacement du fichier de workflow dans le dépôt 50
Figure 19 : Manifests des déploiements des microservices 60
Figure 20 : Manifests des Horizontal Pod Autoscalers 67
Figure 21 : Architecture des microservices déployés dans Kubernetes 69
Figure 22 : Structure du service d'authentification 70
Figure 23 : Structure du service de gestion des produits 75
Figure 24 : Structure du service de gestion des commandes 80
Figure 25 : Onglet Actions du dépôt GitHub 88
Figure 26 : Exécution réussie du workflow CI/CD 88
Figure 27 : Exécution du workflow en échec lors du déploiement 89

Figure 28 : Détail des étapes d'une tâche de déploiement réussie 89
Figure 29 : Journaux d'un déploiement en échec suivi d'un retour arrière automatique . . . 90
Figure 30 : État des nœuds du cluster Kubernetes 91
Figure 31 : Pods système du cluster dans le namespace kube-system 92
Figure 32 : Composants du plan de contrôle 92
Figure 33 : Pods CoreDNS assurant la résolution DNS interne 92
Figure 34 : Collecte des métriques des nœuds et des pods 93
Figure 35 : Namespaces du cluster 94
Figure 36 : Pods des microservices et leur répartition sur les nœuds 94
Figure 37 : Services ClusterIP des microservices 95
Figure 38 : Contrôleur NGINX Ingress et règles d'Ingress du namespace microservices . . 95
Figure 39 : Metrics Server et consommation des ressources des microservices 96
Figure 40 : Horizontal Pod Autoscalers des microservices au repos 96
Figure 41 : Réaction du HPA de auth-service à la montée en charge 97
Figure 42 : Consommation des pods et création d'un nouveau pod de auth-service 97
Figure 43 : Réponses healthz des services avant le changement (version v-3169) 98
Figure 44 : Commit du changement depuis l'éditeur de code 99
Figure 45 : Workflow déclenché par le push, en cours d'exécution 99
Figure 46 : Tâche build : construction de l'image de auth-service en version v-2058 . . . 100
Figure 47 : Tâche test : validation du démarrage du conteneur 100
Figure 48 : Tâche deploy : déploiement progressif et validation des pods 101
Figure 49 : Tâche cypress : résultats des tests de bout en bout 102
Figure 50 : Réponses healthz après le déploiement (auth-service en version v-2058) . . . 102

LISTE DES TABLEAUX

Tableau 1 : Organisation du système de formation pédagogique de l'École 6
Tableau 2 : Mentions et parcours au sein de l'ENI 6
Tableau 3 : Liste des formations existantes à l'ENI 7
Tableau 4 : Débouchés éventuels des jeunes diplômés 9
Tableau 5 : Coordonnées de l'entreprise TETIKA 12
Tableau 6 : Moyens humains nécessaires à la réalisation du projet 17
Tableau 7 : Moyens matériels nécessaires à la réalisation du projet 18
Tableau 8 : Moyens logiciels nécessaires à la réalisation du projet 18
Tableau 9 : Récapitulatif du chronogramme de travail 19
Tableau 10 : Points forts et limites de l'architecture actuelle de GESCOM 23
Tableau 11 : Caractéristiques du poste de développement 26
Tableau 12 : Caractéristiques du routeur d'accès à Internet 26
Tableau 13 : Répartition des serveurs selon leur rôle 28
Tableau 14 : Comparaison des gestionnaires de versions 32
Tableau 15 : Comparaison des solutions de conteneurisation 33
Tableau 16 : Comparaison des plateformes de CI/CD 35
Tableau 17 : Comparaison des solutions de virtualisation 38
Tableau 18 : Comparaison des plateformes d'orchestration de conteneurs 39
Tableau 19 : Types de runners GitHub utilisés dans le projet 45
Tableau 20 : Rôle des ressources Deployment et Service dans le projet 60
Tableau 21 : Services de l'application déployée 69

LISTE DES ABRÉVIATIONS

API : Application Programming Interface

ASI : Audit des Systèmes d'Information

ASR : Administration des Systèmes et Réseaux

AUF : Agence Universitaire de la Francophonie

AWS : Amazon Web Services

BDD : Base De Données

BPO : Business Process Outsourcing

CD : Continuous Deployment (Déploiement Continu)

CI : Continuous Integration (Intégration Continue)

CLI : Command Line Interface

CNH : Commission Nationale d'Habilitation

CNI : Container Network Interface

CPU : Central Processing Unit

CRM : Customer Relationship Management

CVS : Concurrent Versions System

DAF : Direction Administrative et Financière

DevOps : Development and Operations

DevSecOps : Development, Security and Operations

DG : Direction Générale

DNS : Domain Name System

E2E : End-to-End (test de bout en bout)

ENI : École Nationale d'Informatique

GB : Génie Logiciel et Base de Données

GCP : Google Cloud Platform

GHCR : GitHub Container Registry

GID : Gouvernance et Ingénierie de Données

HPA : Horizontal Pod Autoscaler

HTTP : HyperText Transfer Protocol

HTTPS : HyperText Transfer Protocol Secure

IA : Intelligence Artificielle

IG : Informatique Générale

IoT : Internet of Things

IP : Internet Protocol

JSON : JavaScript Object Notation

JWT : JSON Web Token

K8s : Kubernetes

LAN : Local Area Network

LMD : Licence – Master – Doctorat

LTE : Long Term Evolution

LTS : Long Term Support

MDi : Métiers du Digital

MEP : Mise En Production

MVC : Modèle – Vue – Contrôleur

OCC : Objets Connectés et Cybersécurité

OS : Operating System (Système d'exploitation)

PC : Personal Computer

PHP : Hypertext Preprocessor

PME : Petite et Moyenne Entreprise

POC : Proof Of Concept

RAM : Random Access Memory

REST : Representational State Transfer

RH : Ressources Humaines

RSSI : Responsable de la Sécurité des Systèmes d'Information

SGBD : Système de Gestion de Base de Données

SI : Système d'Information

SOC : Security Operations Center

SQL : Structured Query Language

SSH : Secure Shell

SSII : Société de Services en Ingénierie Informatique

SVN : Subversion

TIC : Technologies de l'Information et de la Communication

TPE : Très Petite Entreprise

UPST : Université Paul Sabatier de Toulouse

VHOST : Virtual Host

VM : Virtual Machine (machine virtuelle)

VPN : Virtual Private Network

WAN : Wide Area Network

YAML : YAML Ain't Markup Language

INTRODUCTION GÉNÉRALE

L’évolution rapide des technologies de l’information a profondément modifié la manière de concevoir, développer et déployer les applications. Face à la complexité croissante des systèmes monolithiques, les entreprises recherchent aujourd’hui des solutions leur permettant d’assurer rapidité, fiabilité et continuité dans le cycle de vie logiciel. L’architecture microservices, associée aux pratiques DevOps, répond à ces besoins en favorisant la modularité, l’automatisation et la flexibilité du développement. Cependant, la gestion de plusieurs services indépendants peut s’avérer complexe.

Ce projet a pour but de concevoir et de mettre en œuvre une infrastructure d’intégration et de déploiement continus (CI/CD) spécifiquement adaptée à une application reposant sur une architecture microservices. Il vise à automatiser et à optimiser le processus de livraison logicielle tout en garantissant la fiabilité et la cohérence des déploiements. En intégrant un mécanisme d’orchestration des services, cette infrastructure permettra également d’assurer une meilleure gestion des ressources, une scalabilité accrue et une haute disponibilité de l’application dans son ensemble.

Dès lors, comment automatiser de manière fiable l’intégration, le test et le déploiement d’une application composée de plusieurs services indépendants, tout en garantissant sa disponibilité et sa capacité à s’adapter à la charge ?

Le présent mémoire est structuré en trois grandes parties. La première partie introduit le cadre général du travail en présentant l’École Nationale d’Informatique, l’entreprise d’accueil et la description du projet. La deuxième partie traite de l’analyse de l’existant, de la problématique identifiée et du choix des technologies et outils retenus pour la mise en œuvre du pipeline CI/CD et de l’orchestration. Enfin, la troisième partie expose les différentes étapes de réalisation du projet, de l’installation et de la configuration des outils jusqu’à la vérification des déploiements et des résultats obtenus. Une conclusion générale récapitule les principaux acquis de ce travail et ouvre des perspectives d’amélioration.

PARTIE I. PRÉSENTATIONS

Chapitre 1. Présentation de l'ENI

L'École Nationale d'Informatique (ENI) est reconnue comme un établissement de référence à Madagascar dans le domaine de la formation en informatique.

1.1. Information d'ordre général

L'École Nationale d'Informatique, en abrégé ENI, est un établissement d'enseignement supérieur rattaché académiquement et administrativement à l'Université de Fianarantsoa. Le siège de l'École se trouve à Tanambao-Antaninarenina à Fianarantsoa. L'adresse pour la prise de contact avec l'École est la suivante : École Nationale d'Informatique (ENI) Tanambao, Fianarantsoa. Le numéro de sa boîte postale est 1487 avec le code postal 301. Téléphone : 038 96 566 96 ou 034 76 591 95. Son adresse électronique est la suivante : **eni@eni.mg**. Elle dispose également d'un site web : **www.eni.mg**.

1.2. Missions et historique

L'ENI se positionne sur l'échiquier socio-éducatif malgache comme étant le plus puissant secteur de diffusion et de vulgarisation des connaissances et des technologies informatiques. Cette École Supérieure peut être considérée aujourd'hui comme la vitrine et la pépinière des élites informaticiennes du pays.

De façon formelle, l'ENI a été créée par le décret N° 83-185 du 24 mai 1983, comme étant le seul établissement universitaire professionnalisé au niveau national, destiné à former des techniciens et des ingénieurs de haut niveau, aptes à répondre aux besoins et exigences d'informatisation des entreprises, des sociétés et des organes implantés à Madagascar.

L'ENI a par conséquent pour mission de former des spécialistes informaticiens compétents et opérationnels de différents niveaux, notamment :

- en fournissant à des étudiants des connaissances de base en informatique ;
- en leur transmettant le savoir-faire requis, à travers la professionnalisation des formations dispensées et en essayant d'assurer une meilleure adéquation des formations par rapport aux besoins évolutifs des sociétés et des entreprises ;
- en initiant les étudiants aux activités de recherche dans les différents domaines des Technologies de l'Information et de la Communication (TIC).

La filière de formation d'Analystes Programmeurs a été mise en place à l'École en 1983, et a été gelée par la suite en 1996, tandis que la filière de formation d'ingénieurs a été ouverte à l'École en 1986.

Une formation de troisième cycle a été ouverte à l'École depuis l'année 2003-2004 grâce à la coopération académique et scientifique entre l'Université de Fianarantsoa, pour le

compte de l'ENI, et l'Université Paul Sabatier de Toulouse (UPST). Cette filière avait pour objectif de former certains étudiants à la recherche dans les différents domaines de l'informatique, et notamment de préparer la relève des enseignants-chercheurs qui étaient en poste.

Pendant l'année 2007-2008, la formation en vue de l'obtention du diplôme de Licence Professionnelle en Informatique a été mise en place à l'ENI avec les deux parcours de formation :

- Génie Logiciel et Base de Données ;
- Administration des Systèmes et Réseaux.

La mise en place à l'École de ces deux options de formation devait répondre au besoin de basculement vers le système Licence – Master – Doctorat (LMD).

En vue de surmonter les difficultés de limitation de l'effectif des étudiants accueillis à l'École, notamment à cause du manque d'infrastructures, un système de « Formation Hybride » a été mis en place à partir de l'année 2010. Il s'agit en effet d'un système de formation semi-présentielle et à distance avec l'utilisation de la visioconférence pour la formation à distance. Le système de formation hybride a ainsi été créé à Fianarantsoa ainsi qu'à l'Université de Toliara. Cette formation est à l'origine du parcours Informatique Générale.

En 2023, la mention Intelligence Artificielle (IA) a été ouverte au sein de l'École pour répondre aux besoins des entreprises. La formation est destinée aux étudiants titulaires du diplôme de licence (Bac +3) en mathématiques, en statistiques ou en informatique. La mention IA comporte deux parcours :

- Gouvernance et Ingénierie de Données (GID) ;
- Objets Connectés et Cybersécurité (OCC).

Le parcours GID offre aux étudiants des compétences scientifiques et techniques spécialisées en science des données. Le parcours OCC confère aux étudiants une double spécialité, d'abord en internet des objets, ensuite en cybersécurité. La formation de master est axée sur l'ensemble des applications de l'intelligence artificielle.

Depuis l'année universitaire 2024-2025, la mention Expertise Digitale est officiellement ouverte au grand public. Cette mention propose deux (2) parcours distincts :

- **Métiers du Digital (MDi)** – disponible aux niveaux Licence et Master. Ce parcours est destiné aux étudiants titulaires du diplôme de Baccalauréat, toutes séries confondues. Il est élaboré pour équiper les étudiants des compétences essentielles dans les domaines du marketing digital, de la communication digitale et de la gestion de projets digitaux.
- **Audit des Systèmes d'Information (ASI)** – proposé au niveau Master uniquement. Ce parcours s'adresse aux étudiants titulaires d'une licence (Bac +3) en

mathématiques, en statistiques ou en informatique. Il est structuré pour approfondir les connaissances théoriques et pratiques nécessaires à l'audit efficace des systèmes d'information.

Le système d'enseignement adopté pour cette mention est un système en ligne. Ainsi, les cours et ressources pédagogiques sont accessibles en permanence via la plateforme numérique Moodle. Par ailleurs, des regroupements en présentiel ou en ligne sont organisés périodiquement, conformément au planning défini par les responsables pédagogiques.

1.3. Organigramme institutionnel

L'organigramme de l'École est inspiré des dispositions du décret N° 83-185 du 24 mai 1983. L'ENI est administrée par un Conseil d'École, et dirigée par un directeur nommé par un décret adopté en Conseil des Ministres. Le Collège des enseignants, regroupant tous les enseignants-chercheurs permanents de l'École, est chargé de résoudre les problèmes liés à l'organisation pédagogique des enseignements. Le Conseil Scientifique propose les orientations pédagogiques et scientifiques de l'établissement, en tenant compte notamment de l'évolution du marché du travail et de l'adéquation des formations dispensées par rapport aux besoins des entreprises.

La figure 1 représente l'organigramme actuel de l'École Nationale d'Informatique.

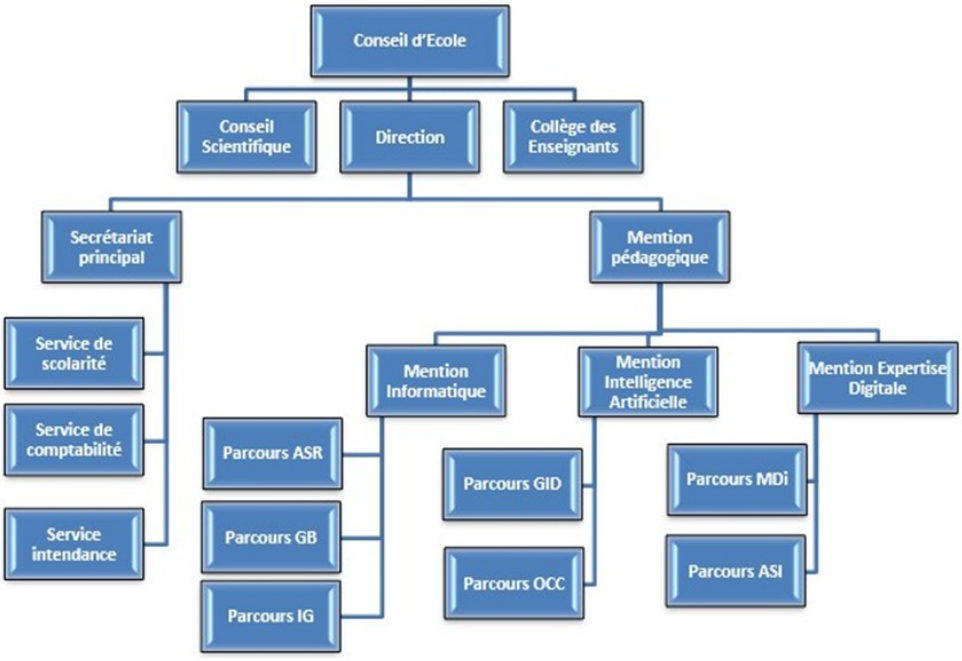


Figure 1 : Organigramme de l'École Nationale d'Informatique

1.4. Domaines de spécialisation

Les activités de formation et de recherche organisées à l'ENI portent sur les domaines suivants :

- Génie Logiciel et Base de Données ;
- Administration des Systèmes et Réseaux ;
- Informatique Générale ;
- Modélisation informatique et mathématique des systèmes complexes ;
- Intelligence artificielle.

Le tableau 1 décrit l'organisation du système de formation pédagogique de l'École.

Tableau 1 : Organisation du système de formation pédagogique de l'École

Formation théorique	Formation pratique
• Enseignement théorique • Travaux dirigés • Travaux pratiques • Conférences	• Études de cas • Travaux de réalisation • Projets / Projets tutorés • Voyages d'études • Stages en entreprise

1.5. Architecture des formations pédagogiques

Le recrutement des étudiants à l'ENI se fait uniquement par voie de concours d'envergure nationale en première année. Les offres de formation organisées à l'École ont été validées par la Commission Nationale d'Habilitation (CNH). Au sein de l'ENI, il existe trois mentions et sept parcours. Le tableau 2 récapitule les mentions et les parcours au sein de l'École.

Tableau 2 : Mentions et parcours au sein de l'ENI

Mention	Parcours
Informatique	Génie Logiciel et Base de Données (GB) Administration des Systèmes et Réseaux (ASR) Informatique Générale (IG)
Intelligence Artificielle	Gouvernance et Ingénierie de Données (GID) Objets Connectés et Cybersécurité (OCC)
Expertise Digitale	Métiers du Digital (MDi) Audit des Systèmes d'Information (ASI)

La figure 2 représente l'architecture des études correspondant au système LMD.

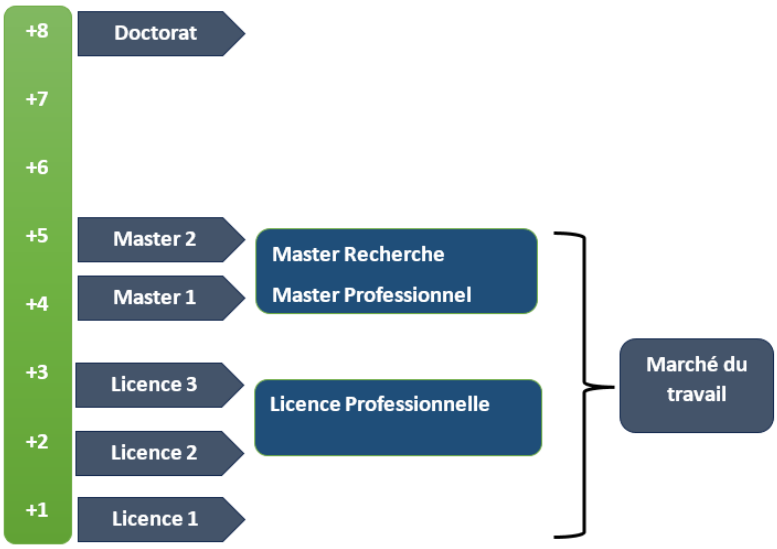


Figure 2 : Architecture des études correspondant au système LMD

La licence peut avoir une vocation générale ou professionnelle. Le master peut avoir une vocation professionnelle ou de recherche. L'accès en première année de master se fait automatiquement pour les étudiants de l'École qui ont obtenu le diplôme de Licence Professionnelle.

Le tableau 3 illustre la liste des formations existantes à l'ENI.

Tableau 3 : Liste des formations existantes à l'ENI

Formation	Licence professionnelle	Master
Condition d'admission	Par voie de concours	Par voie de concours pour la mention IA
Condition d'accès	Bac de série C, D ou Technique	Être titulaire d'une licence professionnelle
Durée de formation	3 ans	2 ans
Diplôme délivré	Diplôme de Licence Professionnelle	Diplôme de Master Professionnel Diplôme de Master Recherche

Le Master Recherche permet à son titulaire de poursuivre directement des études en doctorat et de s'inscrire directement dans une École Doctorale. Les étudiants diplômés de l'École sont plutôt bien accueillis dans les instituts universitaires étrangers (Canada, Suisse, France…).

1.6. Relation de l'ENI avec les organismes externes

Les stages effectués chaque année par les étudiants mettent l'École en rapport permanent avec plus de 400 entreprises et organismes publics, semi-publics et privés, nationaux et internationaux. L'École dispose ainsi d'un réseau d'entreprises, de sociétés et d'organismes publics et privés qui sont des partenaires par l'accueil en stage de ses étudiants, et éventuellement pour le recrutement après l'obtention des diplômes. Les compétences que l'École cherche à développer chez ses étudiants sont l'adaptabilité, le sens de la responsabilité, le travail en équipe, le goût de l'expérimentation et l'innovation.

En effet, la vocation de l'ENI est de former des licenciés et des ingénieurs de niveau master avec des qualités scientifiques, techniques et humaines reconnues, capables d'évoluer professionnellement dans des secteurs d'activité variés intégrant l'informatique. Les stages en milieu professionnel permettent de favoriser une meilleure adéquation entre les formations à l'École et les besoins évolutifs du marché de l'emploi.

Parmi les sociétés, les entreprises et les organismes partenaires de l'École, on peut citer : ACCENTURE Mauritius, AKATA Goavana, Air Madagascar, Ambre Associates, Airtel, Agence Universitaire de la Francophonie (AUF), AXIAN, B2B, Banque Centrale, BIANCO, BlueLine, CNaPS, Bureau National de Gestion des Risques et des Catastrophes (BNGRC), CEDII-Fianarantsoa, Data Consulting, Central Test, Centre National Antiacridien, CNRE, COLAS, Direction Générale des Douanes, DLC, E-Tech Consulting, FID, FIHARY Soft, FTM, GNOSYS, GENIUS AT WORK, Hello Tana, IBONIA, INGENOSIA, INSTAT, IOGA, JIRAMA, JOUVE, MADADEV, MAEP, MANAO, MEF, MEN, MESupRES, MFB, MININTER, Ministère des Postes, des Télécommunications et du Développement Numérique, NEOV MAD, Ny Havana, Madagascar National Parks, OMNITEC, ORANGE, OTME, PRACCESS, QMM Fort-Dauphin, SG Madagasikara, SMMC, SMMEC, SNEDADRS Antsirabe, Sénat, Société d'Exploitation du Port de Toamasina (SEPT), SOFTWELL, Strategy Consulting, TELMA, VIVETEC, Société LAZAN'I BETSILEO, WWF, UGD, ARATO, MNDPT, NG ACADEMY, Relia, Spoon Consulting…

1.7. Débouchés professionnels des diplômés

Les formations proposées par l'École permettent aux diplômés d'être immédiatement opérationnels sur le marché du travail avec la connaissance d'un métier complet lié à l'informatique et aux TIC. L'École apporte à ses étudiants un savoir-faire et un savoir-être qui les accompagnent tout au long de leur vie professionnelle. Elle a une vocation professionnalisante. Les diplômés en licence et en master issus de l'ENI peuvent faire carrière dans différents secteurs.

L'École bénéficie aujourd'hui de plus de 40 années d'expérience pédagogique et de reconnaissance auprès des sociétés, des entreprises et des organismes. C'est une École

Supérieure de référence en matière informatique.

D’une manière générale, les diplômés de l’ENI n’éprouvent pas de difficultés particulières à être recrutés au terme de leurs études. Cependant, l’ENI recommande à ses diplômés de promouvoir l’entrepreneuriat en TIC et de créer des cybercafés, des SSII ou des bureaux d’études. Le tableau 4 représente les débouchés éventuels selon la mention.

Tableau 4 : Débouchés éventuels des jeunes diplômés

Mention	Débouchés professionnels
INFORMATIQUE	**GB :** Développeur logiciel / développeur full-stack, Ingénieur en développement d'applications (desktop, web, mobile), Architecte logiciel, Intégrateur d'applications, Développeur d'API, Administrateur de bases de données (DBA), Ingénieur en conception de bases de données, Développeur SQL / PL-SQL, Spécialiste en migration et optimisation de bases de données, Data engineer, Analyste programmeur, Analyste fonctionnel, Chef de projet informatique, Scrum master / Product owner, Ingénieur QA (Quality Assurance), Testeur logiciel / testeur automatisé, Spécialiste maintenance applicative, Expert en refactoring et optimisation de code, Analyste de données, Concepteur d'outils décisionnels (BI), Ingénieur Big Data, Machine Learning Engineer, Consultant fonctionnel ou freelance. **ASR :** Administrateur systèmes et réseaux, Ingénieur systèmes et réseaux, Administrateur systèmes Linux / Windows, Administrateur cloud (AWS, Azure, Google Cloud), Ingénieur virtualisation (VMware, Hyper-V, Proxmox), Administrateur sécurité, Ingénieur cybersécurité, Analyste SOC, Spécialiste firewall / VPN, RSSI (avec expérience), Technicien support informatique (N2/N3), Technicien réseaux et télécoms, Ingénieur exploitation, Administrateur messagerie, Technicien datacenter, Architecte réseaux, Architecte systèmes, Consultant infrastructure, Chef de projet systèmes et réseaux, Ingénieur DevOps, Administrateur Kubernetes / Docker, Cloud engineer, Spécialiste automatisation (Ansible, Terraform). **IG :** débouchés des parcours GB et ASR à la fois.
INTELLIGENCE ARTIFICIELLE	**GID :** Développeur IA, Data Scientist, Entrepreneur IA, Consultant IA, Responsable marketing IA, Business Developer IA, Product Owner, Consultant en cybersécurité, Directeur de l'innovation. **OCC :** Analyste en cybersécurité, Ingénieur en cybersécurité, Spécialiste en réponse aux incidents, Architecte de sécurité, Consultant et auditeur en cybersécurité, Expert en sécurité des données, Responsable de la sécurité des informations, Chercheur et formateur en cybersécurité, Ingénieur, développeur et architecte IoT, Spécialiste en sécurité IoT, Analyste de données.

Mention	Débouchés professionnels
EXPERTISE DIGITALE	**MDi :** Spécialiste en marketing digital, Gestionnaire de contenu web, Analyste UX/UI, Product Owner, Chef de projet digital, Stratège en médias sociaux, Concepteur-rédacteur web, Analyste de performance digitale, Expert en commerce électronique, Spécialiste SEO/SEM, Community Manager, Conseiller en stratégie digitale, Responsable CRM, Spécialiste en Business Intelligence, Consultant en transformation digitale, Responsable des partenariats digitaux, Directeur de l'innovation digitale, Manager de l'e-réputation. **ASI :** Auditeur des systèmes d'information, Consultant en sécurité des systèmes d'information, Analyste de risques IT, Chef de projet SI, Responsable de la conformité IT, Spécialiste en cyber forensics, Directeur de l'audit et de la sécurité des systèmes d'information, Expert en gouvernance IT, Formateur en sécurité IT, Chercheur en sécurité IT.

1.8. Ressources humaines

Les ressources humaines sont citées ci-dessous selon leurs responsabilités :

- Directeur de l'École : Monsieur MAHATODY Thomas, Professeur ;
- Responsable de la mention « Informatique » : Monsieur RALAIVAO Jean Christian, Maître de Conférences ;
- Responsable de la mention « Intelligence Artificielle » : Monsieur DIMBISOA William Germain, Maître de Conférences.

L'ENI compte douze (12) enseignants permanents dont un (01) Professeur Titulaire, deux (02) Professeurs, sept (07) Maîtres de Conférences et deux (02) Assistants d'Enseignement Supérieur et de Recherche, ainsi que dix (10) enseignants vacataires et quarante et un (41) membres du personnel administratif.

Chapitre 2. Présentation de TETIKA

Ce chapitre présente l'entreprise TETIKA, société d'accueil dans le cadre de ce projet de fin d'études. TETIKA est aujourd'hui la société principale du groupe MANAO.

2.1. Information générale sur la société

La société TETIKA a son siège social à Ankadivory Ambohimiandra, Lot VP 26 Q – Immeuble KINTANA, Antananarivo 101 – Madagascar. Elle dispose également de bureaux secondaires à Toamasina et à Fianarantsoa, ainsi que d'un établissement en France, à Montpellier, assurant le lien avec les partenaires et clients internationaux.

2.1.1. Coordonnées pour la prise de contact avec l'entreprise

Tableau 5 : Coordonnées de l'entreprise TETIKA

Site	Adresse	Téléphone
France (siège du groupe)	14 rue des Anoubles, 34070 Montpellier, France	+33 (0)4 69 66 59 52
Antananarivo (siège social)	Lot VP 26 Q Immeuble KINTANA, Ankadivory Ambohimiandra, Antananarivo 101	+261 (0)32 11 659 00
Fianarantsoa	Lot 232AG/3602 Ankofafa Ambony, Fianarantsoa 301	+261 (0)32 11 659 17 +261 (0)32 11 659 00

2.2. Missions et historique

2.2.1. Historique : de MANAO à TETIKA, « Grandir Ensemble »

Créée en 2008 à Montpellier, l'entreprise MANAO avait pour mission initiale le développement de logiciels de gestion en ligne. En 2013, elle s'est implantée à Antananarivo, puis s'est développée rapidement à Toamasina (2018) et à Fianarantsoa (2019). Au fil du temps, plusieurs entités spécialisées ont été créées :

- **Manao Logiciels :** édition de logiciels pour les entreprises et les établissements scolaires ;
- **TETIKA :** conseil et assistance en gestion ;
- **Ezaka :** externalisation administrative et BPO ;
- **Sidina :** développement web et mobile ;
- **Bambootech :** industrie et exportation de bambou.

En 2025, dans un objectif de simplification et d'efficacité, le groupe MANAO a fusionné ses principales activités sous la marque TETIKA, qui est désormais la société

porteuse de l'ensemble des pôles (conseil, gestion, développement, formation). TETIKA perpétue la philosophie historique de MANAO : « Grandir Ensemble », en valorisant les compétences malgaches dans un contexte international.

2.2.2. Réussir à partir de Madagascar

Le projet du groupe d'entreprises est basé sur la réussite des collaborateurs malgaches. C'est parce que les collaborateurs de l'entreprise réussiront que l'entreprise réussira ; c'est parce que les Malgaches réussissent que le groupe réussit. L'ambition est d'avoir un groupe d'entreprises performantes, qui travaillent à l'international à partir de Madagascar : conquérir le monde à partir de Madagascar.

2.2.3. Valeurs : Confiance, Progression, Responsabilité

Confiance. Les collaborateurs se font mutuellement confiance, dans le travail comme dans le comportement. Aucune triche n'est tolérée : « 0 triche ».

Progression. Chacun a droit à l'erreur et doit s'en servir pour progresser, et faire progresser ses collaborateurs. La performance, l'exigence et l'expertise sont les éléments clés qui permettent de progresser jour après jour. Chacun doit être expert dans son domaine.

Responsabilité. Tout le monde n'est pas chef, mais chaque personne est responsable de ses tâches, de ses missions et des réalisations qu'elle effectue. On agit donc avec intelligence, en analysant le contexte, les situations et les conséquences de ce que l'on réalise. Les collaborateurs ne sont pas de simples exécutants : ils se doivent d'agir et d'être responsables.

2.3. Organigramme fonctionnel et hiérarchique de TETIKA

L'entreprise est sur le point d'entrer dans le stade de Petite et Moyenne Entreprise (PME), aussi l'organigramme commence-t-il à prendre forme. Dans l'optique de coordonner l'ensemble, des directions ont été mises en place, notamment une direction gestion et commerciale, avec à sa tête Monsieur Pascal AMET, et une direction technique et développement, coordonnée par Monsieur Guillaume VIGOUROUX.

L'organigramme, étant l'outil le plus utilisé pour schématiser une structure donnée, permet un aperçu global de l'ensemble. La figure 3 représente celui de TETIKA.

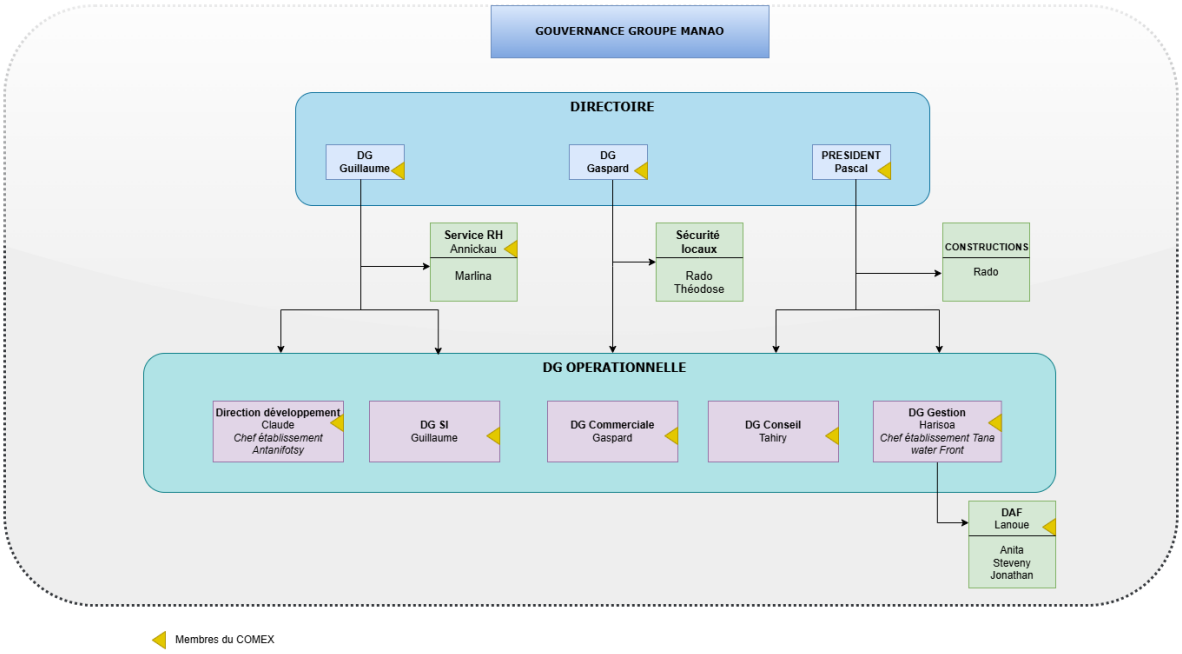


Figure 3 : Organigramme fonctionnel et hiérarchique de l'entreprise TETIKA

Pour illustrer plus en détail la répartition des fonctions au sein de la direction technique et du développement, un organigramme spécifique est présenté dans la figure 4. Il met en évidence les équipes, les rôles et les responsabilités propres à ce service, dont l'équipe des tests automatisés au sein de laquelle s'inscrit ce travail.

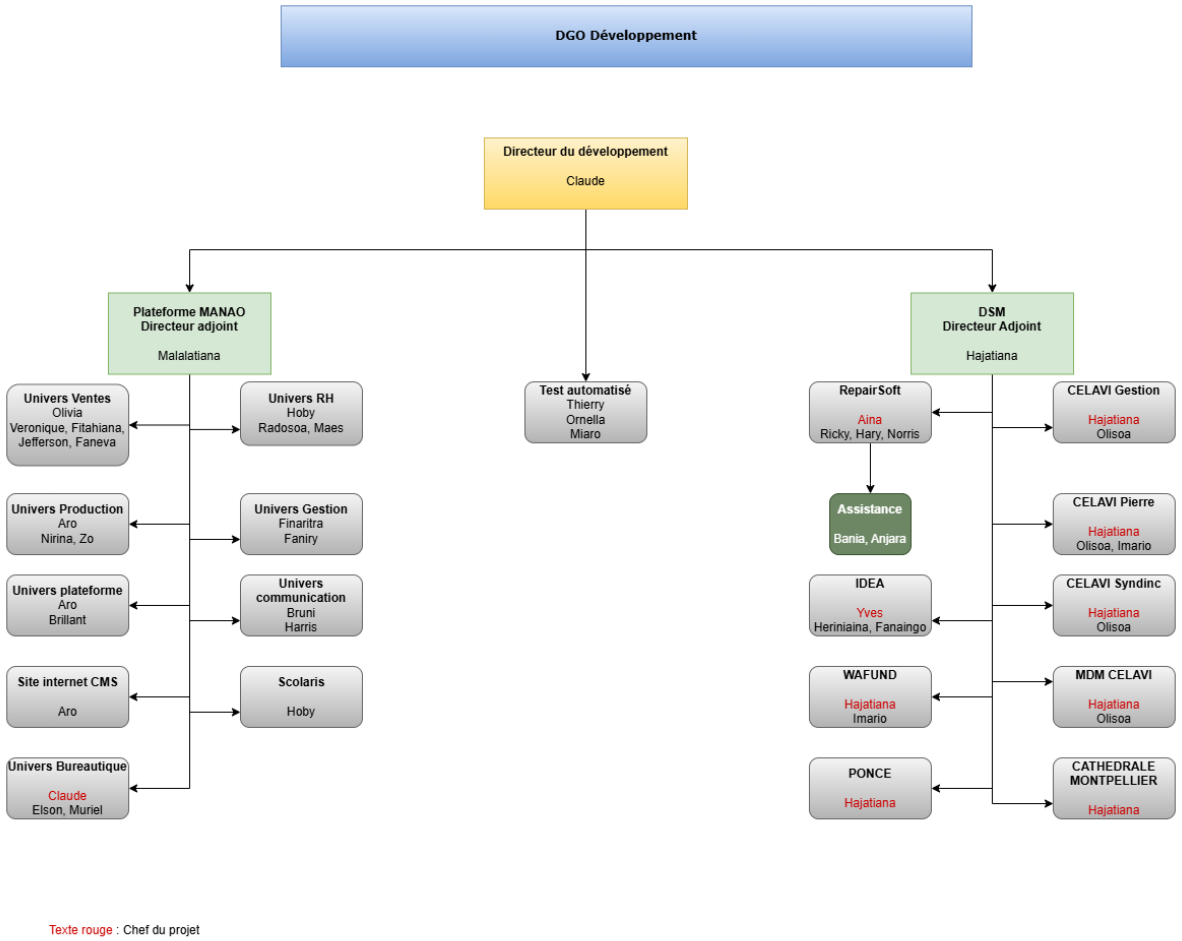


Figure 4 : Organigramme fonctionnel de la direction du développement

2.4. Domaines d'expertise

2.4.1. Compétences métiers

TETIKA dispose de compétences en gestion et en développement informatique, que ce soit auprès de TPE, de PME ou de grands groupes.

En gestion :

- comptabilité ;
- fiscalité ;
- social ;
- organisation en entreprise.

En développement informatique :

- développement d'applications de gestion (logiciels, sites e-commerce) ;
- développement web (PHP, HTML, Python, Java) ;

- développement mobile ;
- compétences sur des domaines connexes : bases de données, infrastructure serveurs, data center, hébergement, télécommunication, assistance, formation.

2.4.2. Branches et activités

Le groupe est divisé en branches, chacune correspondant à une activité :

- Manao Logiciels : www.manao.eu
- Sidina : www.sidina.eu
- TETIKA : www.tetika.eu
- Ezaka : www.ezaka.eu

2.5. Métiers

2.5.1. Métiers productifs

- Les développeurs ;
- les gestionnaires.

2.5.2. Services supports transversaux

- Les gestionnaires (DAF) ;
- la gestion du personnel et le recrutement ;
- les commerciaux ;
- les graphistes ;
- les techniciens d’assistance ;
- les techniciens système et réseau.

Chapitre 3. Description du projet

Ce chapitre décrit le projet afin de comprendre ce qui a été réalisé, pourquoi cette solution a été retenue, quels moyens ont été mobilisés et quels résultats sont attendus.

3.1. Formulation

Le projet porte sur l'étude de la mise en place de l'intégration et du déploiement continus d'une application en architecture microservices, combinée à l'orchestration des services déployés.

3.2. Objectif du projet

Les applications modernes basées sur des architectures microservices nécessitent une coordination efficace des différents services pour maintenir leur fiabilité et leur disponibilité.

Ce projet vise donc à concevoir et à mettre en œuvre un pipeline CI/CD et un mécanisme d'orchestration, afin d'automatiser le processus de livraison des microservices, de sécuriser les déploiements et de garantir une gestion fluide et évolutive des services.

3.3. Moyens nécessaires à la réalisation du projet

La réussite de ce projet repose sur la mise à disposition de ressources adaptées, tant humaines que matérielles et logicielles, permettant de concevoir, déployer et tester efficacement le pipeline CI/CD et l'orchestration des microservices.

3.3.1. Moyens humains

La réalisation de ce projet nécessite l'implication de profils disposant de compétences techniques variées. La coordination et la collaboration entre ces différents acteurs sont primordiales pour assurer la cohérence des déploiements, le respect des bonnes pratiques et la qualité globale du système. Les moyens humains sont présentés dans le tableau 6.

Tableau 6 : Moyens humains nécessaires à la réalisation du projet

Désignation	Rôle
Développeurs	Maîtrisent Node.js et Python ; développent et maintiennent les microservices.
Ingénieur DevOps	Conçoit et maintient le pipeline CI/CD, l'infrastructure et l'orchestration des services.
Encadreurs	Suivent l'évolution du travail, orientent les choix techniques et méthodologiques et valident la conformité du travail réalisé.

3.3.2. Moyens matériels

Le projet requiert un environnement de développement et de déploiement virtualisé. Des serveurs virtuels hébergés sur une machine locale via VirtualBox et Vagrant sont utilisés afin de simuler l'environnement de production. Ces machines virtuelles permettent de tester le pipeline CI/CD, de déployer les microservices et de vérifier leur orchestration dans des conditions proches du réel. Une connexion Internet stable est également indispensable pour accéder aux plateformes de gestion de code, aux images Docker et aux outils distants.

Tableau 7 : Moyens matériels nécessaires à la réalisation du projet

Désignation	Caractéristiques
Machine hôte	Ordinateur capable d'héberger trois machines virtuelles (8 Go de RAM réservés au cluster)
Machines virtuelles	1 nœud master (4 Go de RAM, 2 vCPU) et 2 nœuds workers (2 Go de RAM, 1 vCPU chacun) sous Ubuntu 20.04 LTS
Connexion Internet	Accès stable à GitHub, au registre d'images et aux dépôts de paquets

3.3.3. Moyens logiciels

Plusieurs outils et technologies sont nécessaires afin de concevoir, automatiser et orchestrer efficacement l'ensemble des microservices et du pipeline CI/CD. Ils sont décrits dans le tableau 8.

Tableau 8 : Moyens logiciels nécessaires à la réalisation du projet

Outil	Utilité
Git	Assurer la gestion de versions et faciliter la collaboration entre les équipes
Docker	Conteneuriser les microservices et standardiser les environnements de développement et de déploiement
Kubernetes	Orchestrer les services et garantir la scalabilité et la haute disponibilité de l'application
GitHub Actions	Automatiser le pipeline CI/CD et les processus de build et de déploiement
YAML	Rédiger les scripts d'automatisation du pipeline et les manifests Kubernetes
GraphQL	Gérer la communication et l'échange de données entre les microservices
Vagrant / VirtualBox	Provisionner et exécuter les machines virtuelles du cluster
Cypress	Exécuter les tests automatisés de bout en bout après déploiement

3.4. Résultats attendus

À terme, ce projet devrait permettre d'automatiser l'intégration et le déploiement des microservices via un pipeline CI/CD, garantissant ainsi une livraison rapide, fiable et cohérente des services tout en réduisant les risques d'erreurs liées aux interventions manuelles. Cette automatisation contribuera à améliorer la qualité globale des déploiements et à accélérer le cycle de vie des applications.

Par ailleurs, l'orchestration des services assurera leur scalabilité et leur haute disponibilité, permettant à l'application de fonctionner de manière continue, même en cas de charge importante ou de défaillance d'un service. La conteneurisation et la gestion centralisée des environnements via des outils comme Docker et Kubernetes faciliteront la maintenance, la portabilité et la standardisation des microservices.

3.5. Chronogramme de travail

Le diagramme de Gantt est un outil de planification qui permet de visualiser l'enchaînement des différentes tâches d'un projet. Il montre la durée prévue pour chaque étape et sa position dans le temps, ce qui facilite le suivi de l'avancement et l'organisation du travail. La réalisation de ce projet s'est déroulée sur une période de douze semaines, organisée en plusieurs phases distinctes présentées dans la figure 5.

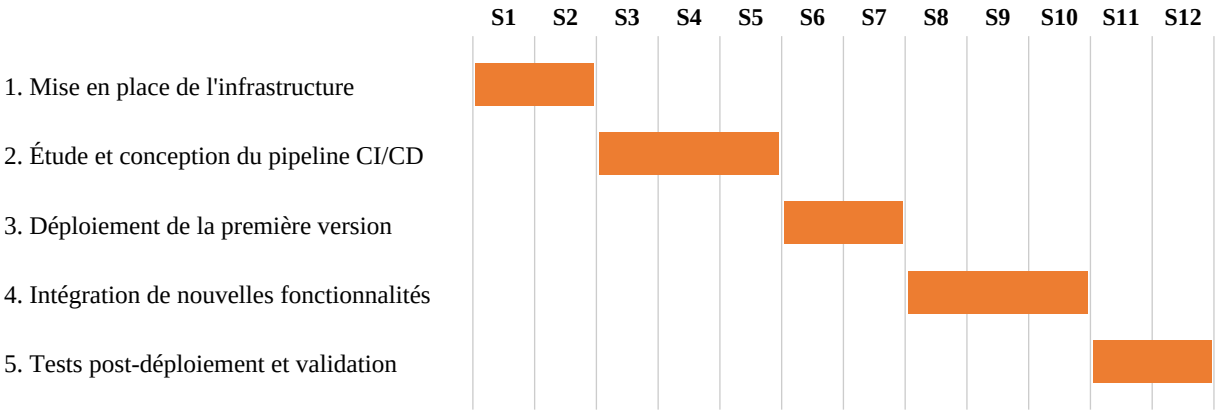


Figure 5 : Diagramme de Gantt du projet

Pour clarifier la figure 5, le tableau 9 propose un récapitulatif des phases et de leur planification.

Tableau 9 : Récapitulatif du chronogramme de travail

Phase	Activités principales	Durée	Semaines
1. Mise en place de l'infrastructure	Installation des serveurs virtuels, configuration des environnements de développement et de déploiement	2 semaines	S1 – S2
2. Étude et conception du pipeline CI/CD	Définition des workflows, scripts d'automatisation, règles d'intégration et de déploiement continus	3 semaines	S3 – S5
3. Déploiement de la première version de l'application	Intégration initiale des microservices, tests de fonctionnement du pipeline et de l'orchestration	2 semaines	S6 – S7
4. Intégration et déploiement de nouvelles fonctionnalités	Déploiement des mises à jour dans les microservices existants, validation de la cohérence et de la stabilité	3 semaines	S8 – S10
5. Tests post-déploiement et validation finale	Vérification de la fiabilité, de la scalabilité et de la performance globale, documentation des résultats	2 semaines	S11 – S12

PARTIE II. ANALYSE PRÉALABLE ET CHOIX DES OUTILS

Chapitre 4. Analyse de l’existant et problématique

Afin de définir une solution adaptée, il est nécessaire de comprendre le fonctionnement actuel de l’application concernée, son processus de déploiement, sa méthodologie de test ainsi que l’infrastructure sur laquelle elle repose.

4.1. Analyse de l’existant

4.1.1. Architecture logicielle : GESCOM

L’application GESCOM (Gestion commerciale) est actuellement développée sous forme monolithique, ce qui signifie qu’un seul bloc applicatif gère l’ensemble des fonctionnalités, sans séparation en modules indépendants. Elle est développée en PHP avec le framework CodeIgniter suivant le modèle MVC. L’application expose également des endpoints REST permettant une interaction avec d’autres systèmes, mais la structure monolithique limite la flexibilité de cette intégration. La figure 6 présente cette architecture.

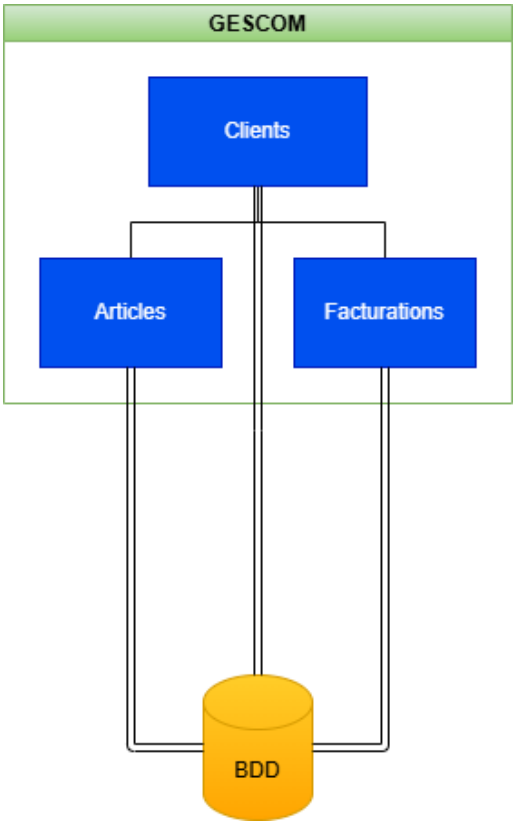


Figure 6 : Architecture logicielle actuelle de l’application GESCOM

La base de données associée à l’application est unique et contient plus de 200 tables. Ces tables sont fortement couplées aux différentes fonctionnalités de l’application, ce qui rend toute modification ou évolution complexe. Les relations multiples entre les tables augmentent la dépendance entre les modules et compliquent la maintenance.

Les fonctionnalités principales de l'application incluent la gestion des clients, la gestion des articles et la gestion des facturations. Ces modules sont intégrés dans le même monolithe, ce qui centralise toutes les opérations mais limite la modularité et la flexibilité pour faire évoluer l'application ou la mettre à l'échelle. Le tableau 10 synthétise les points forts et les limites de cette architecture.

Tableau 10 : Points forts et limites de l'architecture actuelle de GESCOM

Points forts	Limites / contraintes
Solution centralisée avec toutes les fonctionnalités dans la même application	Faible modularité : difficile de déployer ou de faire évoluer une seule fonctionnalité
Utilisation d'un framework connu et documenté (CodeIgniter)	Maintenance difficile en raison du couplage élevé et de la taille de la base de données
Les endpoints REST permettent une intégration externe	Scalabilité limitée : il est nécessaire de mettre à l'échelle l'ensemble du monolithe, et non seulement certains modules
	Testabilité réduite : difficile de tester les fonctionnalités de manière isolée
	Risque de régression élevé lors des modifications
	Évolutivité technologique limitée : migration vers des architectures modernes plus complexe

4.1.2. Processus de déploiement

Le déploiement de l'application GESCOM se fait à partir d'un serveur de production sur lequel un hôte virtuel (VHOST) est configuré pour héberger l'application. L'environnement de développement est local, sur le poste du développeur, et suit le principe Git Flow. Chaque nouvelle fonctionnalité ou correction est développée dans une branche *feature*, testée localement, puis fusionnée dans la branche *develop*.

Une fois les fonctionnalités consolidées et validées, la branche *develop* fait l'objet d'une fusion et d'un *finish* pour créer une *release*, qui est ensuite fusionnée dans la branche *master*. Le serveur de production récupère ensuite la dernière version stable en effectuant un `git pull` sur la branche *master*. Après ce *pull*, l'application est immédiatement opérationnelle sur le serveur de production, grâce au VHOST déjà configuré. La figure 7 illustre ce processus.

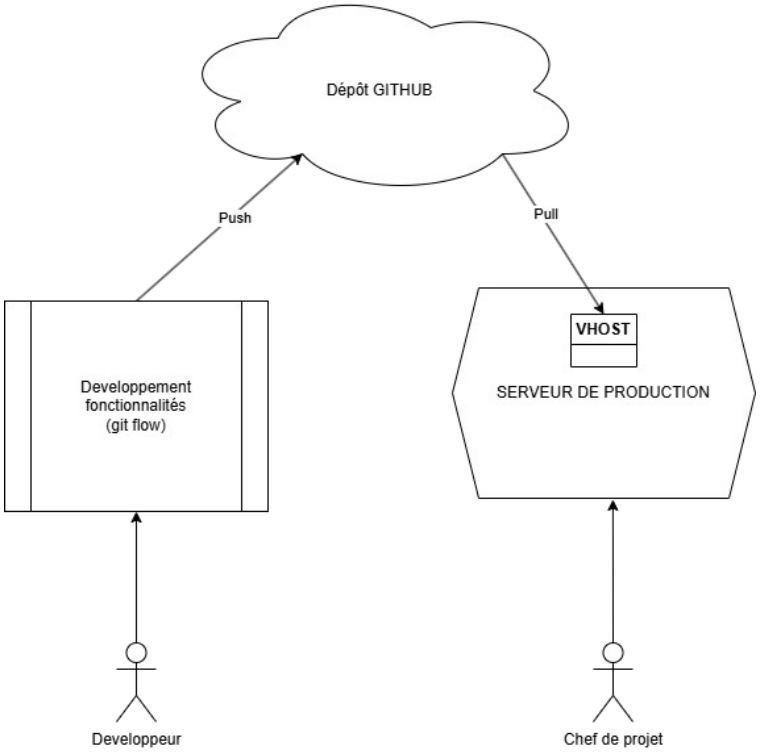


Figure 7 : Processus de déploiement actuel de GESCOM

Ce processus est simple et direct, mais présente certaines limites : il repose entièrement sur un déploiement manuel, la mise à jour du serveur de production est sensible aux erreurs humaines, et il n'existe pas de mécanisme automatique de retour arrière (*rollback*) en cas de problème. La scalabilité et la gestion simultanée de plusieurs environnements restent également limitées.

4.1.3. Méthodologie de test

La méthodologie de test appliquée dans le cadre de l'application GESCOM repose principalement sur des tests manuels effectués après chaque mise en production (MEP). À chaque nouveau déploiement, seules les nouvelles fonctionnalités introduites ou modifiées sont testées manuellement afin de vérifier leur bon fonctionnement dans l'environnement de production.

Les autres fonctionnalités existantes ne sont pas systématiquement retestées, sauf en cas de bug signalé ou d'anomalie constatée par les utilisateurs. Cette méthodologie présente les limites suivantes :

- absence de tests de non-régression sur les fonctionnalités existantes ;
- détection tardive des anomalies, souvent remontées par les utilisateurs eux-mêmes ;
- tests effectués directement en production, après la mise à disposition de la nouvelle version ;

- qualité des tests dépendante de la disponibilité et de la rigueur des personnes qui les réalisent ;
- absence de traçabilité des tests réalisés et de leurs résultats.

4.1.4. Gestion des régressions

Lorsqu'une défaillance est détectée après un déploiement, le problème est remonté à l'équipe de développement. Le correctif est ensuite appliqué localement, puis redéployé en production après validation.

En cas d'échec majeur ou d'instabilité après la mise en production, il est possible de revenir manuellement à la dernière version stable qui fonctionnait correctement. Cette restauration est suivie d'une nouvelle phase de correction et de test avant de relancer le déploiement. Dans certains cas urgents, une branche *hotfix* est créée pour corriger rapidement le problème directement sur la branche *master*, avant de propager les changements dans les autres branches Git.

Les limites de cette méthodologie sont les suivantes :

- processus de correction et de redéploiement lent et manuel ;
- risque d'erreur humaine lors du retour arrière ou du redéploiement ;
- gestion complexe des branches après un *hotfix* (risque de désynchronisation entre *master* et *develop*) ;
- aucune validation automatique des correctifs avant la mise en production ;
- possibilité d'introduire de nouveaux bugs par précipitation.

4.1.5. Topologie de l'infrastructure

a) Environnement de développement local

La topologie de l'infrastructure actuellement utilisée pour le développement de l'application GESCOM est très simple et repose sur un environnement local. Le développeur travaille depuis un ordinateur (PC de développement) sur lequel sont installés l'environnement PHP, le framework CodeIgniter et les outils nécessaires au développement.

Ce poste est connecté à un routeur qui assure l'accès à Internet, notamment pour le téléchargement des dépendances, la gestion du dépôt Git et la synchronisation des modifications avec le serveur distant. Les développements, tests unitaires et validations se font localement avant la mise à jour du dépôt Git. La figure 8 représente cet environnement.

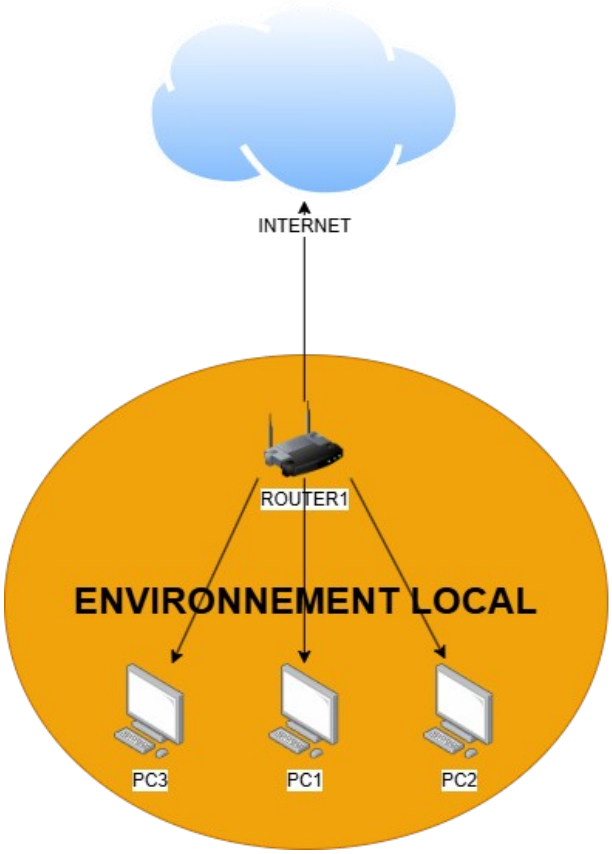


Figure 8 : Topologie de l'environnement de développement local

Les caractéristiques du poste de développement et du routeur sont présentées respectivement dans les tableaux 11 et 12.

Tableau 11 : Caractéristiques du poste de développement

Élément	Spécifications techniques
Type de poste	PC de développement
Marque / Modèle	Dell
Processeur (CPU)	Intel Core i5 9e génération – 2,90 GHz
Mémoire vive (RAM)	8 Go
Système d'exploitation	Windows 10
Connexion réseau	Routeur pour l'accès à Internet

Tableau 12 : Caractéristiques du routeur d'accès à Internet

Catégorie	Détails
Modèle	Huawei B612-233 (Flybox 4G+)
Type de réseau	4G LTE Catégorie 6 (descendant jusqu'à ~300 Mbit/s, montant jusqu'à ~50 Mbit/s)
Fréquences LTE supportées	Bandes 4G : B1/B3/B7/B8/B20/B38/B40/B41/B42/B43 selon la variante
Wi-Fi	Standard 802.11 b/g/n, bande 2,4 GHz, jusqu'à ~32 appareils connectés
Ports Ethernet	4 × Gigabit RJ45 (3 LAN + 1 LAN/WAN)
Port téléphonique	1 × RJ11 pour ligne fixe / VoIP selon le modèle
Emplacement SIM	Micro-SIM
Alimentation	DC 12 V / 1 A (AC 100-240 V via adaptateur)
Dimensions / Poids	Environ 155 mm × 240 mm × 78 mm / ~430 g (hors adaptateur)

b) Topologie globale et serveurs

L'infrastructure réseau de l'entreprise se compose de deux parties principales : le réseau local (LAN) et le réseau hébergé dans le cloud.

Le réseau local est constitué de plusieurs bureaux répartis sur différents sites, non interconnectés entre eux. Chaque bureau dispose de sa propre connexion Internet, fournie soit par une Flybox Orange, soit par une connexion Starlink, selon la couverture réseau disponible. À l'intérieur de chaque site, les ordinateurs et imprimantes sont reliés à un commutateur (*switch*) qui assure l'interconnexion des équipements et leur liaison avec la passerelle Internet. Cette configuration permet de garantir une connectivité stable et fiable pour l'ensemble des utilisateurs, y compris dans les zones à faible couverture réseau.

c) Réseau dans le cloud

Les serveurs de l'entreprise sont hébergés chez plusieurs fournisseurs de services situés en France (OVH, SoYouStart, Ikoula, IONOS). Ils sont organisés en six groupes principaux :

- **Serveurs applicatifs :** ils hébergent les applications internes et métiers utilisées par l'entreprise ;
- **Serveurs clients :** ils assurent l'hébergement des sites web et applications destinés aux clients ;
- **Serveurs de sauvegarde :** ils stockent les copies de sauvegarde des données des applications et des clients ;

- **Serveur de fichiers :** il facilite le partage et la synchronisation des fichiers entre les différentes équipes et serveurs ;
- **Serveurs de secours :** ils répliquent les applications critiques et prennent le relais en cas de défaillance des serveurs principaux ;
- **Serveurs de répartition de charge (*load balancing*) :** ils optimisent la distribution du trafic et garantissent la disponibilité continue des services.

Cette architecture garantit une haute disponibilité, une sécurité renforcée, une continuité de service et une gestion centralisée des applications et données critiques.

d) Types de réseaux et connexions Internet

L'entreprise utilise deux types de réseaux : le réseau local (LAN) et le réseau étendu (WAN).

Au sein du site principal, les employés utilisent un réseau filaire d'une capacité de 100 Mbit/s. Les connexions sont établies à l'aide de câbles RJ45 de catégorie 5, reliant les ordinateurs, les routeurs et les commutateurs. Le câblage est de type droit pour les connexions entre les postes de travail et le commutateur, et croisé pour les liaisons directes entre deux machines.

Chaque bureau dispose de sa propre passerelle Internet, assurée soit par une Flybox Orange, soit par une connexion Starlink, garantissant une redondance et une stabilité du service même en cas de coupure d'un fournisseur.

Le réseau étendu (WAN) connecte les serveurs hébergés en France aux bureaux locaux via une connexion VPN sécurisée, permettant aux équipes d'accéder en toute sécurité aux environnements de préproduction et de production dans le cloud.

e) Serveurs

L'entreprise s'appuie sur une infrastructure de serveurs performante et redondante pour héberger et gérer ses différentes plateformes applicatives. Ces serveurs sont principalement hébergés dans les datacenters de SoYouStart (OVH Cloud), réputés pour leur fiabilité et leurs performances.

À la suite d'un incident majeur survenu dans un datacenter d'OVH en mars 2021, l'entreprise a renforcé sa politique de sauvegarde et de continuité d'activité. Les sauvegardes des données sont désormais répliquées dans les datacenters d'Ikoula, tandis que des serveurs de secours sont hébergés chez IONOS afin d'assurer une reprise rapide des services en cas de panne. L'infrastructure actuelle compte 40 serveurs, répartis selon les rôles présentés dans le tableau 13.

Tableau 13 : Répartition des serveurs selon leur rôle

Rôle du serveur	Nombre
Plateforme principale	16
Plateforme secondaire	1
Site web institutionnel	1
Atelier de développement logiciel	2
GitLab et gestion des sources	1
Applications clients	8
Répartition de charge (*load balancer*)	2
Sauvegardes et réplications	6
Serveurs de secours	3

La figure 9 donne une vue d'ensemble de cette topologie.

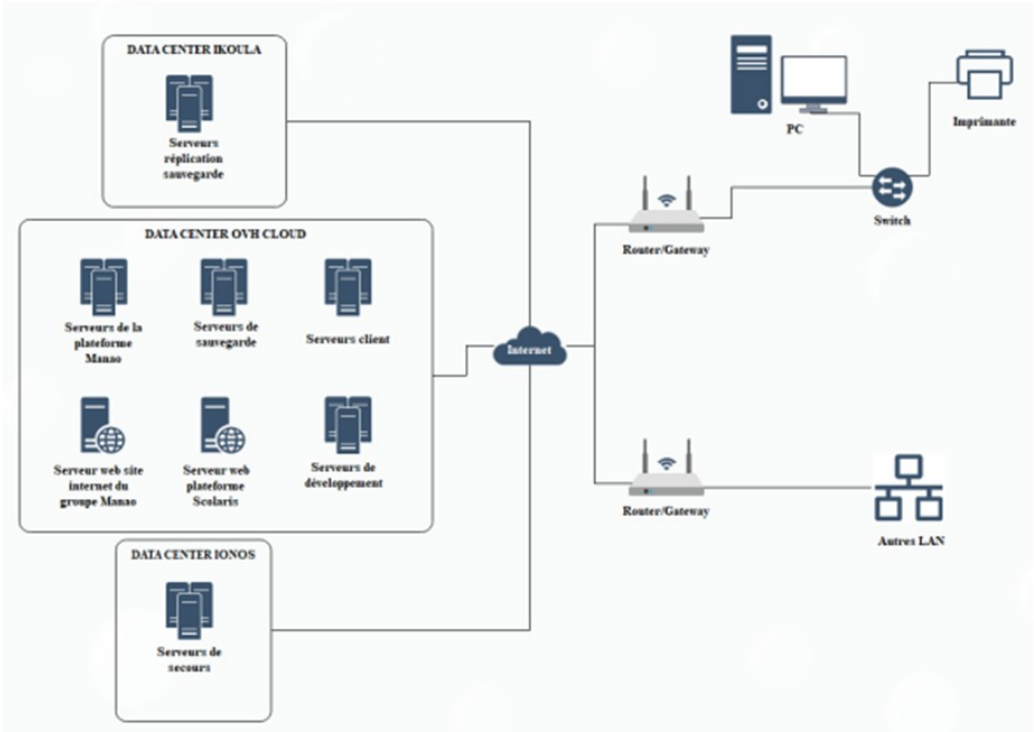


Figure 9 : Topologie globale de l'infrastructure de l'entreprise

4.2. Problématique

L’application GESCOM, telle qu’elle existe aujourd’hui, est construite selon une architecture monolithique, avec une base de données fortement couplée et le framework PHP CodeIgniter. Son cycle de développement reste dépendant d’un poste local, les déploiements sont manuels via des *pull* de la branche *master*, et les tests sont majoritairement effectués manuellement sur les nouvelles fonctionnalités, laissant un risque élevé pour les fonctionnalités existantes.

Cette organisation limite la flexibilité, la scalabilité et la rapidité des mises à jour, tout en rendant la maintenance complexe et sensible aux erreurs humaines.

Dans ce contexte, la problématique est la suivante : **comment concevoir et mettre en œuvre une architecture moderne, modulaire et indépendante des contraintes du monolithe, qui permette de fiabiliser le cycle de développement, de déploiement et de test, tout en assurant la continuité et la qualité des mises à jour d’une application de gestion commerciale ?**

4.3. Proposition de solution

Pour dépasser les limites du monolithe, il est proposé de migrer vers une architecture modulaire, où chaque fonctionnalité majeure devient un service autonome disposant de sa propre base de données. Cette approche réduit le couplage, facilite la maintenance et limite les risques de régression. L’architecture microservices permet une organisation du développement orientée composants, où chaque équipe est responsable de l’intégralité du cycle de vie de son service et peut choisir le langage de programmation adapté à celui-ci.

Un pipeline CI/CD automatisé assurera la construction, l’exécution des tests et le déploiement de chaque service, garantissant que seules les fonctionnalités validées sont mises en production. Les tests automatisés, notamment fonctionnels et de bout en bout avec Cypress, permettront de couvrir à la fois les nouvelles fonctionnalités et les fonctions critiques existantes, améliorant ainsi la qualité globale.

L’infrastructure sera conteneurisée avec Docker et orchestrée via Kubernetes, garantissant la portabilité, la scalabilité et la tolérance aux pannes. L’objectif est de rendre le cycle de vie de l’application plus modulaire, fiable et automatisé, et d’assurer la continuité des services ainsi que la qualité des mises à jour.

Chapitre 5. Études théoriques et choix des outils

Afin de mettre en œuvre une architecture modulaire et automatisée, il est nécessaire de choisir des outils adaptés à chaque étape du cycle de développement, de déploiement et de test. Ce chapitre présente les notions fondamentales du projet, les outils envisagés, leur comparaison et les choix retenus.

5.1. Notions de microservices

L'architecture microservices constitue aujourd'hui l'un des paradigmes les plus utilisés pour la conception d'applications modernes, flexibles et évolutives [1][6]. Contrairement à l'architecture monolithique, où toutes les fonctionnalités d'une application sont regroupées au sein d'un seul bloc, le modèle microservices repose sur la décomposition du système en plusieurs services indépendants, chacun ayant une responsabilité spécifique et communiquant généralement via des API légères, souvent en REST ou en GraphQL.

Chaque microservice peut être développé, testé, déployé et mis à jour de manière autonome, ce qui favorise l'agilité et la rapidité des cycles de développement. De plus, les équipes peuvent choisir des langages, frameworks ou bases de données adaptés aux besoins spécifiques de chaque service, sans dépendre d'une technologie unique. Cette indépendance technologique améliore la maintenabilité et permet une meilleure répartition des charges de travail entre les équipes.

Les microservices offrent également une grande résilience : en cas de défaillance d'un service, le reste de l'application continue de fonctionner, ce qui renforce la fiabilité globale du système. Ils facilitent aussi la scalabilité horizontale : il est possible d'augmenter les ressources uniquement pour les services les plus sollicités, sans impacter l'ensemble de l'application.

Cependant, cette approche introduit une certaine complexité, notamment dans la gestion de la communication entre services, la supervision, la sécurité et le déploiement. C'est pourquoi elle est souvent associée à des outils d'orchestration comme Kubernetes, à des systèmes de CI/CD pour automatiser les mises à jour, et à des outils de supervision pour assurer une visibilité complète sur le comportement des services.

En somme, les microservices favorisent la modularité, la flexibilité et la résilience des applications, tout en exigeant une infrastructure bien pensée et automatisée pour en exploiter pleinement le potentiel.

5.2. Gestionnaire de versions

Un gestionnaire de versions est un outil qui permet de suivre et de gérer les modifications apportées aux fichiers d'un projet, généralement du code source. Le tableau 14 compare les principaux outils existants.

Tableau 14 : Comparaison des gestionnaires de versions

Outil	Type	Avantages	Limites	Cas d'utilisation typique
Git [7]	Distribué	Branches flexibles, travail hors ligne, intégration CI/CD, adoption massive	Courbe d'apprentissage initiale, complexité des commandes avancées	Projets collaboratifs, DevOps, microservices
Mercurial	Distribué	Interface simple, travail hors ligne, versionnage fiable	Moins répandu, communauté plus réduite	Projets moyens ou équipes recherchant la simplicité
Subversion (SVN)	Centralisé	Simple à utiliser, historique et permissions centralisés	Pas de travail hors ligne complet, création de branches et fusion plus difficiles	Projets mono-équipe, environnements d'entreprise traditionnels
CVS	Centralisé	Historique centralisé, simple à configurer	Très ancien, peu flexible, faible support	Projets historiques ou maintenance d'anciens logiciels

Pour ce projet, **Git** est retenu pour sa flexibilité, son intégration naturelle avec les pipelines CI/CD, sa large communauté et sa capacité à gérer efficacement le développement distribué et les microservices [5].

Git est l'un des systèmes de gestion de versions les plus utilisés dans le développement logiciel. Il conserve l'historique complet des modifications, permet de comparer différentes versions et de revenir à un état antérieur si nécessaire. Dans le cadre de ce projet, Git permettra de :

- suivre toutes les modifications du code et des configurations ;
- restaurer une version précédente en cas d'erreur ou de bug critique ;
- faciliter la collaboration entre plusieurs développeurs travaillant sur différents services ;
- intégrer le code avec les outils de CI/CD pour automatiser les tests et le déploiement.

L'utilisation de Git est essentielle pour assurer la traçabilité, la sécurité et la fiabilité du code. Elle permet de réduire les risques de conflits, de gérer efficacement les corrections et les

améliorations, et de conserver un historique complet, indispensable à la maintenance et à l'évolution d'une application modulaire. Par exemple :

- un développeur peut corriger un bug dans un service et valider (*commit*) les changements avec un message décrivant la modification ;
- l'équipe peut cloner le dépôt pour travailler localement, puis pousser (*push*) les modifications sur le serveur central pour intégration ;
- Git permet également d'étiqueter (*tag*) des versions stables avant chaque mise en production afin de pouvoir revenir facilement à une version précédente si nécessaire.

5.3. Conteneurisation

La conteneurisation est une technique permettant d'isoler une application et toutes ses dépendances dans un conteneur léger et portable, garantissant qu'elle fonctionne de manière identique quel que soit l'environnement. Docker est la plateforme de conteneurisation la plus utilisée ; elle facilite la création, le déploiement et l'exécution d'applications dans des environnements isolés [8]. Le tableau 15 compare les principales solutions.

Tableau 15 : Comparaison des solutions de conteneurisation

Outil	Type	Avantages	Limites	Cas d'utilisation typique
Docker [8]	Conteneurisation légère	Isolation des applications, portabilité, images reproductibles, large écosystème, intégration CI/CD	Consommation mémoire si trop de conteneurs, sécurité dépendante du noyau hôte	Microservices, CI/CD, tests automatisés, déploiement en cloud ou en local
Podman	Conteneurisation légère sans démon	Pas de démon root, compatible avec la CLI Docker, sécurisé	Communauté moins large, outils d'orchestration moins matures	Environnements sensibles à la sécurité, microservices isolés
LXC/LXD	Conteneurs système	Isolation complète, virtualisation légère, simule un OS entier	Complexité de configuration, moins adapté aux microservices	Virtualisation de serveurs, tests de systèmes complets
rkt (Rocket)	Conteneurisation orientée sécurité	Sécurisé, intégration avec Kubernetes	Moins populaire, fin du support officiel	Déploiements sécurisés, expérimentations avec Kubernetes

Docker est retenu pour ce projet grâce à sa simplicité, sa compatibilité avec Kubernetes, son écosystème riche et sa facilité d'intégration dans les pipelines CI/CD. Il

permettra de construire, tester et déployer rapidement les microservices dans un environnement cohérent et portable. Docker est utilisé pour :

- isoler chaque service de l'application (clients, articles, facturations) afin qu'ils puissent fonctionner indépendamment ;
- assurer la cohérence entre les environnements de développement, de test et de production ;
- simplifier le déploiement et la mise à jour des services sans affecter les autres parties de l'application ;
- faciliter la création d'environnements de test pour valider les correctifs avant la mise en production.

La conteneurisation permet d'améliorer la portabilité, la scalabilité et la résilience des applications. Chaque service peut être démarré, arrêté ou mis à jour indépendamment, ce qui réduit les risques de régression. Docker est également un élément clé de toute infrastructure DevOps, car il s'intègre facilement dans les pipelines CI/CD et dans des systèmes d'orchestration comme Kubernetes. Par exemple :

- un développeur peut créer un conteneur Docker contenant un service de gestion des articles avec toutes ses dépendances et le tester localement sans modifier son environnement principal ;
- lors de la mise en production, ce même conteneur peut être déployé sur le serveur de manière identique, garantissant un comportement prévisible ;
- Docker permet également de lancer simultanément plusieurs conteneurs pour simuler l'intégration de différents services de l'application dans un environnement de test complet.

5.4. Notions d'intégration continue et de déploiement continu

L'intégration continue (CI) et le déploiement continu (CD) sont deux pratiques fondamentales du DevOps, visant à automatiser le cycle de vie des applications pour le rendre plus fiable, rapide et reproductible [2][3].

L'intégration continue consiste à fusionner régulièrement les modifications de code provenant de différents développeurs dans un dépôt centralisé. Chaque modification déclenche automatiquement des tests unitaires, fonctionnels et de conformité, ce qui permet de détecter rapidement les erreurs ou les conflits avant qu'ils n'affectent l'ensemble du système.

Le déploiement continu, quant à lui, permet de mettre automatiquement à disposition les modifications validées dans des environnements de test, de préproduction ou directement de production. Cette pratique garantit que chaque nouvelle fonctionnalité ou correction de bug est rapidement et sûrement disponible pour les utilisateurs finaux, tout en maintenant la

stabilité de l’application.

La CI et le CD sont donc complémentaires : la CI se concentre sur la qualité et la validation du code, tandis que le CD s’assure de sa livraison fiable et rapide. Dans le cadre d’une application modulaire, l’intégration et le déploiement continus permettent de :

- vérifier automatiquement la compatibilité et la stabilité de chaque service indépendamment ;
- détecter immédiatement les régressions et les erreurs après chaque modification, réduisant ainsi les risques pour la production ;
- déployer rapidement de nouvelles fonctionnalités ou des correctifs sur les environnements de test et de préproduction avant leur mise en production ;
- maintenir la cohérence entre les différents environnements et faciliter la collaboration entre plusieurs développeurs travaillant sur des services distincts.

La CI/CD est essentielle pour améliorer la qualité globale du logiciel, réduire les risques liés aux erreurs humaines et accélérer le cycle de développement et de mise en production. Elle favorise également la traçabilité complète des changements, permet de générer des *builds* reproductibles et d’assurer la cohérence entre les environnements locaux, de test et de production. Dans un projet basé sur une architecture modulaire et conteneurisée, ces pratiques sont indispensables pour gérer efficacement les services indépendants, automatiser les tests et orchestrer les déploiements. Par exemple :

- lorsqu’un développeur pousse une modification sur le dépôt Git, un serveur CI/CD déclenche automatiquement la construction et l’exécution des tests du service concerné, garantissant que tout fonctionne avant la fusion ;
- si les tests sont validés, le pipeline déploie automatiquement le service dans un environnement de préproduction ou de production, sans intervention manuelle ;
- en cas d’échec d’un test, le pipeline bloque le déploiement et alerte l’équipe, permettant une correction rapide et évitant que le bug n’atteigne les utilisateurs ;
- les pipelines CI/CD peuvent également générer automatiquement des rapports de test et des artefacts (images Docker, fichiers de configuration) pour assurer la traçabilité et faciliter la maintenance.

Grâce à l’intégration et au déploiement continus, le projet bénéficie d’une agilité accrue, d’une réduction du délai entre le développement et la production, et d’une qualité logicielle améliorée. Le tableau 16 compare les principales plateformes de CI/CD.

Tableau 16 : Comparaison des plateformes de CI/CD

Outil / Plateforme	Type	Avantages	Limites	Cas d'utilisation typique
GitHub Actions [9]	CI/CD intégré à GitHub	Intégré au dépôt, facile à configurer, YAML déclaratif, exécutions automatisées sur *push* et *pull request*	Limité aux dépôts GitHub, ressources gratuites limitées	Microservices, pipelines CI/CD rapides, tests et déploiements automatisés
GitLab CI/CD	CI/CD intégré à GitLab	Intégré au dépôt, *runners* personnalisables, YAML déclaratif	Complexité pour les déploiements externes, courbe d'apprentissage	Projets internes, pipelines complexes, multi-langages
Jenkins	Serveur CI/CD autonome	Très flexible, nombreux plugins, large adoption	Installation et maintenance complexes, nécessite un serveur dédié	Projets de grande envergure, environnements hétérogènes
CircleCI	CI/CD en SaaS	Cloud ou sur site, pipelines rapides, intégration Docker	Limité hors cloud, coût pour un usage intensif	Déploiements cloud, pipelines automatisés pour microservices

Pour ce projet, **GitHub Actions** est retenu grâce à sa facilité d'intégration avec les dépôts GitHub, sa configuration déclarative via YAML et sa capacité à automatiser les tests et les déploiements des microservices. Cette solution permet de construire un pipeline CI/CD complet, garantissant la cohérence et la fiabilité des versions déployées.

5.5. Notions d'orchestration

L'orchestration désigne l'ensemble des mécanismes et outils permettant d'automatiser la gestion, la coordination et la supervision des conteneurs au sein d'une infrastructure distribuée. Elle consiste à planifier le déploiement, la montée en charge, la mise à jour et la tolérance aux pannes des services, tout en garantissant la stabilité et la continuité du système.

Dans une architecture basée sur des microservices, où chaque composant fonctionne dans son propre conteneur, l'orchestration est indispensable pour gérer efficacement les ressources et les communications entre services. Elle intervient à plusieurs niveaux du cycle de vie des applications :

- automatisation du déploiement de conteneurs sur différents environnements (développement, test, production) ;
- gestion de la scalabilité, en ajustant dynamiquement le nombre d'instances en fonction de la charge ;

- supervision de l'état des conteneurs et redémarrage automatique en cas de défaillance pour assurer la haute disponibilité ;
- administration du réseau interne entre les services conteneurisés (attribution d'adresses IP, ouverture de ports, politiques de sécurité) ;
- mise à jour continue des services via des stratégies de déploiement sans interruption (*rolling updates*).

L'orchestration constitue un pilier central d'une infrastructure DevOps moderne. Elle permet d'obtenir un système automatisé, hautement disponible et résilient, en réduisant les interventions manuelles et les erreurs humaines. Grâce à elle, les applications deviennent plus faciles à maintenir, à faire évoluer et à adapter aux besoins en ressources. Elle assure également une gestion centralisée des configurations et des ressources, tout en améliorant la scalabilité horizontale des applications conteneurisées. Par exemple :

- un orchestrateur peut déployer automatiquement plusieurs conteneurs d'une même application sur différents serveurs physiques ou virtuels ;
- en cas d'échec d'un conteneur, celui-ci est redémarré automatiquement sans affecter les autres services ;
- lors du déploiement d'une nouvelle version, l'orchestrateur effectue une mise à jour progressive pour éviter toute interruption ;
- des fichiers de configuration (souvent en YAML) décrivent le comportement de chaque service, rendant les déploiements reproductibles et standardisés.

Ainsi, l'orchestration représente une composante essentielle de la gestion d'applications conteneurisées à grande échelle. Elle garantit non seulement la continuité de service, mais aussi la flexibilité et l'efficacité opérationnelle nécessaires à la mise en œuvre des pratiques DevOps.

5.6. Choix de la solution de virtualisation

La virtualisation est une technologie fondamentale qui consiste à créer une version virtuelle d'une ressource physique telle qu'un serveur, un système d'exploitation, un espace de stockage ou un réseau. Elle permet de faire fonctionner plusieurs environnements isolés sur une même machine physique, favorisant ainsi une utilisation optimale des ressources matérielles.

Son principe repose sur la séparation entre le matériel et les environnements logiciels, rendant possible la création de plusieurs machines virtuelles indépendantes, chacune disposant de son propre système d'exploitation et de ses applications. Cette abstraction simplifie considérablement la gestion, la maintenance et le déploiement des environnements informatiques.

Dans le contexte du DevOps et du développement applicatif moderne, la virtualisation offre de nombreux avantages. Elle permet notamment de reproduire fidèlement les environnements de production, de tester différentes configurations sans perturber l'infrastructure principale et d'assurer une meilleure isolation entre les services. Grâce à cette approche, les équipes de développement et d'exploitation gagnent en flexibilité et en rapidité, tout en réduisant les risques liés aux incompatibilités d'environnement. Le tableau 17 compare les principales solutions.

Tableau 17 : Comparaison des solutions de virtualisation

Outil	Type	Avantages	Limites	Cas d'utilisation typique
VirtualBox [12]	Virtualisation complète	Open source, multiplateforme, simple à configurer, bonne intégration avec Vagrant	Moins performant pour de très gros clusters, consommation mémoire importante	Machines de test, environnements locaux de développement, clusters Kubernetes légers
VMware Workstation / Player	Virtualisation complète	Performances élevées, fonctionnalités avancées, instantanés (*snapshots*)	Payant pour la version complète, moins simple à automatiser	Environnements de test et de production nécessitant robustesse et support
Hyper-V	Virtualisation intégrée à Windows	Intégré à Windows, bonnes performances, instantanés	Limité aux hôtes Windows, moins flexible pour Linux	Tests et développement sur machines Windows
KVM / QEMU	Virtualisation Linux	Très performant, open source, flexible	Complexité de configuration, interface moins conviviale	Déploiement de serveurs Linux, virtualisation à grande échelle

VirtualBox est retenu pour sa simplicité d'utilisation, sa compatibilité multiplateforme et son intégration avec Vagrant [11]. Cette combinaison permet de provisionner rapidement des machines virtuelles identiques et de reproduire un cluster Kubernetes complet.

5.7. Scripts YAML

Les scripts YAML (*YAML Ain't Markup Language*) sont largement utilisés pour décrire des configurations de manière lisible et structurée [18]. Dans le contexte du DevOps, ils jouent un rôle essentiel dans l'automatisation des processus d'intégration et de déploiement continus. Ces fichiers permettent de définir, étape par étape, le comportement d'un pipeline : compilation, tests, génération d'images, déploiement ou encore notifications.

Un exemple concret d'utilisation de YAML se retrouve dans les *workflows* GitHub, au cœur de GitHub Actions. Ce système permet d'automatiser des tâches à chaque événement du cycle de développement, comme un *push*, une *pull request* ou une *release*. Le fichier YAML, placé dans le répertoire `.github/workflows/`, décrit la suite des actions à exécuter dans un ordre précis, les environnements utilisés (Node.js, Python, Docker…) ainsi que les conditions de déclenchement.

Grâce à cette approche déclarative, les équipes peuvent mettre en place des pipelines reproductibles et faciles à maintenir, sans nécessiter de serveur d'intégration dédié. Les *workflows* GitHub s'intègrent naturellement aux dépôts de code, favorisant la transparence, la collaboration et la traçabilité des déploiements. Ainsi, l'usage des scripts YAML constitue un élément clé de l'automatisation DevOps moderne, garantissant la cohérence et la fiabilité des processus de CI/CD.

5.8. Plateformes d'orchestration de conteneurs Docker

L'orchestration de conteneurs est une composante essentielle de la gestion d'applications modernes déployées sous forme de microservices. Elle vise à automatiser la mise en œuvre, la gestion, la mise à l'échelle et la supervision des conteneurs, afin d'assurer une disponibilité continue et une répartition optimale des ressources.

Lorsqu'un projet comporte plusieurs conteneurs interdépendants, la gestion manuelle devient rapidement complexe. Les plateformes d'orchestration permettent alors de définir des stratégies de déploiement, de gérer les dépendances entre services, de surveiller leur état et d'assurer une tolérance aux pannes. Elles prennent également en charge la mise à jour continue (*rolling updates*), la reprise automatique après incident (*self-healing*) et le dimensionnement dynamique selon la charge du système. Le tableau 18 compare les principales plateformes.

Tableau 18 : Comparaison des plateformes d'orchestration de conteneurs

Outil / Plateforme	Type	Avantages	Limites	Cas d'utilisation typique
Kubernetes [10]	Orchestration complète	Scalabilité, tolérance aux pannes, gestion déclarative des ressources, large communauté et écosystème riche	Courbe d'apprentissage élevée, configuration complexe pour les débutants	Microservices, clusters distribués, déploiements en production et en test
Docker Swarm	Orchestration intégrée à Docker	Facile à configurer, intégré à la CLI Docker, adapté aux petites architectures	Fonctionnalités avancées limitées, moins adapté aux grands clusters	Projets de petite à moyenne taille, prototypes ou POC
Apache Mesos / Marathon	Orchestration distribuée	Très scalable, gestion de ressources multi-applications	Complexité de configuration, communauté moins active	Grands environnements multi-services, applications distribuées complexes
Nomad (HashiCorp)	Orchestration légère	Simplicité, multi-cloud, intégration facile avec Consul	Fonctionnalités avancées limitées par rapport à Kubernetes	Petits clusters, charges de travail hétérogènes

Parmi ces solutions, Kubernetes s'impose comme la référence du domaine [4]. Développé initialement par Google, il offre une approche déclarative de la gestion d'infrastructure et prend en charge l'automatisation complète du cycle de vie des conteneurs Docker. Il permet notamment la gestion de clusters distribués, le déploiement évolutif et la surveillance centralisée des services. D'autres outils comme Docker Swarm ou Apache Mesos proposent des fonctionnalités similaires, mais Kubernetes se distingue par sa robustesse, sa communauté active et son écosystème étendu.

Kubernetes est donc retenu pour sa robustesse, sa capacité à gérer un cluster complet de microservices et sa compatibilité avec Docker. Il permet de déployer les services de manière scalable, résiliente et automatisée, tout en offrant un environnement proche de la production pour tester et orchestrer les microservices.

PARTIE III. MISE EN ŒUVRE

Chapitre 6. Installations et configurations

Cette partie présente la phase pratique du projet, où les concepts théoriques et les choix technologiques précédemment étudiés sont concrètement appliqués. Elle détaille la mise en place de l'infrastructure, la configuration des outils DevOps ainsi que le déploiement effectif d'une application conçue selon une architecture microservices. L'objectif est de démontrer comment l'automatisation du cycle de vie applicatif, de l'intégration au déploiement, peut améliorer la fiabilité, la rapidité et la cohérence des livrables.

L'application développée dans ce cadre est une application de commande de produits. Elle permet aux utilisateurs de consulter des produits et d'effectuer des commandes via une interface ou une API. Elle est découpée en trois services indépendants : la gestion des utilisateurs et de l'authentification, la gestion du catalogue de produits et la gestion des commandes, qui communiquent entre eux dans une architecture distribuée.

Ce choix d'application permet de mettre en pratique les principes fondamentaux du DevOps, notamment la conteneurisation, l'orchestration, l'intégration continue et le déploiement automatisé, tout en démontrant la flexibilité et la scalabilité offertes par une architecture découplée.

6.1. Infrastructure

Pour déployer et tester les microservices dans un environnement proche de la production, un cluster Kubernetes local composé d'un nœud *master* et de deux nœuds *workers* est utilisé. Ce cluster sert de base pour orchestrer les conteneurs, gérer la communication entre services et expérimenter les déploiements automatisés.

Pour mettre en place ce cluster, Vagrant est associé à VirtualBox. Vagrant permet d'automatiser la création, la configuration et le provisionnement de machines virtuelles, tandis que VirtualBox fournit la virtualisation nécessaire pour exécuter ces machines sur le poste hôte.

Le cluster est composé de trois nœuds : un *master* et deux *workers*, chacun disposant d'une adresse IP privée sur un réseau isolé. Le *master* orchestre le cluster et gère le plan de contrôle, tandis que les *workers* exécutent les conteneurs des microservices. Chaque machine virtuelle est configurée avec des ressources adaptées : le *master* dispose de 4 Go de RAM et de 2 vCPU, et les *workers* de 2 Go de RAM et d'1 vCPU chacun, garantissant une performance équilibrée pour un environnement local de test. Le fichier `Vagrantfile` utilisé est le suivant :

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/focal64"
  config.vm.boot_timeout = 600

  nodes = {
    "k8s-master"  => "192.168.56.10",
    "k8s-worker1" => "192.168.56.20",
    "k8s-worker2" => "192.168.56.21"
  }

  def provision_node(ip, hostname, is_master, nodes)
    <<-SHELL
      sudo apt-get update -y
      sudo apt-get install -y apt-transport-https ca-certificates curl gnupg lsb-release
          software-properties-common

      # Docker & containerd
      curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
      sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu focal
          stable"
      sudo apt-get update -y
      sudo apt-get install -y docker-ce docker-ce-cli containerd.io
      sudo mkdir -p /etc/containerd
      sudo containerd config default | sudo tee /etc/containerd/config.toml
      sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
      sudo systemctl restart containerd
      sudo systemctl enable containerd

      # Kubernetes repos
      sudo mkdir -p /etc/apt/keyrings
      curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | sudo gpg --dearmor
          -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
      echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
          https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /" | sudo tee
          /etc/apt/sources.list.d/kubernetes.list
      sudo apt-get update -y
      sudo apt-get install -y kubelet kubeadm kubectl
      sudo apt-mark hold kubelet kubeadm kubectl

      # Networking tweaks
      sudo modprobe br_netfilter
      echo "br_netfilter" | sudo tee /etc/modules-load.d/br_netfilter.conf
      echo "net.bridge.bridge-nf-call-iptables = 1" | sudo tee /etc/sysctl.d/99-k8s.conf
      echo "net.ipv4.ip_forward = 1" | sudo tee -a /etc/sysctl.d/99-k8s.conf
      sudo sysctl --system

      # /etc/hosts for all nodes
      #{nodes.map { |n, h| "echo '#{h} #{n}' | sudo tee -a /etc/hosts" }.join("\n")}

      if [ "#{is_master}" = "true" ]; then
        sudo kubeadm init --apiserver-advertise-address=#{ip} --pod-network-cidr=192.168.0.0/16
        mkdir -p /home/vagrant/.kube
        sudo cp -i /etc/kubernetes/admin.conf /home/vagrant/.kube/config
        sudo chown vagrant:vagrant /home/vagrant/.kube/config
        sudo -u vagrant kubectl apply -f https://projectcalico.docs.tigera.io/manifests/calico.y
            aml
        kubeadm token create --print-join-command > /vagrant/join.sh
      else
        while [ ! -f /vagrant/join.sh ]; do sleep 5; done
        sudo bash /vagrant/join.sh
      fi
    SHELL
```

```
  end

  # Master node
  config.vm.define "k8s-master" do |master|
    master.vm.hostname = "k8s-master"
    master.vm.network "private_network", ip: nodes["k8s-master"]
    master.vm.provider "virtualbox" do |vb|
      vb.memory = 4096
      vb.cpus = 2
    end
    master.vm.provision "shell", inline: provision_node(nodes["k8s-master"], "k8s-master",
        true, nodes)
  end

  # Worker1 & Worker2 (2 Go chacun)
  ["k8s-worker1", "k8s-worker2"].each do |worker|
    config.vm.define worker do |w|
      w.vm.hostname = worker
      w.vm.network "private_network", ip: nodes[worker]
      w.vm.provider "virtualbox" do |vb|
        vb.memory = 2048
        vb.cpus = 1
      end
      w.vm.provision "shell", inline: provision_node(nodes[worker], worker, false, nodes)
    end
  end
end
```

Le provisionnement des machines est entièrement automatisé via le script shell intégré dans le `Vagrantfile`. Ce script comprend plusieurs étapes :

- mise à jour du système d'exploitation (Ubuntu 20.04 LTS) et installation des prérequis, comme `curl` et les utilitaires réseau ;
- installation et configuration de Docker et de containerd, nécessaires pour exécuter les conteneurs ;
- installation des composants Kubernetes (`kubeadm`, `kubelet`, `kubectl`) et configuration des paramètres réseau permettant la communication entre nœuds et la gestion des pods ;
- mise à jour du fichier `/etc/hosts` sur chaque machine pour résoudre les noms de tous les nœuds, assurant une communication inter-nœuds fiable ;
- initialisation du *master* via `kubeadm init`, suivie de l'application d'un réseau de pods (Calico [13]) pour gérer le routage et la sécurité des communications ;
- connexion des *workers* au cluster via un script généré automatiquement par le *master* (`join.sh`), garantissant que les nœuds *workers* sont immédiatement opérationnels.

Cette approche permet de disposer d'un environnement Kubernetes complet et reproductible, où chaque machine est configurée de manière cohérente et où les microservices peuvent être déployés, testés et orchestrés de manière fiable. Grâce à Vagrant et VirtualBox, l'infrastructure peut être recréée rapidement sur n'importe quel poste de développement, facilitant la cohérence entre les équipes et la reproductibilité des tests. On obtient ensuite la topologie présentée dans la figure 10 : les trois nœuds sont placés dans un cluster Kubernetes avec Calico, qui permet aux pods de chaque nœud de communiquer entre eux même lorsqu'ils

sont physiquement séparés.

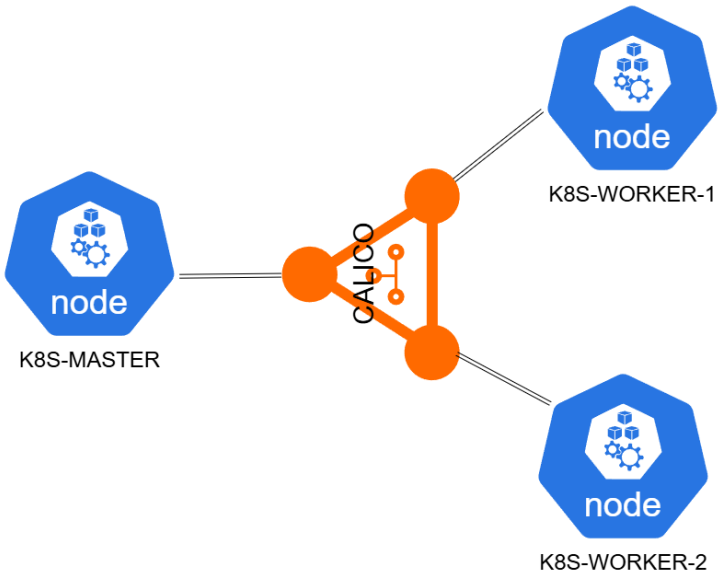


Figure 10 : Topologie du cluster Kubernetes

6.2. Automatisation CI/CD

Pour automatiser l'intégration et le déploiement des microservices, le projet utilise GitHub Actions avec des *runners* dédiés. Cette approche permet de déclencher automatiquement des *workflows* à chaque modification du code, garantissant la qualité, la cohérence et la réactivité des livraisons.

Dans un pipeline CI/CD, les *runners* GitHub jouent un rôle central : ce sont les agents chargés d'exécuter les *workflows* définis dans les fichiers YAML et d'enchaîner les différentes étapes du pipeline, de la compilation jusqu'au déploiement. Le tableau 19 présente les deux types de *runners* utilisés.

Tableau 19 : Types de runners GitHub utilisés dans le projet

Type de runner	Environnement	Avantages	Limites	Utilisation dans le projet
Runners cloud (hébergés par GitHub)	Serveurs GitHub	Maintenance minimale, ressources extensibles, exécution parallèle, rapides pour les *builds* et les tests	Ressources gratuites limitées, dépendance au cloud	Construction des images Docker, exécution des tests automatisés
Runner auto-hébergé (*self-hosted*)	Serveur local ou machine virtuelle	Contrôle total de l'environnement, accès direct au cluster, exécution des tâches nécessitant un accès au réseau interne	Nécessite une maintenance et une configuration manuelles	Déploiement des microservices sur le cluster Kubernetes local (*runner* du nœud *master*)

6.2.1. Installation du runner

Dans l'architecture CI/CD du projet, les *runners* cloud sont utilisés pour construire et tester les microservices, ce qui permet de bénéficier d'une exécution rapide et parallèle sans surcharger l'infrastructure locale. Le *runner* auto-hébergé, installé sur le nœud *master* du cluster Kubernetes, prend en charge les déploiements, garantissant que les microservices sont mis à jour dans un environnement cohérent et proche de la production.

Le fonctionnement du pipeline est le suivant : lorsqu'un développeur effectue un *push* sur la branche principale, les tâches de construction et de test sont exécutées automatiquement sur les *runners* cloud. Une fois validées, le *runner* du nœud *master* applique les manifests Kubernetes pour déployer les services sur le cluster, assurant ainsi la mise à jour automatique des microservices. Cette configuration offre une séparation claire des responsabilités, optimise l'utilisation des ressources et garantit que chaque étape du pipeline est fiable, reproductible et traçable.

Pour configurer un agent sur le nœud *master* du cluster afin qu'il serve de *runner*, il est nécessaire de disposer d'un compte GitHub et d'un accès administrateur sur le nœud. Il faut ensuite se rendre dans le dépôt GitHub concerné, puis accéder à **Settings** → **Actions** → **Runners**, comme le montre la figure 11.

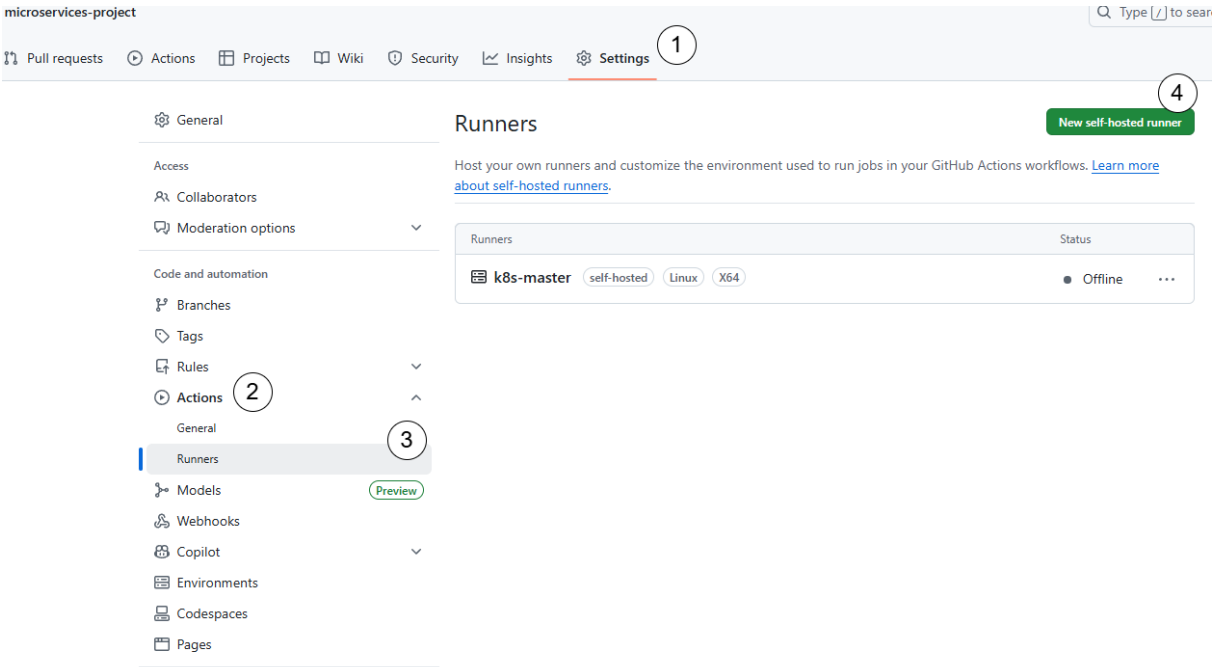


Figure 11 : Accès aux paramètres des runners du dépôt GitHub

À partir de cette section, on choisit d'ajouter un nouveau *runner* et on sélectionne le système d'exploitation de la machine cible, ici Linux x64. GitHub fournit alors les commandes d'installation et d'enregistrement à exécuter sur le nœud *master* (figure 12).

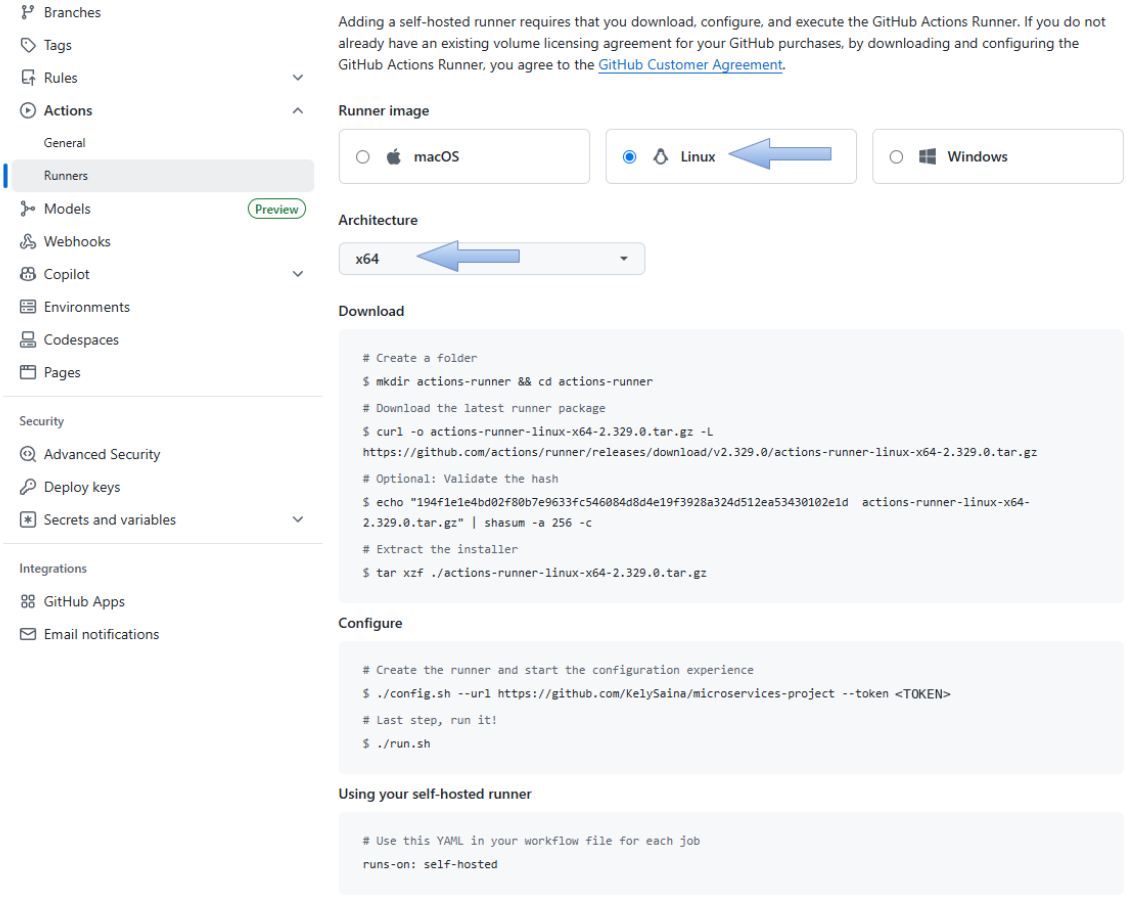


Figure 12 : Instructions d'installation d'un runner auto-hébergé

Sur le nœud *master*, on commence par créer un dossier dédié au *runner*, puis on y télécharge et décompresse la dernière version du paquet fourni par GitHub :

```
master@k8s-master $ mkdir actions-runner && cd actions-runner
master@k8s-master $ curl -o actions-runner-linux-x64-2.329.0.tar.gz -L \
    https://github.com/actions/runner/releases/download/v2.329.0/actions-runner-linux-x64-2.329.
        0.tar.gz
master@k8s-master $ tar xzf ./actions-runner-linux-x64-2.329.0.tar.gz
```

On lance ensuite le processus de configuration en liant le *runner* au dépôt GitHub grâce au jeton d'enregistrement temporaire fourni par GitHub (masqué ci-dessous) :

```
master@k8s-master $ ./config.sh --url https://github.com/KelySaina/microservices-project
    --token <TOKEN>
```

Le script affiche d'abord un message d'accueil, puis vérifie la connexion réseau vers GitHub et les permissions nécessaires, et crée les répertoires de travail destinés aux tâches et aux fichiers temporaires. On choisit ensuite le groupe et le nom du *runner*, qui servira à l'identifier dans l'interface GitHub, ainsi que des étiquettes (*labels*) facultatives permettant de diriger certaines tâches uniquement vers ce *runner* (figure 13).



Figure 13 : Enregistrement du runner auprès du dépôt GitHub

Après la saisie de ces informations, le script copie les fichiers nécessaires dans le répertoire local, génère les scripts d'exécution et confirme que le *runner* est enregistré et que ses paramètres de travail sont sauvegardés (figure 14).

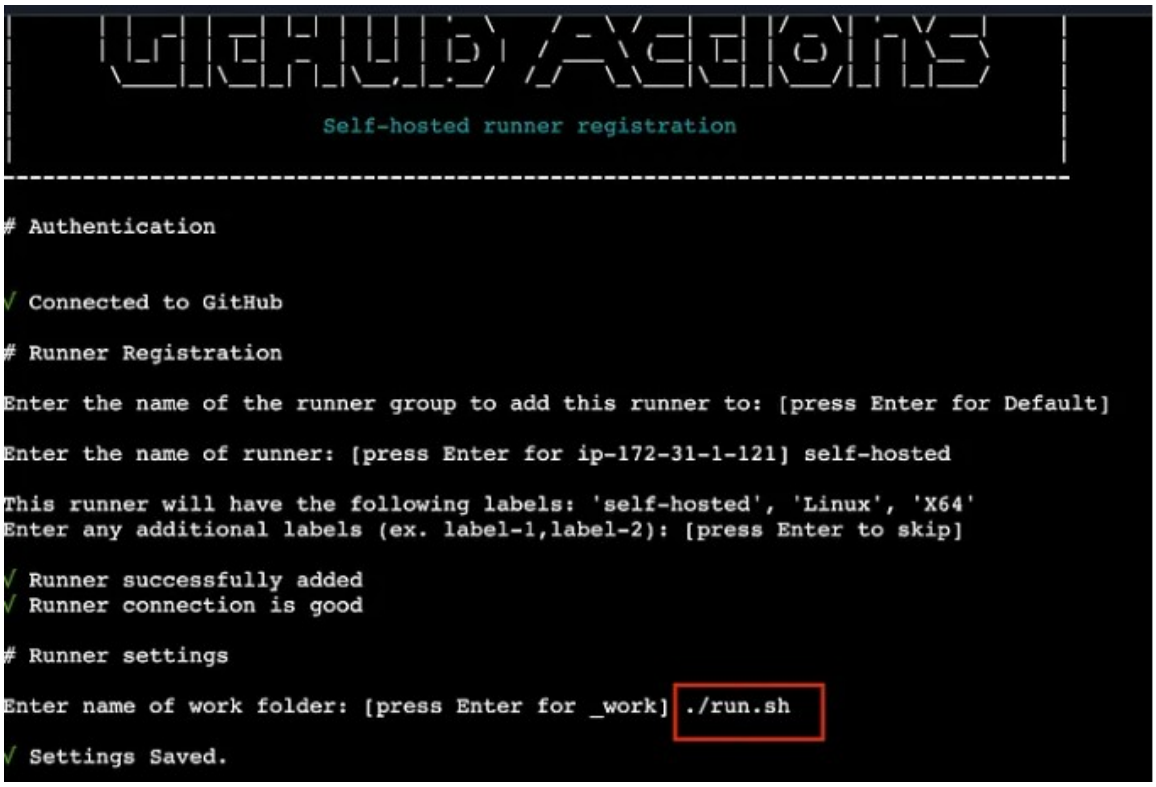


Figure 14 : Configuration du dossier de travail du runner

Une fois la configuration terminée, on démarre le *runner* avec la commande `./run.sh`. Le terminal affiche alors en temps réel que le *runner* est connecté et en attente de tâches

(figure 15).



Figure 15 : Runner démarré et en attente de tâches

Chaque fois qu'un *workflow* GitHub Actions correspondant aux étiquettes configurées est déclenché, le *runner* récupère automatiquement la tâche, télécharge le code, exécute les étapes définies (construction, tests, déploiement) et renvoie les résultats à GitHub. Les figures 16 et 17 illustrent deux scénarios possibles : une tâche qui se termine avec succès, et une tâche dont une étape échoue, auquel cas le terminal signale l'erreur et la tâche est marquée en échec.

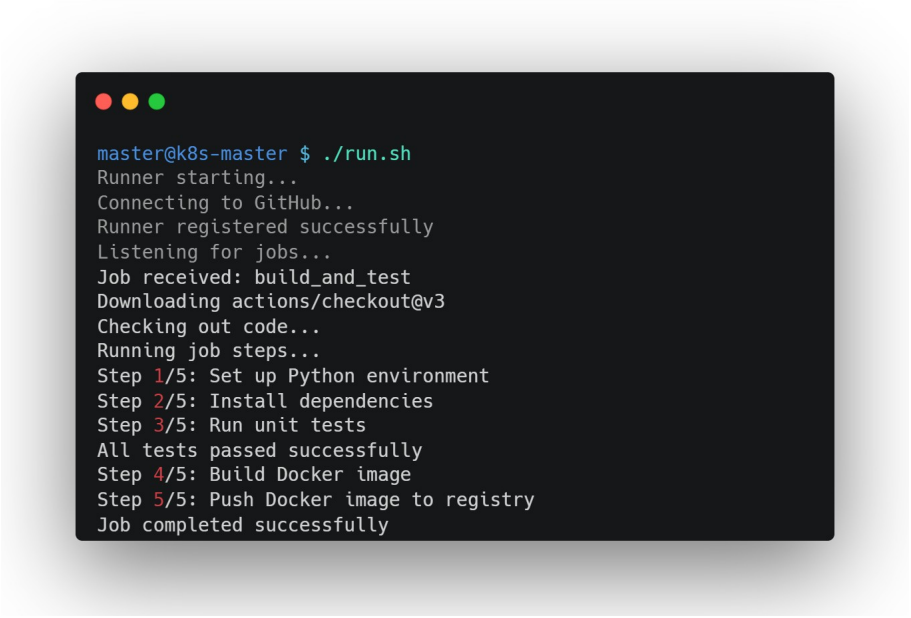


Figure 16 : Exemple d'exécution d'une tâche réussie par le runner

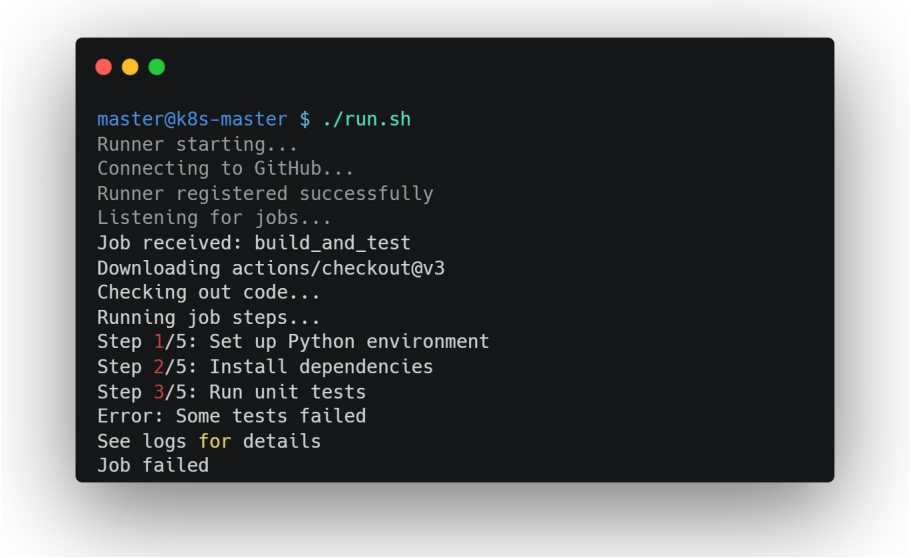


Figure 17 : Exemple d'exécution d'une tâche en échec

6.2.2. Configuration du pipeline

Le pipeline CI/CD est l'élément central qui automatise la construction, les tests et le déploiement des microservices. Pour ce projet, il est défini avec GitHub Actions dans un fichier YAML qui décrit les différentes étapes à exécuter lorsqu'un événement déclencheur se produit. Ce fichier, `ci.yml`, est placé dans le répertoire `.github/workflows/` du projet (figure 18).

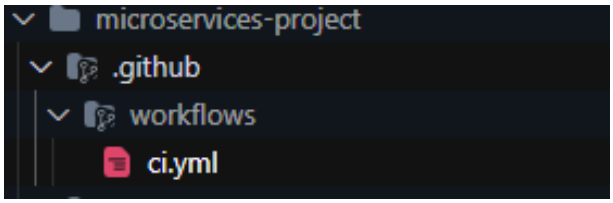


Figure 18 : Emplacement du fichier de workflow dans le dépôt

Le *workflow* est déclenché automatiquement à chaque *push* sur la branche `main`. Il définit des variables d'environnement globales qui centralisent les paramètres réutilisables : le registre de conteneurs, le propriétaire des images, le *namespace* Kubernetes et le délai d'attente de santé des pods.

```
name: CI/CD

on:
  push:
    branches: ["main"]

env:
  REGISTRY: ghcr.io
  IMAGE_OWNER: kelysaina
  NAMESPACE: microservices
  HEALTH_TIMEOUT: 180 # seconds
```

Le pipeline est structuré en quatre tâches (*jobs*) : `build`, `test`, `deploy` et `cypress`, orchestrées avec des dépendances explicites (`needs`) pour garantir que les étapes critiques s'exécutent dans le bon ordre.

a) Tâche build

La tâche `build` s'exécute sur un *runner* GitHub standard (`ubuntu-latest`). Elle récupère le code du dépôt, puis se connecte au GitHub Container Registry pour pouvoir y pousser les images Docker. Une version globale est générée pour chaque exécution ; elle sert d'étiquette (*tag*) aux images Docker et est transmise aux tâches suivantes. Le pipeline détermine ensuite les services modifiés depuis le dernier *commit* en filtrant les dossiers `services/` du dépôt, ce qui permet de construire et de pousser uniquement les images concernées, économisant ainsi du temps et des ressources. Chaque image est étiquetée à la fois avec la version générée et avec `latest`.

```
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      changed_services: ${{ steps.changes.outputs.CHANGED_SERVICES }}
      version: ${{ steps.version.outputs.VERSION }}

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Log in to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Generate global version
        id: version
        run: |
          VERSION="v-$((RANDOM%9000+1000))"
          BUILD_DATE=$(date +%F)
          echo "VERSION=$VERSION" >> $GITHUB_ENV
          echo "BUILD_DATE=$BUILD_DATE" >> $GITHUB_ENV
          echo "VERSION=$VERSION" >> $GITHUB_OUTPUT
          echo "Generated global version: $VERSION, build date: $BUILD_DATE"

      - name: Determine changed services
        id: changes
        run: |
          git fetch --depth=2
          PREV_COMMIT=${{ github.event.before }}
          if [ -z "$PREV_COMMIT" ] || [ "$PREV_COMMIT" =
              "0000000000000000000000000000000000000000" ]; then
            PREV_COMMIT=$(git rev-parse HEAD~1)
          fi

          CHANGED_SERVICES=$(git diff --name-only "$PREV_COMMIT" HEAD \
            | grep -E '^services/' \
            | cut -d/ -f2 \
            | sort -u \
            | tr '\n' ' ')
          echo "Changed services: $CHANGED_SERVICES"
          echo "CHANGED_SERVICES=$CHANGED_SERVICES" >> $GITHUB_OUTPUT

      - name: Build and Push changed images
        run: |
          if [ -z "${{ steps.changes.outputs.CHANGED_SERVICES }}" ]; then
            echo "No services changed. Skipping build."
          else
            for service in ${{ steps.changes.outputs.CHANGED_SERVICES }}; do
              echo "Building and pushing $service:$VERSION"
              docker build \
                --build-arg VERSION=$VERSION \
                --build-arg BUILD_DATE=$BUILD_DATE \
                -t $REGISTRY/$IMAGE_OWNER/$service:$VERSION \
                -t $REGISTRY/$IMAGE_OWNER/$service:latest \
                ./services/$service

              docker push $REGISTRY/$IMAGE_OWNER/$service:$VERSION
              docker push $REGISTRY/$IMAGE_OWNER/$service:latest
```

```
            done
          fi
```

b) Tâche test

La tâche `test` s'exécute après la construction, également sur un *runner* GitHub standard. Elle récupère les images Docker construites pour les services modifiés, puis valide chacune d'elles : le conteneur est démarré, le pipeline vérifie qu'il est toujours en cours d'exécution après quelques secondes et, si le service expose un port, il interroge son point d'accès `/graphql`. Un conteneur qui ne démarre pas ou s'arrête immédiatement fait échouer la tâche, ce qui bloque le déploiement.

```
  test:
    runs-on: ubuntu-latest
    needs: build
    env:
      VERSION: ${{ needs.build.outputs.version }}
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Log in to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Pull built images
        run: |
          for service in ${{ needs.build.outputs.changed_services }}; do
            echo "Pulling image for $service:$VERSION"
            docker pull $REGISTRY/$IMAGE_OWNER/$service:$VERSION
          done

      - name: Validate built images
        run: |
          for service in ${{ needs.build.outputs.changed_services }}; do
            echo "[>>] Testing container startup for $service:$VERSION"

            CONTAINER_ID=$(docker run -d $REGISTRY/$IMAGE_OWNER/$service:$VERSION || true)

            if [ -z "$CONTAINER_ID" ]; then
              echo "[ERREUR] Failed to start $service container"
              exit 1
            fi

            # Give it a few seconds to spin up
            sleep 10

            # Check if the container is still running
            STATUS=$(docker inspect --format '{{.State.Status}}' $CONTAINER_ID)
            if [ "$STATUS" != "running" ]; then
              echo "[ERREUR] $service container exited immediately with status: $STATUS"
              docker logs $CONTAINER_ID
              docker rm -f $CONTAINER_ID
              exit 1
            fi

            # Optional: HTTP probe if service exposes a port
            PORT=$(docker port $CONTAINER_ID | head -n1 | cut -d: -f2)
            if [ -n "$PORT" ]; then
              RESPONSE=$(docker exec $CONTAINER_ID curl -s -o /dev/null -w "%{http_code}"
                  "http://localhost:$PORT/graphql" || echo "")
              if [[ "$RESPONSE" =~ ^2|3$ ]]; then
                echo "[OK] $service responds on port $PORT (HTTP $RESPONSE)"
              else
                echo "[ATTENTION] $service did not respond properly on port $PORT"
              fi
            fi

            docker logs $CONTAINER_ID
            docker rm -f $CONTAINER_ID
            echo "-----------------------------------------------"
          done
```

c) Tâche deploy

La tâche `deploy` s'exécute sur le *runner* auto-hébergé, c'est-à-dire sur le nœud *master* du cluster Kubernetes. Elle vérifie d'abord la configuration de `kubectl` pour s'assurer de la connectivité avec le cluster. Le pipeline applique ensuite les manifests Kubernetes de base : il crée le *namespace* et déploie les services ainsi que les *Horizontal Pod Autoscalers* (HPA). Pour chaque service modifié, il met à jour l'image avec l'étiquette générée et ajoute une annotation de version pour assurer la traçabilité.

```
  deploy:
    runs-on: self-hosted
    needs: [build, test]
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up kubectl
        run: |
          kubectl version --client
          kubectl config get-contexts

      - name: Deploy Namespace
        run: |
          kubectl apply -f k8s/namespace.yaml
          kubectl get namespaces

      - name: Deploy Base Manifests
        run: |
          kubectl apply -n $NAMESPACE -f k8s/svcs/
          kubectl apply -n $NAMESPACE -f k8s/hpas/

      - name: Deploy Changed Services with Version Tag
        run: |
          VERSION=${{ needs.build.outputs.version }}
          for service in ${{ needs.build.outputs.changed_services }}; do
            echo "[*] Deploying $service with version $VERSION"

            # Capture current image and revision
            CURRENT_IMAGE=$(kubectl get deployment $service -n $NAMESPACE -o
                jsonpath='{.spec.template.spec.containers[0].image}' 2>/dev/null || true)
            CURRENT_REV=$(kubectl rollout history deployment/$service -n $NAMESPACE | grep -E
                '^[0-9]+' | tail -n 1 | awk '{print $1}')
            echo "[*] Current image for $service: ${CURRENT_IMAGE:-<none>}, revision:
                ${CURRENT_REV:-<none>}"

            # Deploy new image
            kubectl set image deployment/$service \
              $service=$REGISTRY/$IMAGE_OWNER/$service:$VERSION \
              -n $NAMESPACE --record

            # Annotate deployment for traceability
            kubectl annotate deployment/$service version=$VERSION --overwrite -n $NAMESPACE
          done
```

Un aspect clé de ce pipeline est la validation stricte de l'état des pods après le déploiement. Le pipeline attend que le déploiement soit terminé, puis vérifie que tous les pods sont prêts et effectue une seconde vérification après un court délai pour s'assurer que la mise à jour est stable. En cas de problème, il tente un retour arrière « intelligent » vers l'image précédente non étiquetée `latest`, garantissant que le service reste disponible et fonctionnel. Un délai maximal est également configuré pour éviter les blocages prolongés, et tout incident

fait échouer la tâche afin d’être immédiatement signalé.

```
      - name: Wait and Validate Rollout
        run: |
          VERSION=${{ needs.build.outputs.version }}

          for service in ${{ needs.build.outputs.changed_services }}; do
            echo "[>>] Rolling out $service..."

            # --- Step 1: Wait for rollout status ---
            if ! kubectl rollout status deployment/$service -n $NAMESPACE --timeout=360s; then
              echo "[ERREUR] Rollout failed for $service. Attempting smart rollback..."

              # Get previous non-latest image from rollout history
              PREV_IMAGE=$(kubectl rollout history deployment/$service -n $NAMESPACE \
                            | grep -v "latest" \
                            | grep "$service=" \
                            | tail -2 | head -1 \
                            | sed -E "s/.*$service=([^ ]+).*/\1/")

              if [ -n "$PREV_IMAGE" ]; then
                echo "[ROLLBACK] Rolling back $service to previous non-latest image:
                    $PREV_IMAGE"
                kubectl set image deployment/$service $service=$PREV_IMAGE -n $NAMESPACE
                    --record
                PREV_VERSION=${PREV_IMAGE##*:}
                kubectl annotate deployment/$service version=$PREV_VERSION --overwrite -n
                    $NAMESPACE
              else
                echo "[ATTENTION] No valid previous non-latest image found. Cannot rollback
                    properly!"
              fi
              exit 1
            fi

            # --- Step 2: Double-check pods readiness ---
            echo "[CHECK] Verifying pod readiness for $service..."
            TIMEOUT=60
            INTERVAL=15
            ELAPSED=0

            while [ $ELAPSED -lt $TIMEOUT ]; do
              NOT_READY=$(kubectl get pods -n $NAMESPACE -l app=$service \
                -o jsonpath='{range .items[*]}{.metadata.name}{"="}{.status.containerStatuses[0]
                    .ready}{"\n"}{end}' \
                | grep -v "true" || true)

              if [ -z "$NOT_READY" ]; then
                echo "[OK] All pods ready. Rechecking in $INTERVAL seconds..."
                sleep $INTERVAL

                # Double-check
                NOT_READY_AGAIN=$(kubectl get pods -n $NAMESPACE -l app=$service \
                  -o jsonpath='{range .items[*]}{.metadata.name}{"="}{.status.containerStatuses[
                      0].ready}{"\n"}{end}' \
                  | grep -v "true" || true)

                if [ -z "$NOT_READY_AGAIN" ]; then
                  echo "[OK] Pods still ready. Deployment for $service healthy."
                  break
                else
                  echo "[ERREUR] Pods became unready after recheck. Rolling back..."
                  PREV_IMAGE=$(kubectl rollout history deployment/$service -n $NAMESPACE \
                                | grep -v "latest" \
```

```
                              | grep "$service=" \
                              | tail -2 | head -1 \
                              | sed -E "s/.*$service=([^ ]+).*/\1/")
                  if [ -n "$PREV_IMAGE" ]; then
                    kubectl set image deployment/$service $service=$PREV_IMAGE -n $NAMESPACE
                        --record
                    PREV_VERSION=${PREV_IMAGE##*:}
                    kubectl annotate deployment/$service version=$PREV_VERSION --overwrite -n
                        $NAMESPACE
                  fi
                  exit 1
                fi
              else
                echo "[ATTENTE] Pods not ready yet ($ELAPSED/$TIMEOUTs). Waiting $INTERVAL
                    seconds..."
                sleep $INTERVAL
                ELAPSED=$((ELAPSED + INTERVAL))
              fi
            done

            # --- Step 3: Timeout protection ---
            if [ $ELAPSED -ge $TIMEOUT ]; then
              echo "[ATTENTION] Timeout waiting for pods. Rolling back..."
              PREV_IMAGE=$(kubectl rollout history deployment/$service -n $NAMESPACE \
                            | grep -v "latest" \
                            | grep "$service=" \
                            | tail -2 | head -1 \
                            | sed -E "s/.*$service=([^ ]+).*/\1/")
              if [ -n "$PREV_IMAGE" ]; then
                kubectl set image deployment/$service $service=$PREV_IMAGE -n $NAMESPACE
                    --record
                PREV_VERSION=${PREV_IMAGE##*:}
                kubectl annotate deployment/$service version=$PREV_VERSION --overwrite -n
                    $NAMESPACE
              fi
              exit 1
            fi

            echo "[OK] Rollout successful for $service with all pods healthy."
          done
```

d) Tâche cypress

Enfin, la tâche `cypress` s'exécute après un déploiement réussi, sur le *runner* auto-hébergé. Elle installe les dépendances de l'interface web, démarre celle-ci, puis lance la suite de tests de bout en bout Cypress [17], qui simule les parcours des utilisateurs (connexion, panier, commandes, administration des produits et des commandes) sur la nouvelle version déployée. Les captures produites en cas d'échec sont conservées comme artefacts du *workflow*.

```
  cypress:
    runs-on: self-hosted
    needs: deploy
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '20' # Cypress 15+ needs Node 18+

      - name: Install frontend dependencies
        run: npm install
        working-directory: ./frontend

      - name: Start frontend server
        run: npm run dev &
        working-directory: ./frontend

      - name: Install Cypress in tests folder
        run: npm install cypress --save-dev
        working-directory: ./tests

      - name: Run Cypress tests
        run: BASE_URL=http://localhost:5173 npx cypress run --config baseUrl=$BASE_URL
        working-directory: ./tests

      - name: Upload Cypress artifacts
        uses: actions/upload-artifact@v4
        with:
          name: cypress-artifacts
          path: |
            ./tests/cypress/screenshots
```

Ainsi, ce pipeline assure une automatisation complète et fiable, avec une livraison rapide des microservices, une gestion fine des versions et un mécanisme robuste de retour arrière en cas de défaillance. Chaque étape est traçable, de la détection des changements dans le dépôt jusqu'à la mise à jour des services dans le cluster Kubernetes et leur validation fonctionnelle, garantissant la cohérence et la sécurité du déploiement.

6.3. Configuration de l'orchestration : manifests Kubernetes

Pour orchestrer les microservices dans le cluster Kubernetes, on utilise des manifests Kubernetes au format YAML [10]. Ces fichiers décrivent de manière déclarative les ressources nécessaires à l'exécution et à la gestion des services, garantissant une orchestration reproductible et cohérente.

Chaque microservice possède son propre *Deployment*, qui définit le nombre de réplicas, l'image Docker à utiliser, les étiquettes, les ports exposés et la stratégie de mise à jour. Cela permet au cluster de gérer automatiquement le déploiement, la mise à l'échelle et la disponibilité des applications. Les *Services* Kubernetes associés exposent les microservices à l'intérieur du cluster et peuvent être de type ClusterIP, NodePort ou LoadBalancer selon les besoins.

Pour les microservices critiques, des *Horizontal Pod Autoscalers* (HPA) sont définis afin d'adapter dynamiquement le nombre de pods en fonction de métriques telles que l'utilisation du CPU. Cette approche garantit que les services peuvent supporter des variations de charge sans intervention manuelle.

Les manifests incluent également la configuration d'un *namespace*, permettant d'isoler les microservices dans un environnement spécifique et de faciliter la gestion des ressources et des accès :

```
apiVersion: v1
kind: Namespace
metadata:
  name: microservices
```

Chaque déploiement est annoté avec sa version afin d'assurer une traçabilité complète et de faciliter les retours arrière en cas de problème. En pratique, le pipeline CI/CD applique ces manifests à chaque déploiement via la commande `kubectl apply`, ce qui met à jour les ressources et déclenche un déploiement progressif (*rollout*) automatique des services modifiés. Les étapes de validation vérifient ensuite que tous les pods sont prêts et fonctionnels.

6.3.1. Déploiements

Dans Kubernetes, un *Deployment* est une ressource qui décrit comment déployer une application, gérer ses réplicas et effectuer des mises à jour progressives. Il fournit une abstraction de gestion des pods, permettant à Kubernetes de s'occuper de la mise à l'échelle, de la mise à jour des images et du rétablissement en cas de défaillance. Chaque *Deployment* est associé à un sélecteur (*selector*) qui définit les pods ciblés grâce à des étiquettes. Cela garantit que les modifications apportées au *Deployment* (comme le changement d'image Docker) sont appliquées uniquement aux pods correspondants.

Les *Services* Kubernetes complètent les *Deployments* en fournissant un point d'accès stable aux pods, indépendamment de leur adresse IP dynamique. Un *Service* agit comme une couche d'abstraction réseau et peut être de différents types :

- **ClusterIP :** le type par défaut, qui expose le service uniquement à l'intérieur du cluster. Il convient aux microservices internes de ce projet, qui s'appellent mutuellement sans être directement accessibles depuis l'extérieur ;
- **NodePort :** expose le service sur un port spécifique de chaque nœud du cluster, permettant un accès externe limité ;
- **LoadBalancer :** utilisé dans des environnements cloud, il crée automatiquement un équilibreur de charge pour exposer le service à Internet ;

- **ExternalName :** associe le service à un nom DNS externe, redirigeant le trafic vers un service externe.

Le tableau 20 résume le rôle de ces deux ressources dans le projet.

Tableau 20 : Rôle des ressources Deployment et Service dans le projet

Ressource (*kind*)	Objectif principal	Usage dans le projet
Deployment	Gestion des pods, mise à l'échelle, mise à jour et retour arrière	Chaque microservice dispose d'un *Deployment* pour gérer ses pods
Service	Fournir un point d'accès stable et interne aux pods via des étiquettes	Expose les pods de chaque microservice avec un *Service* ClusterIP

Les manifests des services sont regroupés dans le dossier `k8s/svcs` (figure 19).

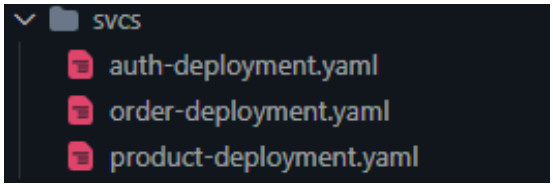


Figure 19 : Manifests des déploiements des microservices

Chaque microservice possède un *Deployment* et un *Service* de type ClusterIP, correspondant à une architecture de microservices interne et isolée du trafic externe. Les *Deployments* utilisent `replicas: 2`, garantissant une disponibilité minimale en cas de défaillance d'un pod. L'attribut `imagePullPolicy: Always` force le cluster à récupérer systématiquement l'image depuis le registre, ce qui est essentiel dans le cadre d'un pipeline CI/CD avec des mises à jour fréquentes. Chaque *Deployment* référence également `imagePullSecrets`, permettant l'authentification auprès du GitHub Container Registry et sécurisant le téléchargement des images privées. Le manifest du service d'authentification est le suivant :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
  namespace: microservices
spec:
  replicas: 2
  selector:
    matchLabels:
      app: auth-service
  template:
    metadata:
      labels:
        app: auth-service
    spec:
      containers:
        - name: auth-service
          image: ghcr.io/kelysaina/auth-service:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 4001
      imagePullSecrets:
        - name: ghcr-secret
---
apiVersion: v1
kind: Service
metadata:
  name: auth-service
  namespace: microservices
spec:
  selector:
    app: auth-service
  ports:
    - port: 4001        # ClusterIP port inside cluster
      targetPort: 4001
  type: ClusterIP       # Internal only
```

Les manifests des services de gestion des commandes et des produits suivent la même structure :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
  namespace: microservices
spec:
  replicas: 2
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
    spec:
      containers:
        - name: order-service
          image: ghcr.io/kelysaina/order-service:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 4002
      imagePullSecrets:
        - name: ghcr-secret
---
apiVersion: v1
kind: Service
metadata:
  name: order-service
  namespace: microservices
spec:
  selector:
    app: order-service
  ports:
    - port: 4002        # ClusterIP port inside the cluster
      targetPort: 4002
  type: ClusterIP       # Internal only
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
  namespace: microservices
spec:
  replicas: 2
  selector:
    matchLabels:
      app: product-service
  template:
    metadata:
      labels:
        app: product-service
    spec:
      containers:
        - name: product-service
          image: ghcr.io/kelysaina/product-service:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 4003
      imagePullSecrets:
        - name: ghcr-secret
---
apiVersion: v1
kind: Service
metadata:
  name: product-service
  namespace: microservices
spec:
  selector:
    app: product-service
  ports:
    - port: 4003        # ClusterIP port inside the cluster
      targetPort: 4003
  type: ClusterIP       # Internal only
```

Prenons l'exemple du manifest de `order-service`. Le *Deployment* définit la manière dont le service est exécuté dans le cluster : il crée deux réplicas du conteneur `order-service`, garantissant la disponibilité même si un pod tombe en panne. Chaque pod exécute l'image Docker `ghcr.io/kelysaina/order-service` et expose le port 4002, ce qui permet aux autres composants du cluster de communiquer avec lui. Le champ `imagePullSecrets` autorise Kubernetes à accéder au registre privé pour récupérer l'image, et le sélecteur basé sur l'étiquette `app: order-service` permet au *Deployment* de gérer exactement les pods correspondants.

Le *Service* associé expose le *Deployment* à l'intérieur du cluster via une adresse ClusterIP stable, que les autres microservices peuvent utiliser pour communiquer avec `order-service`. Il redirige le trafic interne reçu sur le port 4002 vers les pods correspondants. Cette configuration permet à `order-service` de fonctionner de manière fiable et scalable, tout en restant accessible uniquement depuis l'intérieur du cluster, conformément à une architecture de microservices isolée et sécurisée.

6.3.2. Ingress

Un *Ingress* est une ressource Kubernetes qui fournit une méthode pour exposer des services du cluster vers l'extérieur via HTTP et HTTPS. Contrairement aux *Services* de type NodePort ou LoadBalancer, l'*Ingress* centralise la gestion de l'accès externe, le routage et les règles de sécurité, offrant une solution plus flexible et évolutive pour la distribution du trafic.

L'*Ingress* fonctionne avec un contrôleur (*Ingress Controller*) qui applique les règles définies dans les manifests [14]. Ces règles déterminent comment router les requêtes vers les différents services du cluster en fonction du chemin (`/auth`, `/orders`…) ou du nom d'hôte. Cela permet de regrouper plusieurs services derrière un seul point d'entrée, simplifiant la gestion des accès externes et réduisant le nombre d'équilibreurs de charge nécessaires. Dans un contexte de microservices, l'*Ingress* est particulièrement utile pour :

- exposer plusieurs services via un seul point d'entrée, évitant la multiplication des adresses IP externes ;
- router le trafic selon les chemins ou les sous-domaines, permettant à chaque microservice de recevoir uniquement le trafic qui lui est destiné ;
- s'intégrer aux outils CI/CD, facilitant les mises à jour automatiques et le déploiement continu de services accessibles depuis l'extérieur.

L'*Ingress* sert donc de passerelle d'accès contrôlée et centralisée pour tous les services exposés du cluster. Le manifest utilisé est le suivant :

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: microservices-ingress
  namespace: microservices
  annotations:
    kubernetes.io/ingress.class: nginx
    nginx.ingress.kubernetes.io/use-regex: "true"
    nginx.ingress.kubernetes.io/rewrite-target: /graphql
spec:
  rules:
    - host: myapp.local
      http:
        paths:
          # --- AUTH SERVICE ---
          - path: /auth(/|$)
            pathType: ImplementationSpecific
            backend:
              service:
                name: auth-service
                port:
                  number: 4001

          # --- ORDER SERVICE ---
          - path: /orders(/|$)
            pathType: ImplementationSpecific
            backend:
              service:
                name: order-service
                port:
                  number: 4002

          # --- PRODUCT SERVICE ---
          - path: /products(/|$)
            pathType: ImplementationSpecific
            backend:
              service:
                name: product-service
                port:
                  number: 4003
```

Ce manifest configure un *Ingress* centralisant l'accès externe aux trois microservices : `auth-service`, `order-service` et `product-service`. La version d'API `networking.k8s.io/v1` correspond à la version stable de l'API de gestion des *Ingress*, et le type `Ingress` définit la ressource comme un point d'entrée HTTP/HTTPS pour router le trafic externe vers les services internes.

Dans la section `metadata`, le nom identifie l'*Ingress* (`microservices-ingress`) et le *namespace* précise que cette configuration s'applique au *namespace* `microservices`. Les annotations configurent le comportement du contrôleur NGINX :

- `kubernetes.io/ingress.class: nginx` indique que cet *Ingress* doit être géré par un contrôleur NGINX ;
- `nginx.ingress.kubernetes.io/use-regex: "true"` permet l'utilisation d'expressions régulières dans les chemins ;
- `nginx.ingress.kubernetes.io/rewrite-target: /graphql` réécrit le chemin d'accès vers `/graphql` au niveau du service cible.

La section `spec` contient les règles de routage. Le champ `host: myapp.local` définit le domaine utilisé pour accéder aux services, et chaque service est associé à un chemin spécifique :

- `/auth(/|$)` redirige le trafic vers `auth-service` sur le port 4001 ;
- `/orders(/|$)` redirige vers `order-service` sur le port 4002 ;
- `/products(/|$)` redirige vers `product-service` sur le port 4003.

Le type de chemin `ImplementationSpecific` laisse au contrôleur NGINX le soin d'interpréter le routage selon sa logique interne. Dans le contexte du projet, cet *Ingress* offre un point d'accès unique pour tous les microservices via le domaine `myapp.local`, un routage flexible basé sur les chemins qui isole les services tout en conservant un accès centralisé, une réécriture de chemin qui simplifie les requêtes côté client, ainsi qu'une grande facilité d'évolution : de nouveaux services peuvent être ajoutés simplement en créant de nouvelles règles.

6.3.3. Horizontal Pod Autoscalers

Le *Horizontal Pod Autoscaler* (HPA) est une ressource Kubernetes qui ajuste automatiquement le nombre de pods d'un *Deployment*, d'un *ReplicaSet* ou d'un *StatefulSet* en fonction de métriques observées, telles que l'utilisation du CPU, de la mémoire ou de métriques personnalisées. L'objectif est d'assurer que l'application reste performante face aux variations de charge tout en optimisant l'usage des ressources.

Le HPA surveille en continu les métriques d'utilisation des pods et ajuste horizontalement leur nombre (augmentation ou diminution) pour répondre à la demande. Contrairement à une mise à l'échelle manuelle, il réagit automatiquement et en temps réel aux pics de trafic comme aux périodes de faible utilisation. Dans un contexte de microservices, le HPA est particulièrement utile pour :

- maintenir la disponibilité et la réactivité des services critiques sous forte charge ;
- optimiser l'utilisation des ressources du cluster en réduisant le nombre de pods lorsque la demande diminue ;
- soutenir la scalabilité automatique, essentielle lorsque la charge varie d'un service à l'autre ;
- permettre aux nouvelles versions déployées par le pipeline CI/CD de bénéficier immédiatement d'une mise à l'échelle automatique.

La mise en place d'un HPA commence par l'identification du *Deployment* cible auquel la mise à l'échelle automatique doit s'appliquer. Il faut ensuite définir les bornes de mise à l'échelle, c'est-à-dire le nombre minimal et maximal de pods : le service conserve ainsi une disponibilité minimale en période de faible charge, sans consommation excessive de

ressources en période de forte demande. La configuration inclut enfin le choix des métriques à surveiller, le plus souvent l'utilisation du CPU et de la mémoire. Une fois déployé, le HPA est suivi pour s'assurer que la mise à l'échelle fonctionne correctement. Les manifests des HPA sont regroupés dans le dossier `k8s/hpas` (figure 20).

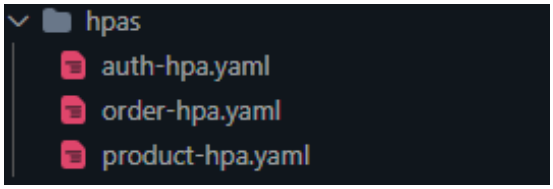


Figure 20 : Manifests des Horizontal Pod Autoscalers

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: auth-service-hpa
  namespace: microservices
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: auth-service
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: order-service-hpa
  namespace: microservices
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-service
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: product-service-hpa
  namespace: microservices
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: product-service
  minReplicas: 2
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 75
```

Ces manifests présentent plusieurs points communs qui reflètent une approche cohérente de la mise à l'échelle automatique dans le projet. Tous ciblent un *Deployment* spécifique via `scaleTargetRef`, en précisant la version d'API, le type de ressource et le nom du microservice concerné. Chaque HPA définit un nombre minimal et maximal de pods pour encadrer la mise à l'échelle, garantissant à la fois la disponibilité minimale du service et une capacité maximale suffisante pour absorber la charge. La métrique utilisée est l'utilisation du CPU, avec un seuil cible (`averageUtilization`) de 70 à 75 %. Enfin, tous les HPA sont déployés dans le *namespace* `microservices`, ce qui centralise la gestion et facilite l'administration et la supervision.

En appliquant l'ensemble de ces manifests, on obtient dans Kubernetes l'architecture présentée dans la figure 21 :

- l'*Ingress* centralise l'accès HTTP et redirige vers les services selon les chemins définis (`/auth`, `/orders`, `/products`) ;
- chaque microservice possède un *Deployment* avec plusieurs réplicas pour la haute disponibilité ;
- les *Services* ClusterIP exposent les pods à l'intérieur du cluster pour la communication interne et pour l'*Ingress* ;
- les HPA ajustent dynamiquement le nombre de pods selon la charge CPU, assurant scalabilité et performance.

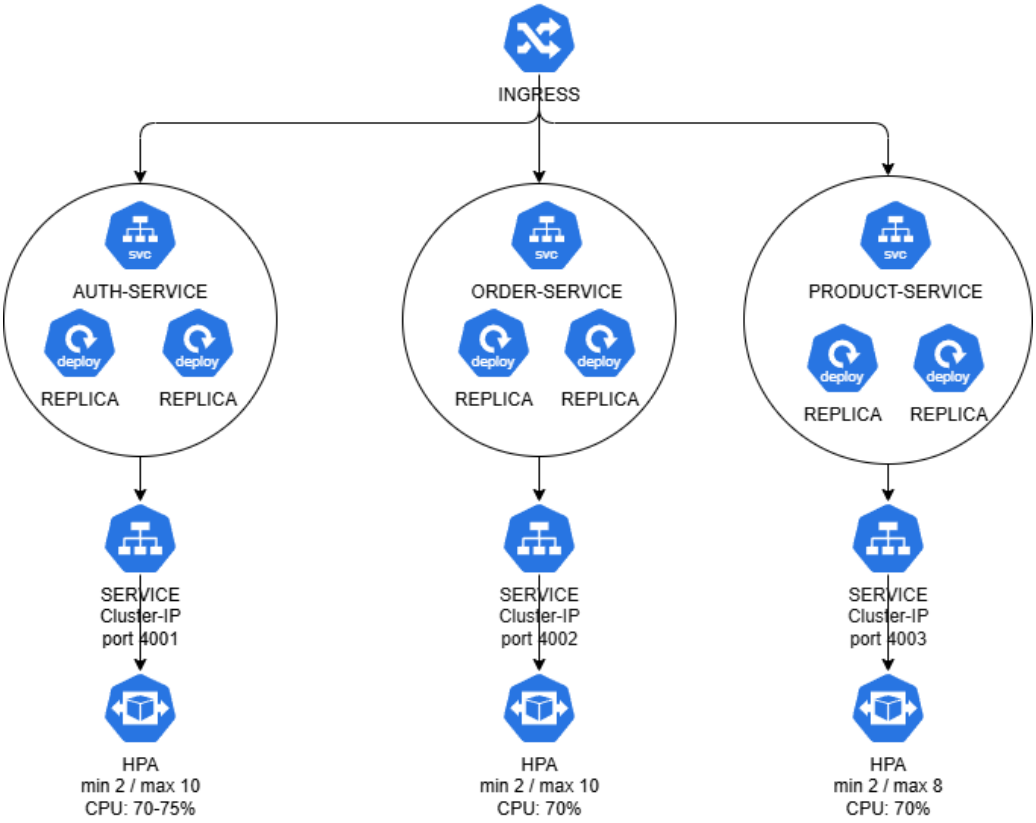


Figure 21 : Architecture des microservices déployés dans Kubernetes

6.4. Application et services à déployer

Le projet repose sur une architecture microservices, chaque service ayant un rôle spécifique et autonome. Les services déployés sont présentés dans le tableau 21.

Tableau 21 : Services de l'application déployée

Service	Rôle principal	Description
Service d'authentification (auth-service)	Authentification et gestion des utilisateurs	Permet la création de comptes, l'authentification, la gestion des profils et des rôles (client, administrateur).
Service des produits (product-service)	Gestion des produits	Gère le catalogue des produits : ajout, mise à jour du stock et consultation des articles disponibles.
Service des commandes (order-service)	Gestion des commandes	Permet aux utilisateurs authentifiés de passer, consulter et suivre leurs commandes. Il communique avec le service des produits pour vérifier la disponibilité et avec le service d'authentification pour obtenir les informations des utilisateurs.

Chaque service fonctionne de manière indépendante et communique via des requêtes GraphQL [16], ce qui assure la cohérence des données et la scalabilité du système. Cette

architecture facilite le déploiement et la maintenance, tout en permettant l'évolution future du projet par l'ajout de nouveaux services ou de nouvelles fonctionnalités. Chaque service écrit également sa version et sa date de construction dans un fichier `version.json` lors de la construction de son image, et les expose via une requête `healthz` utilisée pour vérifier la version effectivement déployée.

6.4.1. Service d'authentification : Node.js / GraphQL / MySQL

Le service d'authentification est développé en Node.js avec GraphQL et utilise MySQL pour la persistance des données. Il est responsable de la création de comptes, de l'authentification des utilisateurs et de la gestion des rôles et des profils. Sa structure repose sur une organisation claire des fichiers, chacun ayant un rôle précis : gestion des utilisateurs, communication avec la base de données et exposition des fonctionnalités via l'API GraphQL (figure 22).

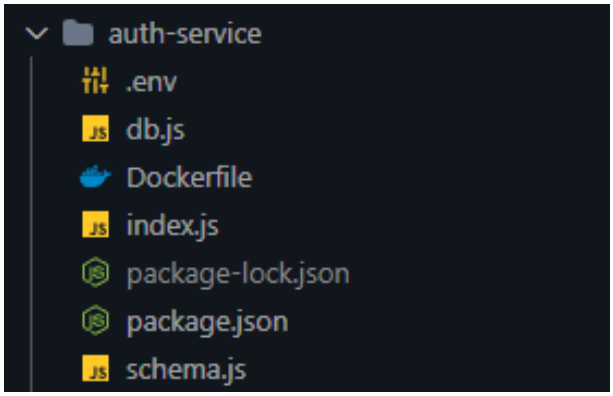


Figure 22 : Structure du service d'authentification

Le fichier `db.js` contient la configuration de la connexion à la base de données MySQL. Il initialise un ensemble (*pool*) de connexions et l'exporte afin qu'il puisse être utilisé dans les autres modules du service.

```
// db.js
import mysql from "mysql2/promise";
import dotenv from 'dotenv';

dotenv.config();

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
});

// Test connection
(async () => {
  try {
    const conn = await pool.getConnection();
    console.log("[OK] MySQL connected successfully!");
    conn.release(); // release the connection back to pool
  } catch (err) {
    console.error("[ERREUR] MySQL connection error:", err);
  }
})();

export default pool;
```

Le `Dockerfile` permet de construire l'image Docker du service. Il facilite le déploiement, assure l'isolation de l'environnement et garantit que le service s'exécute de manière identique sur différentes machines. Les arguments `VERSION` et `BUILD_DATE`, transmis par le pipeline, sont écrits dans le fichier `version.json` de l'image.

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
# Dockerfile
ARG VERSION
ARG BUILD_DATE

# Optional: write version info into a JSON file inside the image
RUN echo "{\"version\":\"${VERSION}\",\"date\":\"${BUILD_DATE}\"}" > /app/version.json

CMD ["node", "index.js"]
```

Le fichier `index.js` constitue le point d'entrée de l'application. Il initialise le serveur Node.js et GraphQL, configure les *middlewares* nécessaires et démarre l'écoute des requêtes entrantes.

```
import express from "express";
import { graphqlHTTP } from "express-graphql";
import schema from "./schema.js";
import dotenv from "dotenv";
import cors from "cors";

dotenv.config();


const app = express();

app.use(cors());

app.use(
  "/graphql",
  graphqlHTTP({
    schema,
    graphiql: true,
  })
);

const PORT = process.env.PORT || 4001;
app.listen(PORT, "0.0.0.0", () => {
  console.log(`Auth service running at ${PORT}/graphql`);
});
```

Le fichier `package.json` gère les dépendances du projet, notamment `express`, `graphql`, `bcryptjs`, `jsonwebtoken` et `mysql2`, et garantit que les versions utilisées sont cohérentes pour tous les développeurs et environnements.

```
{
  "name": "auth-service",
  "version": "1.0.0",
  "main": "server.js",
  "type": "module",
  "scripts": {
    "start": "node server.js",
    "test": "echo \"No tests yet\" && exit 0"
  },
  "dependencies": {
    "bcryptjs": "^3.0.2",
    "cors": "^2.8.5",
    "dotenv": "^17.2.3",
    "express": "^4.21.2",
    "express-graphql": "^0.12.0",
    "graphql": "^15.10.1",
    "jsonwebtoken": "^9.0.2",
    "mysql2": "^3.15.3"
  }
}
```

Enfin, le fichier `schema.js` définit le schéma GraphQL du service. Il décrit le type utilisateur, ainsi que les requêtes (`user`, `users`, `healthz`) et les mutations (`register`, `login`) disponibles. Les mots de passe sont hachés avec bcrypt et la connexion renvoie un jeton JWT valable 24 heures.

```
import fs from "fs";

// Try reading version.json
let VERSION = "v-unknown";
let BUILD_DATE = new Date().toISOString().split("T")[0];

try {
  const data = JSON.parse(fs.readFileSync("/app/version.json", "utf-8"));
  VERSION = data.version;
  BUILD_DATE = data.date;
} catch (err) {
  console.warn("Could not read version.json, using defaults");
}

import {
  GraphQLObjectType,
  GraphQLSchema,
  GraphQLString,
  GraphQLID,
  GraphQLList,
} from "graphql";
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import pool from "./db.js";
import dotenv from "dotenv";

dotenv.config();

// User type
const UserType = new GraphQLObjectType({
  name: "User",
  fields: () => ({
    id: { type: GraphQLID },
    username: { type: GraphQLString },
    email: { type: GraphQLString },
    role: { type: GraphQLString },
    created_at: { type: GraphQLString },
    updated_at: { type: GraphQLString },
  }),
});

const AuthPayloadType = new GraphQLObjectType({
  name: "AuthPayload",
  fields: () => ({
    token: { type: GraphQLString },
    user: { type: UserType },
  }),
});

// Root Query
const RootQuery = new GraphQLObjectType({
  name: "RootQueryType",
  fields: {
    user: {
      type: UserType,
      args: { id: { type: GraphQLID } },
      resolve: async (_, { id }) => {
        const [rows] = await pool.query("SELECT * FROM users WHERE id = ?", [
          id,
        ]);
        return rows[0];
      },
```

```
    },
    users: {
      type: new GraphQLList(UserType),
      resolve: async () => {
        const [rows] = await pool.query("SELECT * FROM users");
        return rows;
      },
    },
    healthz: {
      type: GraphQLString,
      resolve: () =>
        `Auth Service is healthy! ${BUILD_DATE} #${VERSION}`,
    },
  },
});

// Mutations
const Mutation = new GraphQLObjectType({
  name: "Mutation",
  fields: {
    register: {
      type: UserType,
      args: {
        username: { type: GraphQLString },
        email: { type: GraphQLString },
        password: { type: GraphQLString },
        role: { type: GraphQLString },
      },
      resolve: async (_, { username, email, password, role }) => {
        const hashedPassword = await bcrypt.hash(password, 10);
        const [result] = await pool.query(
          "INSERT INTO users (username, email, password_hash, role) VALUES (?, ?, ?, ?)",
          [username, email, hashedPassword, role || "customer"]
        );
        return {
          id: result.insertId,
          username,
          email,
          role: role || "customer",
        };
      },
    },
    login: {
      type: AuthPayloadType,
      args: {
        email: { type: GraphQLString },
        password: { type: GraphQLString },
      },
      resolve: async (_, { email, password }) => {
        const [rows] = await pool.query("SELECT * FROM users WHERE email = ?", [email]);
        const user = rows[0];
        if (!user) throw new Error("User not found");

        const valid = await bcrypt.compare(password, user.password_hash);
        if (!valid) throw new Error("Invalid password");

        const token = jwt.sign(
          { id: user.id, email: user.email, role: user.role },
          process.env.JWT_SECRET,
          { expiresIn: "24h" }
        );
```

```
        return {
          token,
          user: {
            id: user.id,
            username: user.username,
            email: user.email,
            role: user.role,
          },
        };
      },
    },
  },
});

// [OK] Export default for ES module
export default new GraphQLSchema({
  query: RootQuery,
  mutation: Mutation,
});
```

6.4.2. Service de gestion des produits : Python / GraphQL / MySQL

Le service de gestion des produits est développé en Python avec Flask et la bibliothèque GraphQL Ariadne, et utilise MySQL pour la persistance des données. Il est responsable de la création, de la mise à jour du stock et de la consultation des produits du catalogue. Sa structure est présentée dans la figure 23.

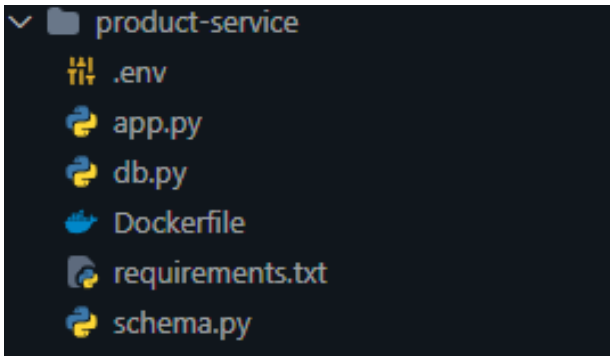


Figure 23 : Structure du service de gestion des produits

Le fichier `db.py` contient la configuration de la connexion à la base de données MySQL et met à disposition un *pool* de connexions pour les autres modules.

```
import mysql.connector
from mysql.connector import pooling
from dotenv import load_dotenv
import os

load_dotenv()

dbconfig = {
    "host": os.getenv("DB_HOST"),
    "user": os.getenv("DB_USER"),
    "password": os.getenv("DB_PASSWORD"),
    "database": os.getenv("DB_NAME")
}

pool = pooling.MySQLConnectionPool(
    pool_name="product_pool",
    pool_size=10,
    **dbconfig
)

def get_connection():
    return pool.get_connection()
```

Le `Dockerfile` permet de construire l'image Docker du service, facilitant le déploiement et l'isolation.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
# Dockerfile
ARG VERSION
ARG BUILD_DATE

# Optional: write version info into a JSON file inside the image
RUN echo "{\"version\":\"${VERSION}\",\"date\":\"${BUILD_DATE}\"}" > /app/version.json

CMD ["python", "app.py"]
```

Le fichier `app.py` constitue le point d'entrée de l'application. Il initialise le serveur Flask, active le CORS, expose l'interface de test GraphQL (requête GET) et l'API GraphQL (requête POST) sur le chemin `/graphql`, puis démarre l'écoute des requêtes sur le port 4003.

```
from flask import Flask, request, jsonify
from flask_cors import CORS
from ariadne import graphql_sync, load_schema_from_path, make_executable_schema
from ariadne.constants import PLAYGROUND_HTML
from schema import schema  # your existing schema
import os
from dotenv import load_dotenv

# Load environment variables from .env
load_dotenv()

app = Flask(__name__)

# Enable CORS for all origins (or restrict with origins=["http://localhost:8080"])
CORS(app)

# GraphQL Playground (GET)
@app.route("/graphql", methods=["GET"])
def graphql_playground():
    return PLAYGROUND_HTML, 200

# GraphQL API (POST)
@app.route("/graphql", methods=["POST"])
def graphql_server():
    data = request.get_json()
    success, result = graphql_sync(
        schema,
        data,
        context_value=request,  # you can add {"user": ...} if needed
        debug=True
    )
    status_code = 200 if success else 400
    return jsonify(result), status_code

if __name__ == "__main__":
    port = int(os.getenv("PORT", 4003))  # default for product service
    app.run(host="0.0.0.0", port=port)
```

Le fichier `requirements.txt` liste les dépendances Python nécessaires au service, comme Flask, Ariadne, `graphql-core` et `mysql-connector-python`.

```
Flask==2.2.5
flask-cors==3.0.10
ariadne==0.17.0
graphql-core==3.2.3
mysql-connector-python
python-dotenv
```

Enfin, le fichier `schema.py` définit le schéma GraphQL du service. Il décrit le type produit, les requêtes de consultation du catalogue (`products`, `product`, `healthz`) et les mutations d’ajout de produit et de mise à jour du stock (`addProduct`, `updateStock`).

```
from ariadne import QueryType, MutationType, make_executable_schema, gql
from db import get_connection
import json

# --- Load version info safely ---
try:
    with open("/app/version.json") as f:
        VERSION_INFO = json.load(f)
except FileNotFoundError:
    VERSION_INFO = {"version": "v-unknown", "date": "unknown"}

# --- GraphQL Schema Definition ---
type_defs = gql("""
type Product {
  id: ID!
  name: String!
  description: String
  price: Float!
  stock: Int!
}

type Query {
  products: [Product!]!
  product(id: ID!): Product
  healthz: String!
}

type Mutation {
  addProduct(name: String!, description: String, price: Float!, stock: Int): Product
  updateStock(id: ID!, stock: Int!): Product
}
""")

query = QueryType()
mutation = MutationType()

# --- Query Resolvers ---
@query.field("products")
def resolve_products(*_):
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM products")
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    return rows


@query.field("product")
def resolve_product(*_, id):
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM products WHERE id=%s", (id,))
    row = cursor.fetchone()
    cursor.close()
    conn.close()
    return row


@query.field("healthz")
def resolve_healthz(*_):
    return f"Product Service is healthy!! {VERSION_INFO['date']} #{VERSION_INFO['version']}"
```

```
# --- Mutation Resolvers ---
@mutation.field("addProduct")
def resolve_add_product(*_, name, description=None, price=0.0, stock=0):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO products (name, description, price, stock) VALUES (%s, %s, %s, %s)",
        (name, description, price, stock)
    )
    conn.commit()
    product_id = cursor.lastrowid

    # Fetch the inserted product for full response
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM products WHERE id=%s", (product_id,))
    row = cursor.fetchone()

    cursor.close()
    conn.close()
    return row


@mutation.field("updateStock")
def resolve_update_stock(*_, id, stock):
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)

    cursor.execute("UPDATE products SET stock=%s WHERE id=%s", (stock, id))
    conn.commit()

    cursor.execute("SELECT * FROM products WHERE id=%s", (id,))
    row = cursor.fetchone()

    cursor.close()
    conn.close()
    return row


# --- Executable Schema ---
schema = make_executable_schema(type_defs, query, mutation)
```

6.4.3. Service de gestion des commandes : Node.js / GraphQL / MySQL

Le service de gestion des commandes est développé en Node.js avec GraphQL et utilise MySQL pour la persistance des données. Il est responsable de la création des commandes, de leur consultation et de la mise à jour de leur statut. Il communique avec le service des produits pour vérifier le prix et le stock disponibles, et avec le service d’authentification pour obtenir les informations des utilisateurs. Sa structure est présentée dans la figure 24.

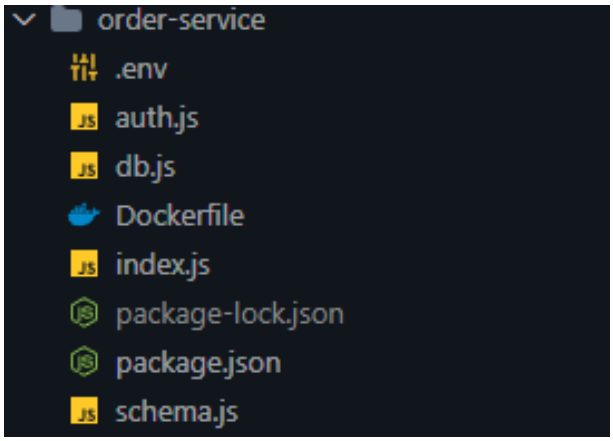


Figure 24 : Structure du service de gestion des commandes

Le fichier `db.js` contient la configuration de la connexion à la base de données MySQL et exporte le *pool* de connexions utilisé par les autres modules du service.

```
import mysql from "mysql2/promise";
import dotenv from "dotenv";
dotenv.config();

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
});

export default pool;
```

Le `Dockerfile` permet de construire l'image Docker du service de gestion des commandes, selon le même principe que pour les autres services.

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
# Dockerfile
ARG VERSION
ARG BUILD_DATE

# Optional: write version info into a JSON file inside the image
RUN echo "{\"version\":\"${VERSION}\",\"date\":\"${BUILD_DATE}\"}" > /app/version.json

CMD ["node", "index.js"]
```

Le fichier `auth.js` fournit une fonction qui sécurise l'accès aux fonctionnalités du service. Elle extrait le jeton JWT de l'en-tête `Authorization` de la requête, le vérifie à l'aide de la clé secrète et retourne les informations de l'utilisateur si le jeton est valide.

```
import jwt from "jsonwebtoken";
import dotenv from "dotenv";
dotenv.config();

export const authenticate = (req) => {
  const authHeader = req.headers.authorization || "";
  const token = authHeader.replace("Bearer ", "");
  if (!token) return null;

  try {
    return jwt.verify(token, process.env.JWT_SECRET);
  } catch (err) {
    return null;
  }
};
```

Le fichier `index.js` constitue le point d’entrée de l’application. Il initialise le serveur Node.js et GraphQL, injecte l’utilisateur authentifié dans le contexte de chaque requête et démarre l’écoute des requêtes entrantes sur le port 4002.

```
import express from "express";
import { graphqlHTTP } from "express-graphql";
import schema from "./schema.js";
import { authenticate } from "./auth.js";
import dotenv from "dotenv";
import cors from "cors";

dotenv.config();

const app = express();

app.use(cors());

app.use(
  "/graphql",
  graphqlHTTP((req) => ({
    schema,
    graphiql: true,
    context: { user: authenticate(req) },
  }))
);

const PORT = process.env.PORT || 4002;
app.listen(PORT, () => {
  console.log(`Order service running at ${PORT}/graphql`);
});
```

Le fichier `package.json` gère les dépendances du service ; on y retrouve notamment `axios`, utilisé pour interroger les autres microservices.

```
{
  "name": "auth-service",
  "version": "1.0.0",
  "main": "server.js",
  "type": "module",
  "scripts": {
    "start": "node server.js",
    "test": "echo \"No tests yet\" && exit 0"
  },
  "dependencies": {
    "axios": "^1.13.0",
    "bcryptjs": "^3.0.2",
    "cors": "^2.8.5",
    "dotenv": "^17.2.3",
    "express": "^4.21.2",
    "express-graphql": "^0.12.0",
    "graphql": "^15.10.1",
    "jsonwebtoken": "^9.0.2",
    "mysql2": "^3.15.3"
  }
}
```

Enfin, le fichier `schema.js` définit le schéma GraphQL du service. Il décrit les types commande et ligne de commande, les requêtes (`myOrders`, `orders`, `healthz`) et les mutations (`createOrder`, `updateOrderStatus`). La création d'une commande interroge le service des produits pour chaque article, vérifie le stock, calcule le montant total et enregistre la commande et ses lignes dans une transaction, annulée en cas d'erreur. Les informations sur les produits et les utilisateurs d'une commande sont résolues à la demande auprès des services correspondants.

```
import fs from "fs";
import {
  GraphQLObjectType,
  GraphQLSchema,
  GraphQLID,
  GraphQLString,
  GraphQLFloat,
  GraphQLInt,
  GraphQLList,
  GraphQLInputObjectType,
} from "graphql";
import pool from "./db.js";
import axios from "axios";
import dotenv from "dotenv";
dotenv.config();

// --- Version info ---
let VERSION = "v-unknown";
let BUILD_DATE = new Date().toISOString().split("T")[0];
try {
  const data = JSON.parse(fs.readFileSync("/app/version.json", "utf-8"));
  VERSION = data.version;
  BUILD_DATE = data.date;
} catch (err) {
  console.warn("Could not read version.json, using defaults");
}

// --- Input type for order items ---
const OrderItemInputType = new GraphQLInputObjectType({
  name: "OrderItemInput",
  fields: {
    product_id: { type: GraphQLID },
    quantity: { type: GraphQLInt },
  },
});

// --- Product type (for resolver) ---
const ProductType = new GraphQLObjectType({
  name: "Product",
  fields: {
    id: { type: GraphQLID },
    name: { type: GraphQLString },
    description: { type: GraphQLString },
    price: { type: GraphQLFloat },
    stock: { type: GraphQLInt },
  },
});

// --- User type (for resolver) ---
const UserType = new GraphQLObjectType({
  name: "User",
  fields: {
    id: { type: GraphQLID },
    username: { type: GraphQLString },
    email: { type: GraphQLString },
    role: { type: GraphQLString },
  },
});

// --- Order Item Type ---
const OrderItemType = new GraphQLObjectType({
  name: "OrderItem",
```

```
  fields: () => ({
    id: { type: GraphQLID },
    product_id: { type: GraphQLID },
    quantity: { type: GraphQLInt },
    unit_price: { type: GraphQLFloat },
    // resolve product info from product-service
    product: {
      type: ProductType,
      resolve: async (parent) => {
        try {
          const query = `
            query($id: ID!) {
              product(id: $id) {
                id
                name
                description
                price
              }
            }
          `;
          const response = await axios.post(process.env.PRODUCT_SERVICE_URL, {
            query,
            variables: { id: parent.product_id },
          });
          return response.data.data.product;
        } catch (err) {
          console.error("Failed to fetch product:", err);
          return null;
        }
      },
    },
  }),
});

// --- Order Type ---
const OrderType = new GraphQLObjectType({
  name: "Order",
  fields: () => ({
    id: { type: GraphQLID },
    user_id: { type: GraphQLID },
    total_amount: { type: GraphQLFloat },
    status: { type: GraphQLString },
    items: {
      type: new GraphQLList(OrderItemType),
      resolve: async (parent) => {
        const [rows] = await pool.query(
          "SELECT * FROM order_items WHERE order_id = ?",
          [parent.id]
        );
        return rows;
      },
    },
    created_at: { type: GraphQLString },
    updated_at: { type: GraphQLString },
    // resolve user from auth-service
    user: {
      type: UserType,
      resolve: async (parent) => {
        try {
          const query = `
            query($id: ID!) {
              user(id: $id) {
```

```
                id
                username
                email
                role
              }
            }
          `;
          const response = await axios.post(process.env.AUTH_SERVICE_URL, {
            query,
            variables: { id: parent.user_id },
          });
          return response.data.data.user;
        } catch (err) {
          console.error("Failed to fetch user:", err);
          return null;
        }
      },
    },
  }),
});

// --- Root Query ---
const RootQuery = new GraphQLObjectType({
  name: "RootQueryType",
  fields: {
    myOrders: {
      type: new GraphQLList(OrderType),
      resolve: async (_, __, context) => {
        if (!context.user) throw new Error("Unauthorized");
        const [rows] = await pool.query(
          "SELECT * FROM orders WHERE user_id = ?",
          [context.user.id]
        );
        return rows;
      },
    },
    orders: {
      type: new GraphQLList(OrderType),
      resolve: async (_, __, context) => {
        if (!context.user) throw new Error("Unauthorized");
        const [rows] = await pool.query("SELECT * FROM orders");
        return rows;
      },
    },
    healthz: {
      type: GraphQLString,
      resolve: () =>
        `Order Service is healthy!! ${BUILD_DATE} #${VERSION}`,
    },
  },
});

// --- Mutations ---
const Mutation = new GraphQLObjectType({
  name: "Mutation",
  fields: {
    createOrder: {
      type: OrderType,
      args: {
        items: { type: new GraphQLList(OrderItemInputType) },
      },
      resolve: async (_, { items }, context) => {
```

```
        if (!context.user) throw new Error("Unauthorized");
        const conn = await pool.getConnection();
        let totalAmount = 0;

        try {
          await conn.beginTransaction();

          // fetch product info and calculate total
          for (const item of items) {
            const query = `
              query($id: ID!) {
                product(id: $id) {
                  id
                  price
                  stock
                }
              }
            `;
            const response = await axios.post(process.env.PRODUCT_SERVICE_URL, {
              query,
              variables: { id: item.product_id },
            });
            const product = response.data.data.product;
            if (!product) throw new Error(`Product ${item.product_id} not found`);
            if (product.stock < item.quantity)
              throw new Error(`Not enough stock for product ${item.product_id}`);

            item.unit_price = product.price;
            totalAmount += product.price * item.quantity;
          }

          // insert order
          const [orderResult] = await conn.query(
            "INSERT INTO orders (user_id, total_amount) VALUES (?, ?)",
            [context.user.id, totalAmount]
          );
          const orderId = orderResult.insertId;

          // insert order_items
          for (const item of items) {
            await conn.query(
              "INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES (?,
                  ?, ?, ?)",
              [orderId, item.product_id, item.quantity, item.unit_price]
            );
          }

          await conn.commit();
          return {
            id: orderId,
            user_id: context.user.id,
            total_amount: totalAmount,
            status: "pending",
          };
        } catch (err) {
          await conn.rollback();
          throw err;
        } finally {
          conn.release();
        }
      },
    },
```

```
    updateOrderStatus: {
      type: OrderType,
      args: {
        orderId: { type: GraphQLID },
        status: { type: GraphQLString },
      },
      resolve: async (_, { orderId, status }, context) => {
        // 1. Authorization check
        if (!context.user) throw new Error("Unauthorized");

        // 2. Validate status
        const validStatuses = ["pending", "paid", "shipped", "cancelled"];
        if (!validStatuses.includes(status)) {
          throw new Error(`Invalid status: ${status}`);
        }

        // 3. Update DB
        const [result] = await pool.query(
          "UPDATE orders SET status = ?, updated_at = CURRENT_TIMESTAMP WHERE id = ?",
          [status, orderId]
        );

        if (result.affectedRows === 0) {
          throw new Error(`Order with ID ${orderId} not found`);
        }

        // 4. Return updated order
        const [rows] = await pool.query("SELECT * FROM orders WHERE id = ?", [
          orderId,
        ]);

        return rows[0];
      },
    },

  },
});

export default new GraphQLSchema({
  query: RootQuery,
  mutation: Mutation,
});
```

6.5. Vérification des jobs

L’intégration et le déploiement de chaque microservice sont automatisés avec GitHub Actions. Cet outil permet de définir des *workflows* déclenchés par des événements sur le dépôt, comme un *push*, une *pull request* ou une planification horaire. Chaque *workflow* est composé de tâches (*jobs*), qui sont des séries d’étapes s’exécutant dans un environnement défini et qui peuvent inclure des tests, des constructions et des déploiements.

Pour accéder à l’interface de GitHub Actions, il faut se connecter à son compte GitHub, ouvrir le dépôt du projet, puis cliquer sur l’onglet **Actions** dans la barre supérieure. Cet onglet affiche la liste des *workflows* configurés dans le dépôt et de leurs exécutions (figure 25).

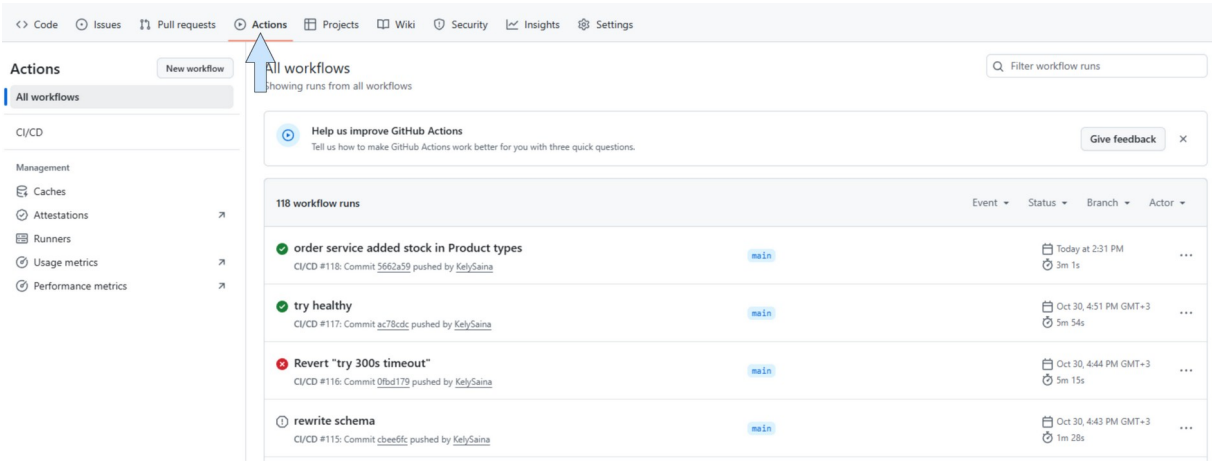


Figure 25 : Onglet Actions du dépôt GitHub

En sélectionnant une exécution, il est possible de consulter son statut (réussie, en échec ou en cours) ainsi que l'enchaînement de ses tâches. Cette interface permet de suivre l'historique des exécutions et de vérifier la qualité et la fiabilité des tâches. Les figures 26 et 27 présentent respectivement une exécution réussie et une exécution en échec lors de la tâche de déploiement.

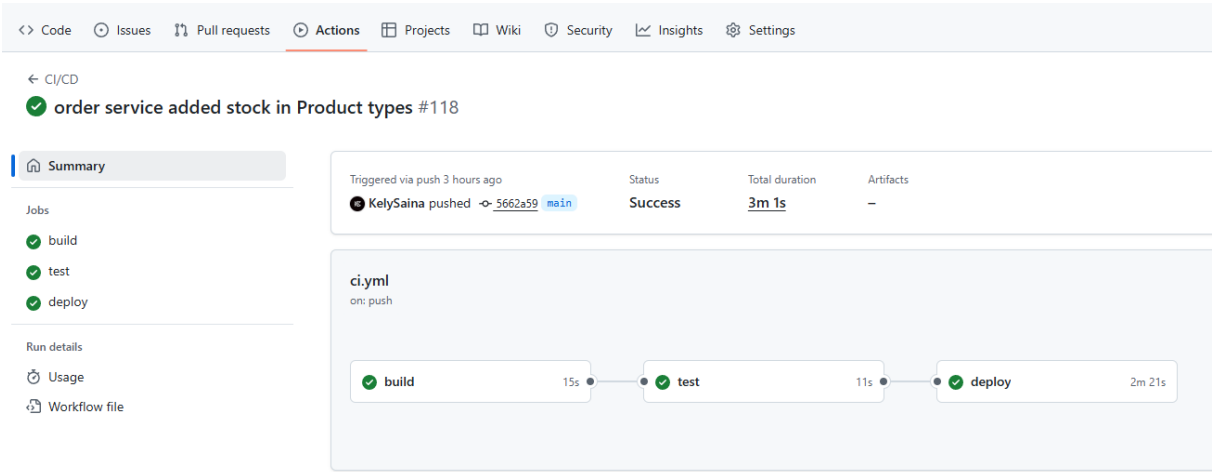


Figure 26 : Exécution réussie du workflow CI/CD

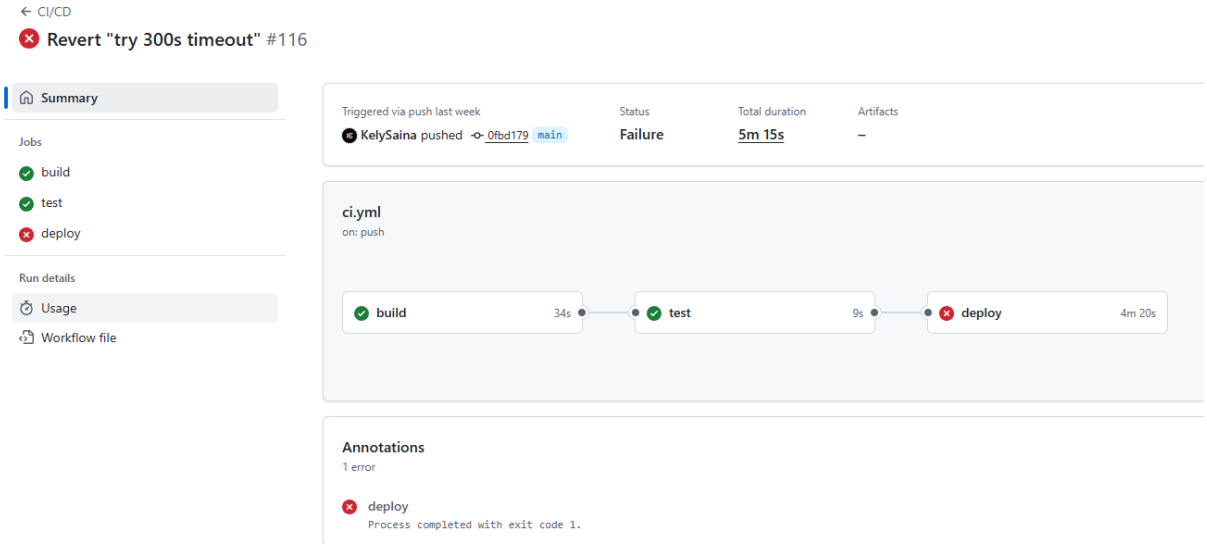


Figure 27 : Exécution du workflow en échec lors du déploiement

Pour une exécution donnée, il est possible d'ouvrir le détail des tâches et de leurs étapes. Chaque étape peut être inspectée individuellement pour vérifier qu'elle s'est exécutée correctement, consulter les journaux ou diagnostiquer les éventuelles erreurs (figure 28). Cette fonctionnalité est essentielle pour assurer la robustesse et la continuité des *workflows* d'intégration et de déploiement continus.

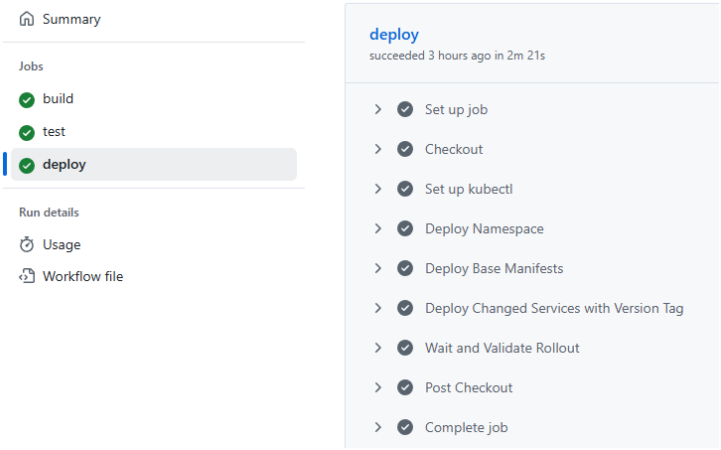


Figure 28 : Détail des étapes d'une tâche de déploiement réussie

La figure 29 montre les journaux d'une tâche de déploiement en échec : le déploiement de `product-service` n'a pas abouti dans le délai imparti, et le pipeline a automatiquement rétabli l'image précédente du service (`v-2896`) avant de signaler l'échec. Le service est ainsi resté disponible malgré la défaillance de la nouvelle version.

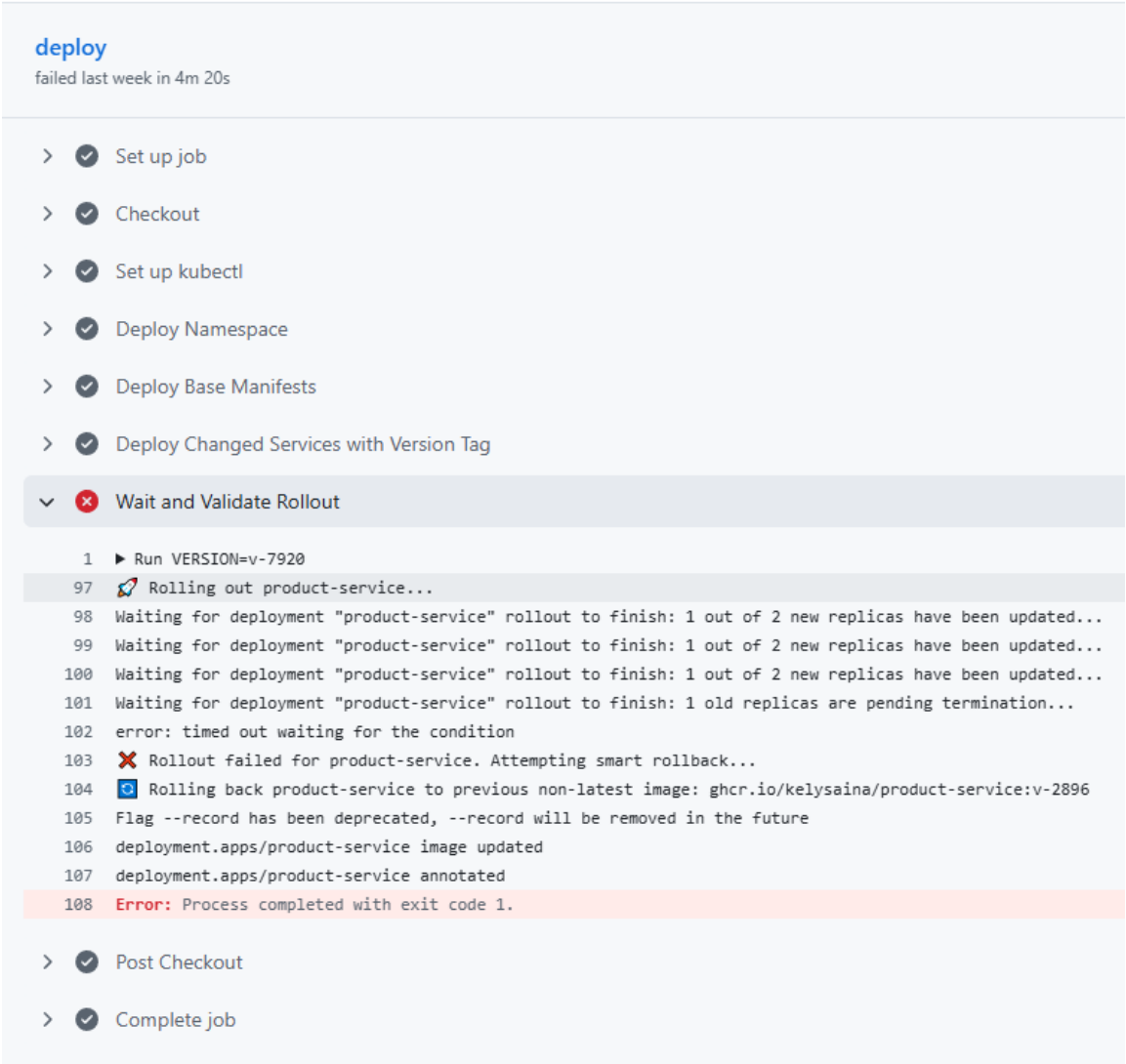


Figure 29 : Journaux d'un déploiement en échec suivi d'un retour arrière automatique

Chapitre 7. Validations et perspectives

La validation constitue une étape essentielle du projet. Elle permet de vérifier la conformité des composants déployés avec les objectifs initiaux, ainsi que la stabilité et la fiabilité de l'environnement. Dans le cadre de cette étude, la validation a porté sur plusieurs volets : la mise en place du cluster Kubernetes, l'orchestration des microservices, la mise à l'échelle automatique, le fonctionnement du pipeline CI/CD et la communication entre les différents services.

7.1. Validation du cluster Kubernetes

La première phase de validation se concentre sur le bon fonctionnement du cluster Kubernetes. Celui-ci repose sur une architecture composée d'un nœud *master* et de deux nœuds *workers*, déployés à l'aide de Vagrant et VirtualBox. L'objectif est de confirmer la capacité du cluster à orchestrer et distribuer les conteneurs de manière dynamique, tout en assurant la haute disponibilité et la tolérance aux pannes.

L'exécution de la commande `kubectl get nodes` permet de vérifier que les trois nœuds apparaissent dans l'état `Ready`. L'option `-o wide` affiche en plus les adresses associées à chaque nœud du cluster (figure 30).

```
$ kubectl get nodes -o wide
```

```
vagrant@k8s-master:~$ kubectl get nodes -o wide
NAME          STATUS   ROLES           AGE   VERSION    INTERNAL-IP
TAINER-RUNTIME
k8s-master    Ready    control-plane   31d   v1.30.14   192.168.56.10
tainerd://1.7.27
k8s-worker1   Ready    <none>          31d   v1.30.14   192.168.56.20
tainerd://1.7.27
k8s-worker2   Ready    <none>          31d   v1.30.14   192.168.56.21
```

Figure 30 : État des nœuds du cluster Kubernetes

Une étape essentielle consiste à confirmer que le réseau de pods est fonctionnel et que la communication entre les différents nœuds se fait sans obstacle. Chaque pod doit pouvoir échanger avec tous les autres pods du cluster, même lorsqu'ils s'exécutent sur des nœuds distincts. Cette vérification passe par l'observation de l'état des pods du *namespace* `kube-system`, notamment ceux du réseau Calico (figure 31).

```
$ kubectl get pods -n kube-system
```

```
vagrant@k8s-master:~$ kubectl get pods -n kube-system
NAME                                       READY   STATUS    RESTARTS
calico-kube-controllers-5b9b456c66-55q28   1/1     Running   11 (142m ago)
calico-node-27smj                          1/1     Running   11 (142m ago)
calico-node-nn22h                          1/1     Running   11 (137m ago)
calico-node-wm2gg                          1/1     Running   11 (135m ago)
coredns-55cb58b774-6xcj5                   1/1     Running   18 (142m ago)
coredns-55cb58b774-nf2qm                   1/1     Running   18 (142m ago)
etcd-k8s-master                            1/1     Running   18 (142m ago)
kube-apiserver-k8s-master                  1/1     Running   52 (140m ago)
kube-controller-manager-k8s-master         1/1     Running   134 (142m ago)
kube-proxy-6lnl9                           1/1     Running   19 (135m ago)
kube-proxy-nm7td                           1/1     Running   19 (137m ago)
kube-proxy-pvvj6                           1/1     Running   18 (142m ago)
kube-scheduler-k8s-master                  1/1     Running   141 (142m ago)
metrics-server-788ffc7bf4-trnhq            1/1     Running   12 (132m ago)
vagrant@k8s-master:~$
```

Figure 31 : Pods système du cluster dans le namespace kube-system

Le plan de contrôle du cluster constitue également un élément critique à valider. Ses composants principaux, tels que le serveur d'API, l'ordonnanceur (*scheduler*) et le gestionnaire de contrôleurs (*controller manager*), doivent être actifs et capables de gérer la planification des pods et la répartition des ressources (figure 32).

```
$ kubectl get pods -n kube-system -l tier=control-plane
```

```
vagrant@k8s-master:~$ kubectl get pods -n kube-system -l tier=control-plane
NAME                                READY   STATUS    RESTARTS          AGE
etcd-k8s-master                     1/1     Running   18 (143m ago)     31d
kube-apiserver-k8s-master           1/1     Running   52 (141m ago)     31d
kube-controller-manager-k8s-master  1/1     Running   134 (143m ago)    31d
kube-scheduler-k8s-master           1/1     Running   141 (143m ago)    31d
vagrant@k8s-master:~$
```

Figure 32 : Composants du plan de contrôle

Le DNS interne du cluster, assuré par CoreDNS, est également vérifié. La résolution DNS permet aux services et aux pods de se localiser mutuellement (figure 33).

```
$ kubectl get pods -n kube-system -l k8s-app=kube-dns
```

```
vagrant@k8s-master:~$ kubectl get pods -n kube-system -l k8s-app=kube-dns
NAME                       READY   STATUS    RESTARTS        AGE
coredns-55cb58b774-6xcj5   1/1     Running   18 (155m ago)   31d
coredns-55cb58b774-nf2qm   1/1     Running   18 (155m ago)   31d
vagrant@k8s-master:~$
```

Figure 33 : Pods CoreDNS assurant la résolution DNS interne

Pour suivre l'utilisation des ressources et permettre la mise à l'échelle automatique des services, Kubernetes s'appuie sur le composant *Metrics Server* [15], déployé dans le *namespace* `kube-system`, qui collecte les données d'utilisation du CPU et de la mémoire de tous les nœuds et pods du cluster. Pour vérifier le bon fonctionnement des métriques, on exécute :

```
$ kubectl get pods -n kube-system | grep metrics-server
$ kubectl top nodes
$ kubectl top pods
```

La première commande permet de s'assurer que le pod `metrics-server` est dans l'état `Running`. Les deux suivantes affichent l'utilisation actuelle des ressources par les nœuds et les pods, confirmant que le cluster collecte correctement les métriques nécessaires au suivi et à la mise à l'échelle automatique (figure 34).

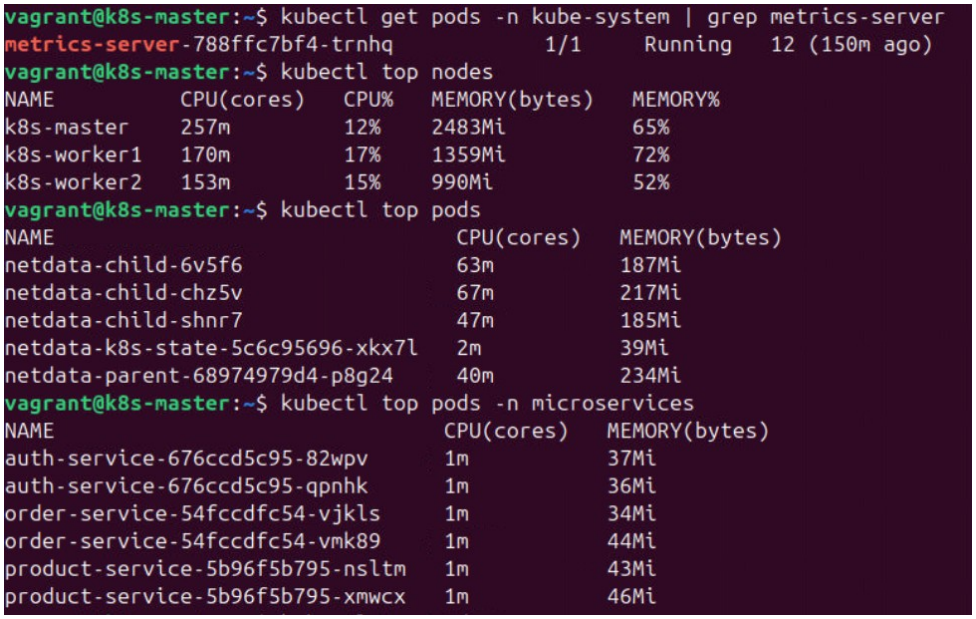


Figure 34 : Collecte des métriques des nœuds et des pods

7.2. Validation des manifests des microservices

7.2.1. Notion de namespace

Dans Kubernetes, un *namespace* constitue un espace logique qui permet d'isoler et d'organiser les ressources au sein d'un même cluster. Chaque *namespace* peut contenir des pods, des services, des déploiements et d'autres objets Kubernetes. L'utilisation des *namespaces* facilite la gestion des ressources, la séparation des environnements (développement, test, production) et l'application de politiques spécifiques à un ensemble de services. Grâce à cette isolation, plusieurs équipes peuvent travailler simultanément dans le même cluster sans interférer entre elles.

La validation des microservices consiste à vérifier que chaque service déployé fonctionne correctement, communique avec les autres services et respecte les configurations prévues. On commence par lister les *namespaces* du cluster, parmi lesquels figure le *namespace* `microservices` (figure 35).

```
$ kubectl get namespace
```

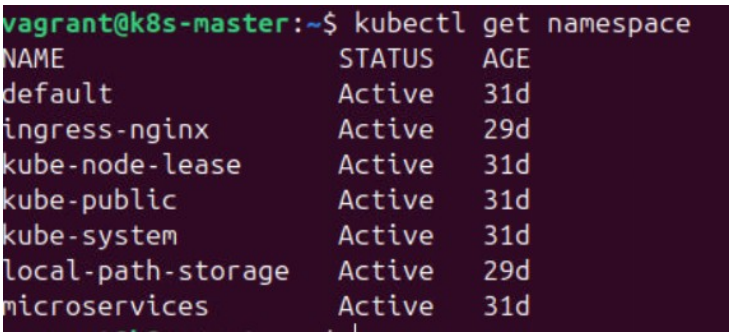


Figure 35 : Namespaces du cluster

7.2.2. Validation des pods et des services

La première étape de la validation des microservices consiste à vérifier l'état des pods dans le *namespace* dédié. Les pods représentent l'unité de base d'exécution dans Kubernetes et contiennent les conteneurs qui exécutent les microservices. Il est donc essentiel de s'assurer que chaque pod est correctement déployé et dans l'état `Running`, garantissant que les services associés peuvent fonctionner normalement.

```
$ kubectl get pods -n microservices
$ kubectl get pods -n microservices -o wide
```

La première commande fournit le nom, le statut, le nombre de redémarrages et l'âge de chaque pod. La seconde affiche en plus le nœud sur lequel chaque pod s'exécute et son adresse IP, facilitant la détection d'éventuels problèmes d'ordonnancement ou de répartition des pods sur les nœuds du cluster. La figure 36 montre que les pods des trois microservices sont en marche et répartis sur les deux nœuds *workers*.

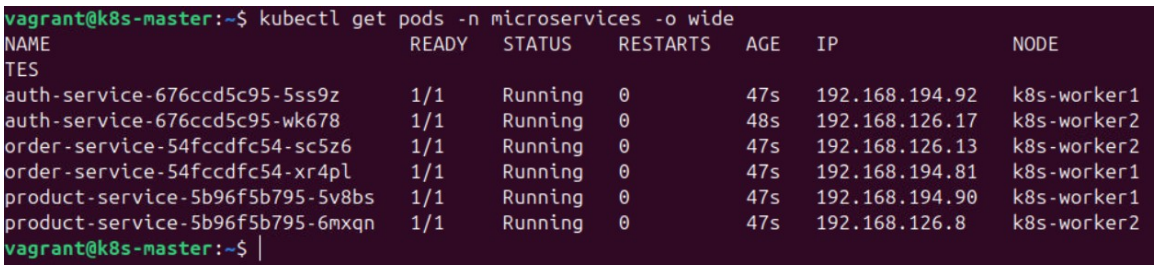


Figure 36 : Pods des microservices et leur répartition sur les nœuds

Une fois les pods validés, il est nécessaire de s'assurer que les *Services* Kubernetes fonctionnent correctement. Un *Service* agit comme une abstraction réseau permettant d'exposer un ensemble de pods via une adresse IP stable ; il assure le routage du trafic et la découverte automatique des pods, même lorsque ceux-ci sont recréés ou déplacés sur d'autres nœuds. La commande suivante affiche la liste des services, leur type, leurs ports et leurs adresses IP ; chaque service doit être correctement configuré et accessible dans le réseau interne du cluster (figure 37).

```
$ kubectl get svc -n microservices
```

```
vagrant@k8s-master:~$ kubectl get svc -n microservices
NAME              TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
auth-service      ClusterIP   10.106.182.66    <none>        4001/TCP
order-service     ClusterIP   10.111.234.207   <none>        4002/TCP
product-service   ClusterIP   10.106.129.70    <none>        4003/TCP
vagrant@k8s-master:~$
```

Figure 37 : Services ClusterIP des microservices

7.2.3. Validation de l'Ingress

Après la vérification des services internes, il est essentiel de valider la couche *Ingress*, qui permet d'exposer les microservices vers l'extérieur du cluster. L'*Ingress* agit comme un point d'entrée unique du trafic HTTP/HTTPS et redirige les requêtes vers les services internes selon les règles définies. Cette couche garantit un accès centralisé, sécurisé et contrôlé aux différentes API de l'application. Le manifest `microservices-ingress.yaml` s'appuie sur le contrôleur NGINX Ingress ; son bon fonctionnement est vérifié avec les commandes suivantes (figure 38) :

```
$ kubectl get pods -n ingress-nginx
$ kubectl get ingress -n microservices
```

```
vagrant@k8s-master:~$ kubectl get pods -n ingress-nginx
NAME                                        READY   STATUS    RESTARTS      AGE
ingress-nginx-controller-5b9fd684f8-j4qd7   1/1     Running   6 (16m ago)   50m
ingress-nginx-controller-5b9fd684f8-z9tq6   1/1     Running   6 (16m ago)   50m
vagrant@k8s-master:~$ kubectl get ingress -n microservices
NAME                     CLASS    HOSTS          ADDRESS          PORTS   AGE
microservices-graphql    <none>   myapp.local    10.105.98.137    80      35d
microservices-health     <none>   myapp.local    10.105.98.137    80      35d
microservices-ingress    <none>   myapp.local    10.105.98.137    80      35d
vagrant@k8s-master:~$
```

Figure 38 : Contrôleur NGINX Ingress et règles d'Ingress du namespace microservices

7.3. Validation de la mise à l'échelle automatique

La validation des *Horizontal Pod Autoscalers* et des métriques vise à s'assurer que le cluster est capable d'adapter automatiquement les ressources à la charge réelle des microservices. Cette étape garantit la scalabilité dynamique et la résilience du système.

Il est d'abord indispensable de confirmer que le *Metrics Server* est installé et opérationnel, puisque c'est lui qui collecte en continu les données d'utilisation du CPU et de la mémoire des pods :

```
$ kubectl get deployment metrics-server -n kube-system
$ kubectl top nodes
$ kubectl top pods -n microservices
```

Ces commandes doivent afficher la consommation de CPU et de mémoire de chaque nœud et de chaque pod ; si aucune donnée ne remonte, cela indique un problème de collecte

des métriques (figure 39).

```
vagrant@k8s-master:~$ kubectl get deployment metrics-server -n kube-system
NAME             READY   UP-TO-DATE   AVAILABLE   AGE
metrics-server   1/1     1            1           13d
vagrant@k8s-master:~$ kubectl top nodes
NAME          CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
k8s-master    259m         12%    2474Mi          64%
k8s-worker1   163m         16%    1066Mi          78%
k8s-worker2   100m         10%    744Mi           54%
vagrant@k8s-master:~$  kubectl top pods -n microservices
NAME                                CPU(cores)   MEMORY(bytes)
auth-service-7ddd79647d-6jkf2       1m           28Mi
auth-service-7ddd79647d-k6cdh       1m           27Mi
order-service-6ddb6c5ff-jbrnx       1m           29Mi
order-service-6ddb6c5ff-tfm4z       1m           29Mi
product-service-69f7486bc8-lgzkj    1m           42Mi
product-service-69f7486bc8-plfb5    1m           42Mi
```

Figure 39 : Metrics Server et consommation des ressources des microservices

Dans les résultats des commandes `kubectl top`, deux unités principales apparaissent :

- **CPU (m) :** le « m » signifie *millicores*. Kubernetes exprime la puissance de calcul en millièmes de cœur : 1000m correspond à un cœur complet, 500m à un demi-cœur et 1m à un millième de cœur. Cela permet une gestion très fine des ressources processeur attribuées à chaque pod ou conteneur.
- **Mémoire (Mi) :** le « Mi » signifie mébioctets (*mebibytes*). 1 Mi vaut 1 024 × 1 024 octets, soit environ 1,05 Mo. Kubernetes utilise cette unité binaire pour mesurer la mémoire avec plus de précision que les unités décimales classiques.

Une fois les métriques disponibles, la validation du mécanisme de mise à l'échelle automatique peut commencer. On liste les HPA du *namespace* des microservices ; chacun doit indiquer sa cible d'utilisation du CPU ainsi que le nombre actuel et les bornes de pods (figure 40).

```
$ kubectl get hpa -n microservices
```

```
vagrant@k8s-master:~$ kubectl get hpa -n microservices -w
NAME                REFERENCE                      TARGETS       MINPODS   MAXPODS   REPLICAS
auth-service-hpa    Deployment/auth-service        cpu: 1%/70%   2         10        2
order-service-hpa   Deployment/order-service       cpu: 1%/70%   2         10        2
product-service-hpa Deployment/product-service     cpu: 1%/75%   2         8         2
```

Figure 40 : Horizontal Pod Autoscalers des microservices au repos

Pour observer le comportement du HPA, on simule une charge sur un service donné, ici `auth-service`, en lançant un pod temporaire qui lui envoie des requêtes en continu :

```
$ kubectl run -i --tty load-generator --image=busybox -- /bin/sh
# while true; do wget -q -O- http://auth-service.microservices.svc.cluster.local:4001; done
```

Pendant la montée en charge, on surveille l'évolution automatique du nombre de pods :

```
$ kubectl get hpa -n microservices -w
$ kubectl get pods -n microservices
```

```
vagrant@k8s-master:~$ kubectl get hpa -n microservices -w
NAME                  REFERENCE                         TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
auth-service-hpa      Deployment/auth-service           cpu: 83%/70%    2         10        2          15d
order-service-hpa     Deployment/order-service          cpu: 1%/70%     2         10        2          15d
product-service-hpa   Deployment/product-service        cpu: 1%/75%     2         8         2          15d
auth-service-hpa      Deployment/auth-service           cpu: 81%/70%    2         10        3          15d
auth-service-hpa      Deployment/auth-service           cpu: 62%/70%    2         10        3          15d
auth-service-hpa      Deployment/auth-service           cpu: 58%/70%    2         10        3          15d
auth-service-hpa      Deployment/auth-service           cpu: 56%/70%    2         10        3          15d
auth-service-hpa      Deployment/auth-service           cpu: 54%/70%    2         10        3          15d
```

Figure 41 : Réaction du HPA de auth-service à la montée en charge

```
vagrant@k8s-master:~$ kubectl top pods -n microservices
NAME                                CPU(cores)   MEMORY(bytes)
auth-service-566b88c6d5-rkdg4       76m          63Mi
auth-service-566b88c6d5-w2d8q       86m          45Mi
order-service-77d55d9c6-gslmr       1m           31Mi
order-service-77d55d9c6-xdtgd       1m           31Mi
product-service-6bdc8854d5-9zsvf    1m           48Mi
product-service-6bdc8854d5-jvnzd    1m           53Mi
vagrant@k8s-master:~$ kubectl get pods -n microservices -w
NAME                                READY   STATUS    RESTARTS   AGE
auth-service-566b88c6d5-rkdg4       1/1     Running   0          24m
auth-service-566b88c6d5-vxbv7       1/1     Running   0          25s
auth-service-566b88c6d5-w2d8q       1/1     Running   0          24m
order-service-77d55d9c6-gslmr       1/1     Running   0          21m
order-service-77d55d9c6-xdtgd       1/1     Running   0          21m
product-service-6bdc8854d5-9zsvf    1/1     Running   0          24m
product-service-6bdc8854d5-jvnzd    1/1     Running   0          24m
```

Figure 42 : Consommation des pods et création d'un nouveau pod de auth-service

Les figures 41 et 42 montrent que l'utilisation du CPU de `auth-service` dépasse le seuil de 70 % (jusqu'à 83 %) et qu'un troisième pod est alors créé automatiquement : le HPA réagit correctement à la charge. Une fois la charge arrêtée, le nombre de pods redescend progressivement au niveau initial, confirmant la capacité d'autorégulation du cluster. Ainsi, la validation des HPA et des métriques garantit que l'orchestration Kubernetes assure une allocation optimale des ressources, tout en maintenant les performances et la stabilité des microservices déployés.

7.4. Validation d'un déploiement de bout en bout

Pour valider le pipeline sur un changement réel d'un service, la procédure est la suivante :

- identifier la version actuelle des services ;
- effectuer un changement sur un service ;
- pousser les changements vers GitHub ;
- laisser GitHub Actions effectuer l'intégration de ce service spécifique ;

- laisser GitHub Actions déployer le service sur le cluster ;
- laisser Cypress lancer les tests automatiques après le déploiement.

La version actuelle des services est identifiée grâce à la requête `healthz` de chaque service, interrogée en continu à travers l'*Ingress* par le script suivant :

```
while true; do
  echo "Request at $(date):"

  curl -s -X POST http://myapp.local:30080/auth \
       -H "Content-Type: application/json" \
       -d '{"query":"{ healthz }"}'
  echo -e "\n--------------------------\n"

  curl -s -X POST http://myapp.local:30080/products \
       -H "Content-Type: application/json" \
       -d '{"query":"{ healthz }"}'
  echo -e "\n--------------------------\n"

  curl -s -X POST http://myapp.local:30080/orders \
       -H "Content-Type: application/json" \
       -d '{"query":"{ healthz }"}'
  echo -e "\n--------------------------\n"

  sleep 5
done
```

La figure 43 montre que les trois services fonctionnent correctement sous la même version `v-3169`.

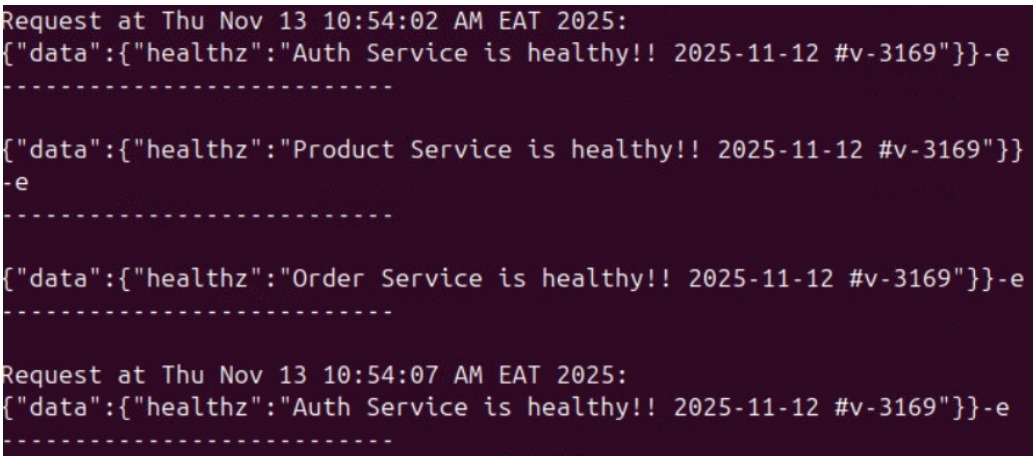


Figure 43 : Réponses healthz des services avant le changement (version v-3169)

Un changement est ensuite apporté à la réponse `healthz` du service d'authentification, qui affichera trois points d'exclamation au lieu de deux. Il s'agit ici d'une modification volontairement mineure, mais le même processus s'applique à un changement majeur, comme l'ajout d'une nouvelle mutation ou d'un correctif.

```
// avant
healthz: {
  type: GraphQLString,
  resolve: () =>
    `Auth Service is healthy!! ${BUILD_DATE} #${VERSION}`,
},

// après
healthz: {
  type: GraphQLString,
  resolve: () =>
    `Auth Service is healthy!!! ${BUILD_DATE} #${VERSION}`,
},
```

Pour remonter ce changement, on effectue un *commit* avec le message « healthz update », puis un *push* vers le dépôt GitHub (figure 44).

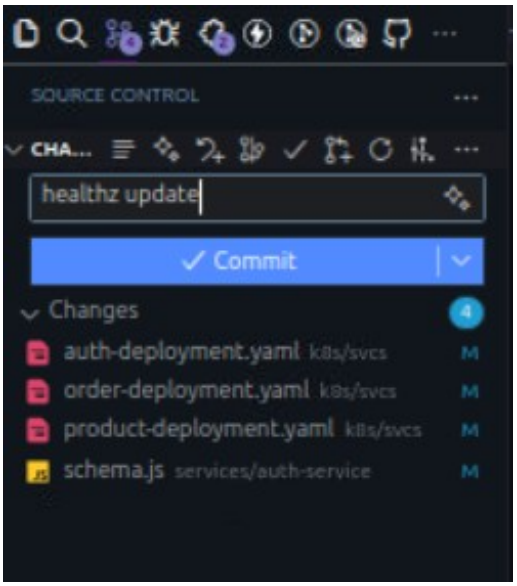


Figure 44 : Commit du changement depuis l'éditeur de code

Sur GitHub Actions, on retrouve facilement le *workflow* déclenché par ce *push* grâce au message du *commit* (figure 45). Chaque étape du *workflow* est accessible pour plus de détails en cliquant dessus.

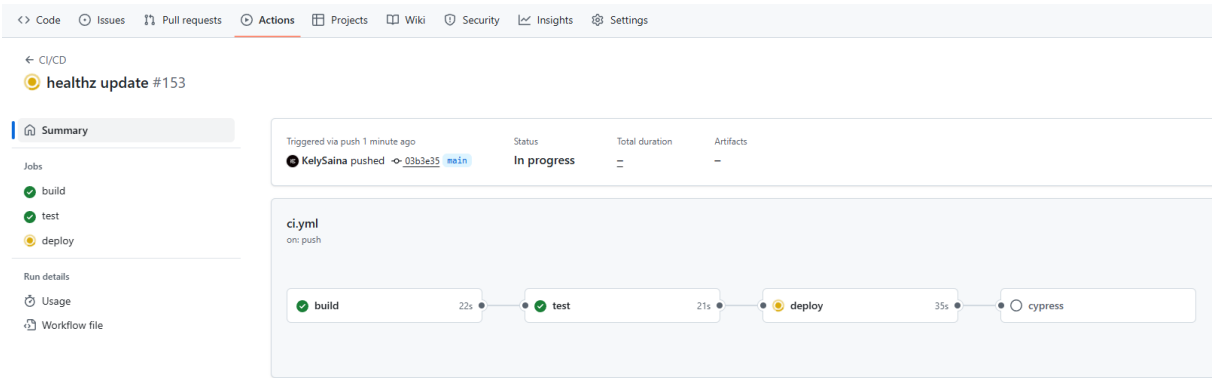


Figure 45 : Workflow déclenché par le push, en cours d'exécution

Le pipeline prend alors le relais pour effectuer l'intégration du service modifié. Seul `auth-service` ayant changé, seule son image est construite et poussée, avec la nouvelle

version `v-2058` (figure 46). L'image est ensuite validée par la tâche de test, qui vérifie que le conteneur démarre correctement (figure 47).

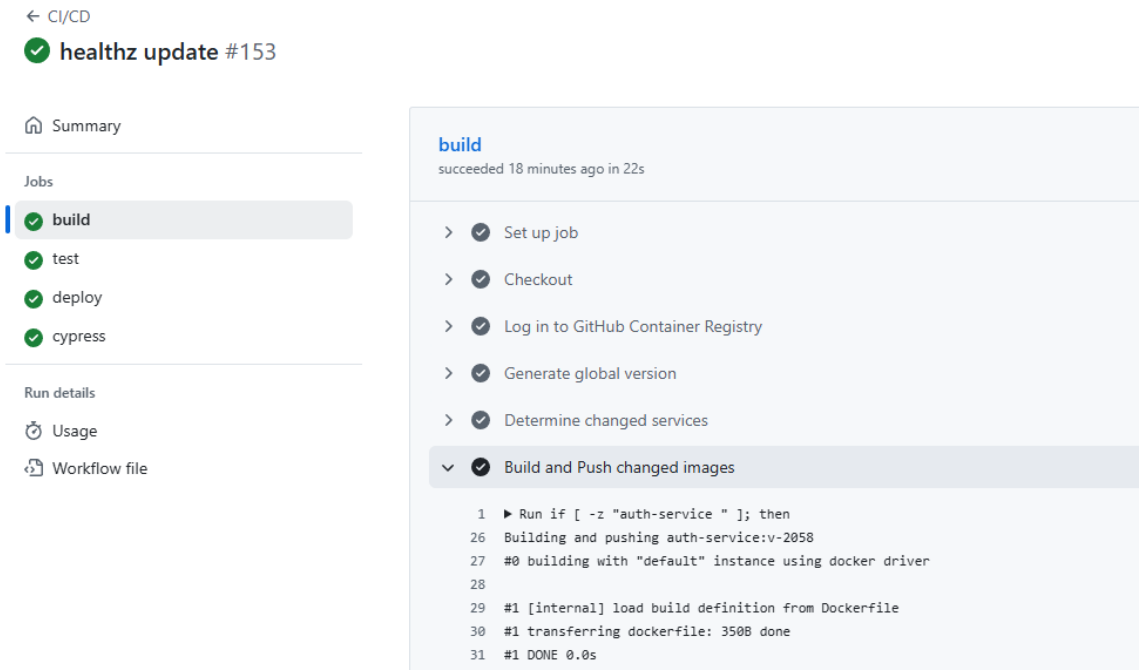


Figure 46 : Tâche build : construction de l'image de auth-service en version v-2058

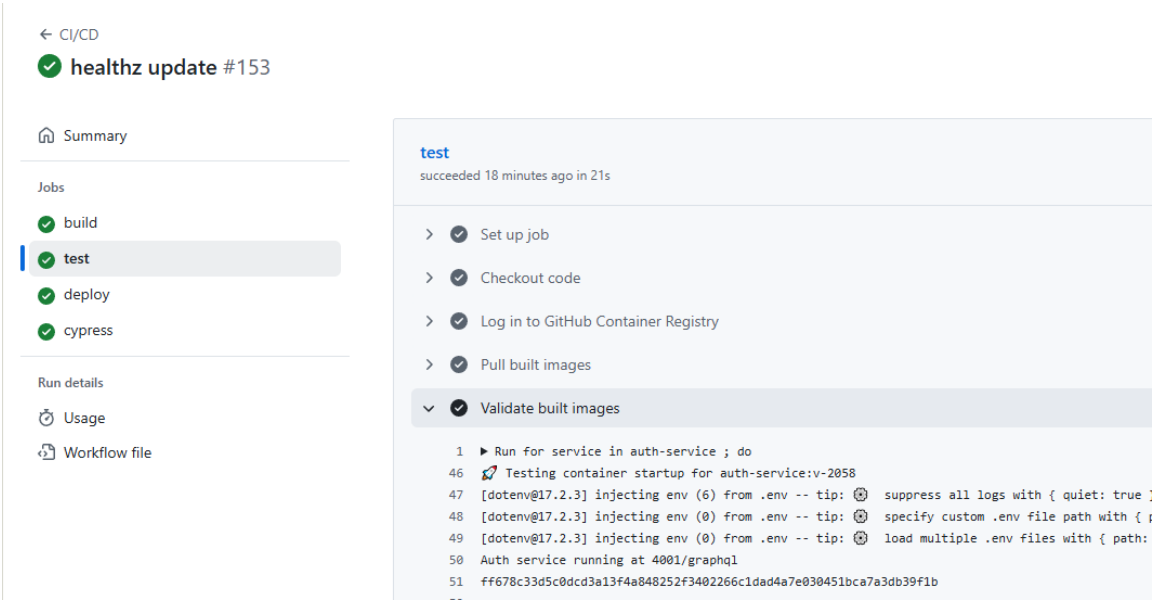


Figure 47 : Tâche test : validation du démarrage du conteneur

Une fois ces étapes validées, GitHub Actions déploie automatiquement le service sur le cluster Kubernetes : la nouvelle version est déployée progressivement, puis la disponibilité des pods est vérifiée à deux reprises avant de considérer le déploiement comme réussi (figure 48).

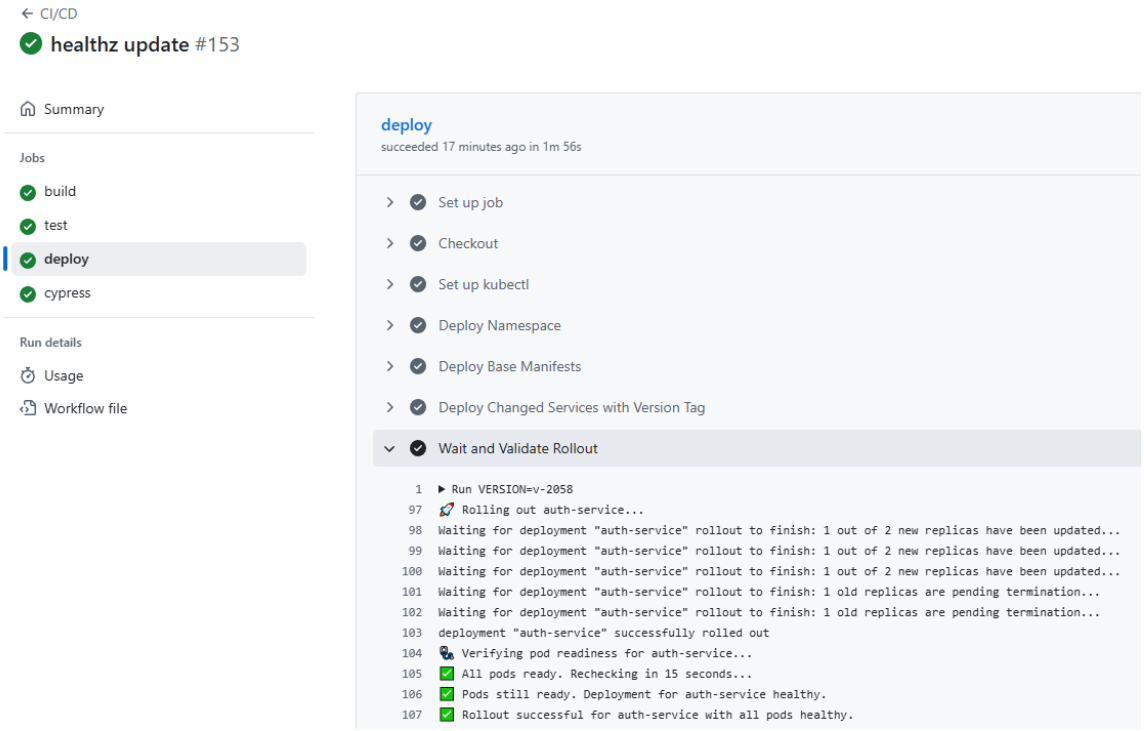


Figure 48 : Tâche deploy : déploiement progressif et validation des pods

Enfin, pour garantir le bon fonctionnement après le déploiement, Cypress lance les tests automatiques de bout en bout. Ces tests vérifient que l'application fonctionne correctement, que les interactions entre les microservices sont maintenues et que les fonctionnalités répondent aux exigences attendues. Les quinze tests des cinq scénarios (panier, connexion, commandes, gestion des commandes et gestion des produits) sont passés avec succès (figure 49).

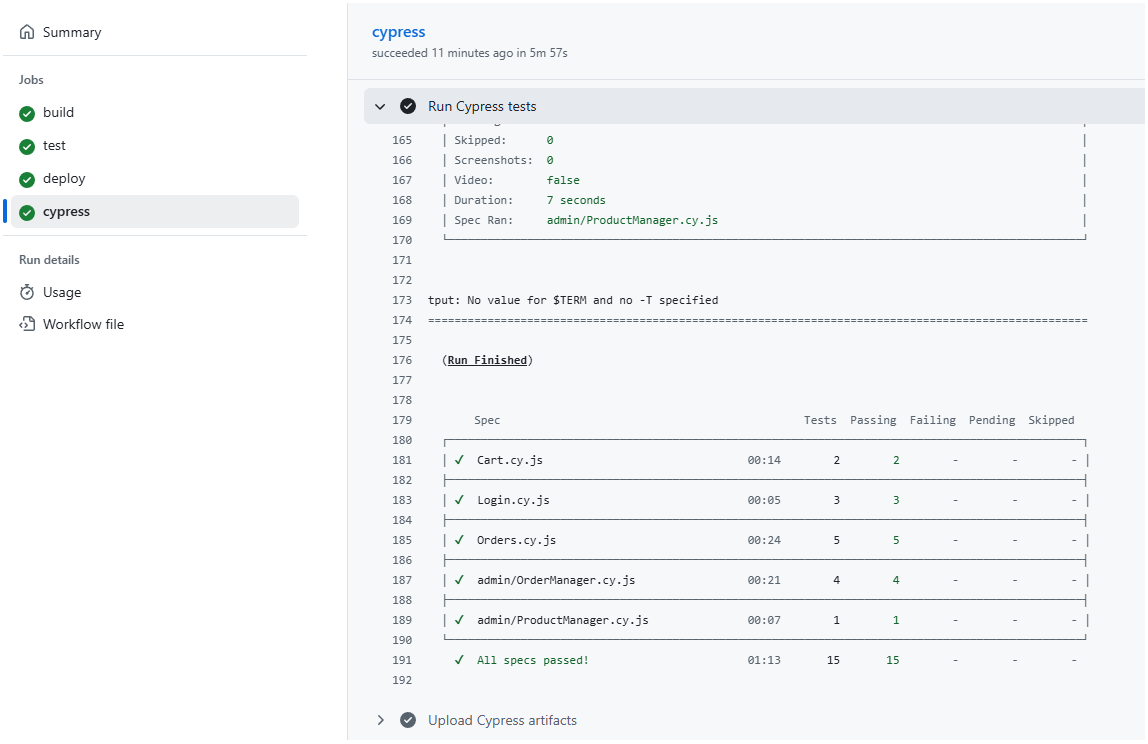


Figure 49 : Tâche cypress : résultats des tests de bout en bout

Une fois le service déployé, la nouvelle version apparaît dans les vérifications, confirmant que le déploiement a été correctement appliqué et que le service fonctionne avec les modifications apportées : la version de `auth-service` devient `v-2058`, tandis que les autres services, non modifiés, restent en version `v-3169` (figure 50).

```
Request at Thu Nov 13 11:37:32 AM EAT 2025:
{"data":{"healthz":"Auth Service is healthy!!! 2025-11-13 #v-2058"}}-e
----------------------------

{"data":{"healthz":"Product Service is healthy!! 2025-11-12 #v-3169"}}
-e
----------------------------

{"data":{"healthz":"Order Service is healthy!! 2025-11-12 #v-3169"}}-e
----------------------------

Request at Thu Nov 13 11:37:37 AM EAT 2025:
{"data":{"healthz":"Auth Service is healthy!!! 2025-11-13 #v-2058"}}-e
----------------------------

{"data":{"healthz":"Product Service is healthy!! 2025-11-12 #v-3169"}}
-e
----------------------------
```

Figure 50 : Réponses healthz après le déploiement (auth-service en version v-2058)

En cas de problème détecté lors des tests post-déploiement ou de dysfonctionnement en production, il est possible de procéder à un retour arrière, rétablissant rapidement la version stable précédente du service afin de garantir la continuité et la fiabilité de l'application. Durant tout le processus de déploiement d'une nouvelle version, la continuité des services est assurée

par la gestion des pods et des réplicas dans le cluster Kubernetes : les anciennes instances continuent de répondre aux requêtes jusqu'à ce que les nouvelles soient opérationnelles et prêtes à prendre le relais.

En somme, l'ensemble du processus de déploiement met en évidence la scalabilité et la haute disponibilité offertes par l'architecture microservices orchestrée avec Kubernetes. Chaque service peut être déployé et mis à jour de manière indépendante, sans impacter les autres, ce qui garantit la continuité du fonctionnement. La gestion automatique des réplicas, la supervision des ressources et la possibilité de retour arrière assurent que les services restent fiables, performants et facilement évolutifs, tout en permettant des mises à jour rapides et sûres.

7.5. Perspectives d'amélioration

Les validations menées montrent que les objectifs du projet sont atteints. Plusieurs axes d'amélioration peuvent néanmoins être envisagés pour aller plus loin :

- **Supervision et alertes avancées :** mettre en place des métriques applicatives et des alertes permettant de surveiller en temps réel la santé des services et d'optimiser l'allocation des ressources ;
- **Pratiques DevSecOps :** intégrer au pipeline des contrôles de sécurité, comme l'analyse des vulnérabilités des images et des dépendances, afin de garantir la protection des données et des services face aux menaces potentielles ;
- **Traçabilité des versions :** remplacer l'option `--record`, désormais dépréciée par Kubernetes comme le signalent les journaux du pipeline, par un mécanisme d'annotation dédié pour l'historique des déploiements ;
- **Migration progressive de GESCOM :** appliquer cette démarche aux modules de l'application GESCOM (clients, articles, facturations), en les extrayant un à un du monolithe vers des microservices indépendants.

CONCLUSION GÉNÉRALE

La mise en œuvre de ce projet a permis d'illustrer l'importance de l'automatisation et de l'orchestration dans le développement moderne d'applications basées sur une architecture microservices. La conception d'un pipeline CI/CD a assuré l'intégration continue et la livraison rapide et fiable des services, tout en réduisant les risques d'erreurs liées aux interventions manuelles. L'utilisation de Docker pour la conteneurisation et de Kubernetes pour l'orchestration a favorisé la portabilité, la scalabilité et la haute disponibilité de l'ensemble des microservices, contribuant à une meilleure gestion des ressources et à une maintenance simplifiée.

Les tests automatisés intégrés au pipeline ont permis de valider le bon fonctionnement des services avant et après leur déploiement, garantissant la cohérence et la qualité des fonctionnalités mises en production. Le mécanisme de retour arrière automatique a par ailleurs démontré sa capacité à préserver la disponibilité d'un service lorsqu'une nouvelle version échoue. L'ensemble de ces pratiques a transformé le cycle de vie de l'application, le rendant plus modulaire, fiable et sûr.

En perspective, plusieurs axes d'amélioration peuvent être envisagés. L'implémentation de métriques et d'alertes avancées permettrait de surveiller en temps réel la santé des services et d'optimiser l'allocation des ressources. L'intégration de pratiques DevSecOps renforcerait la sécurité du pipeline, garantissant la protection des données et des services face aux menaces potentielles. Enfin, cette démarche pourrait être appliquée à la migration progressive des modules de l'application GESCOM vers des microservices.

Ainsi, ce projet constitue une base solide pour le déploiement et la gestion de systèmes modulaires modernes, et ouvre la voie à une amélioration continue des processus de développement et d'exploitation des applications en microservices.

BIBLIOGRAPHIE

[1] NEWMAN Sam. *Building Microservices: Designing Fine-Grained Systems*. 2e édition. Sebastopol : O'Reilly Media, 2021.

[2] HUMBLE Jez, FARLEY David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010.

[3] KIM Gene, HUMBLE Jez, DEBOIS Patrick, WILLIS John. *The DevOps Handbook*. Portland : IT Revolution Press, 2016.

[4] BURNS Brendan, BEDA Joe, HIGHTOWER Kelsey, EVENSON Lachlan. *Kubernetes: Up and Running*. 3e édition. Sebastopol : O'Reilly Media, 2022.

[5] CHACON Scott, STRAUB Ben. *Pro Git*. 2e édition. New York : Apress, 2014.

WEBOGRAPHIE

[6] https://martinfowler.com/articles/microservices.html, *Microservices: a definition of this new architectural term*, FOWLER Martin, LEWIS James, consulté le 25/08/2025

[7] https://git-scm.com/doc, Documentation de Git, consulté le 01/09/2025

[8] https://docs.docker.com/, Documentation de Docker, consulté le 01/09/2025

[9] https://docs.github.com/actions, Documentation de GitHub Actions, consulté le 15/09/2025

[10] https://kubernetes.io/docs/, Documentation de Kubernetes, consulté le 15/09/2025

[11] https://developer.hashicorp.com/vagrant/docs, Documentation de Vagrant, consulté le 22/08/2025

[12] https://www.virtualbox.org/manual/, Manuel de VirtualBox, consulté le 22/08/2025

[13] https://docs.tigera.io/calico/latest/about/, Documentation de Calico, consulté le 25/08/2025

[14] https://kubernetes.github.io/ingress-nginx/, Documentation du contrôleur NGINX Ingress, consulté le 20/10/2025

[15] https://github.com/kubernetes-sigs/metrics-server, Documentation de Metrics Server, consulté le 20/10/2025

[16] https://graphql.org/learn/, Documentation de GraphQL, consulté le 06/10/2025

[17] https://docs.cypress.io/, Documentation de Cypress, consulté le 03/11/2025

[18] https://yaml.org/spec/1.2.2/, Spécification YAML 1.2.2, consulté le 15/09/2025

GLOSSAIRE

Calico : plugin réseau (CNI) de Kubernetes qui assure la communication entre les pods répartis sur différents nœuds et l'application des politiques réseau.

Cluster : ensemble de machines (nœuds) gérées comme un tout par Kubernetes pour exécuter des applications conteneurisées.

Conteneur : environnement d'exécution isolé et léger qui embarque une application avec toutes ses dépendances.

Deployment : ressource Kubernetes décrivant l'état souhaité d'une application (image, nombre de réplicas) et gérant ses mises à jour progressives.

GraphQL : langage de requête pour API permettant au client de demander précisément les données dont il a besoin.

Healthz : point de contrôle de santé exposé par un service pour indiquer qu'il fonctionne, ici avec sa version et sa date de construction.

Horizontal Pod Autoscaler (HPA) : ressource Kubernetes qui ajuste automatiquement le nombre de pods d'un déploiement selon des métriques observées.

Hotfix : correctif urgent appliqué directement sur la version en production.

Image Docker : modèle immuable à partir duquel des conteneurs sont créés ; elle contient le code, l'environnement d'exécution et les dépendances.

Ingress : ressource Kubernetes qui expose des services HTTP/HTTPS vers l'extérieur du cluster et en définit les règles de routage.

Job (tâche) : ensemble d'étapes d'un workflow exécutées sur un même runner.

Manifest : fichier, généralement en YAML, décrivant de manière déclarative une ressource Kubernetes.

Microservice : service autonome, responsable d'une fonctionnalité métier précise, déployable indépendamment des autres.

Monolithe : application dont toutes les fonctionnalités sont regroupées dans un seul bloc déployé d'un seul tenant.

Namespace : espace logique isolant un ensemble de ressources au sein d'un cluster Kubernetes.

Nœud (node) : machine physique ou virtuelle faisant partie d'un cluster Kubernetes ; le nœud master héberge le plan de contrôle, les workers exécutent les pods.

Orchestration : automatisation du déploiement, de la mise à l'échelle, de la mise en réseau et de la supervision des conteneurs.

Pipeline : enchaînement automatisé des étapes de construction, de test et de déploiement d'une application.

Pod : plus petite unité déployable dans Kubernetes, regroupant un ou plusieurs conteneurs partageant le réseau et le stockage.

Registre de conteneurs : service de stockage et de distribution d'images Docker, ici le GitHub Container Registry.

Réplica : instance identique d'un pod ; plusieurs réplicas assurent la disponibilité et la répartition de la charge.

Rollback (retour arrière) : rétablissement de la version précédente d'une application après l'échec d'une mise à jour.

Rolling update : stratégie de mise à jour qui remplace progressivement les anciennes instances par les nouvelles, sans interruption de service.

Runner : agent qui exécute les tâches d'un workflow GitHub Actions ; il peut être hébergé par GitHub ou auto-hébergé.

Service (Kubernetes) : ressource offrant un point d'accès réseau stable à un ensemble de pods.

Test de bout en bout (E2E) : test qui vérifie le fonctionnement d'une application du point de vue de l'utilisateur, à travers l'ensemble de ses composants.

Workflow : processus automatisé défini dans un fichier YAML et déclenché par un événement du dépôt (push, pull request…).

TABLE DES MATIÈRES

CURRICULUM VITAE I

SOMMAIRE III

REMERCIEMENTS VI

LISTE DES FIGURES VII

LISTE DES TABLEAUX IX

LISTE DES ABRÉVIATIONS X

INTRODUCTION GÉNÉRALE 1

PARTIE I. PRÉSENTATIONS 2

Chapitre 1. Présentation de l'ENI 3

1.1. Information d'ordre général 3

1.2. Missions et historique 3

1.3. Organigramme institutionnel 5

1.4. Domaines de spécialisation 5

1.5. Architecture des formations pédagogiques 6

1.6. Relation de l'ENI avec les organismes externes 7

1.7. Débouchés professionnels des diplômés 8

1.8. Ressources humaines 11

Chapitre 2. Présentation de TETIKA 12

2.1. Information générale sur la société 12

2.1.1. Coordonnées pour la prise de contact avec l'entreprise 12

2.2. Missions et historique 12

2.2.1. Historique : de MANAO à TETIKA, « Grandir Ensemble » 12

2.2.2. Réussir à partir de Madagascar 13

2.2.3. Valeurs : Confiance, Progression, Responsabilité 13

2.3. Organigramme fonctionnel et hiérarchique de TETIKA 13
2.4. Domaines d'expertise 15
2.4.1. Compétences métiers 15
2.4.2. Branches et activités 16
2.5. Métiers 16
2.5.1. Métiers productifs 16
2.5.2. Services supports transversaux 16

Chapitre 3. Description du projet 17
3.1. Formulation 17
3.2. Objectif du projet 17
3.3. Moyens nécessaires à la réalisation du projet 17
3.3.1. Moyens humains 17
3.3.2. Moyens matériels 17
3.3.3. Moyens logiciels 18
3.4. Résultats attendus 18
3.5. Chronogramme de travail 19

PARTIE II. ANALYSE PRÉALABLE ET CHOIX DES OUTILS 21

Chapitre 4. Analyse de l'existant et problématique 22
4.1. Analyse de l'existant 22
4.1.1. Architecture logicielle : GESCOM 22
4.1.2. Processus de déploiement 23
4.1.3. Méthodologie de test 24
4.1.4. Gestion des régressions 25
4.1.5. Topologie de l'infrastructure 25
4.2. Problématique 29
4.3. Proposition de solution 30

Chapitre 5. Études théoriques et choix des outils 31
5.1. Notions de microservices 31
5.2. Gestionnaire de versions 31

5.3. Conteneurisation 33
5.4. Notions d'intégration continue et de déploiement continu 34
5.5. Notions d'orchestration 36
5.6. Choix de la solution de virtualisation 37
5.7. Scripts YAML 38
5.8. Plateformes d'orchestration de conteneurs Docker 39

PARTIE III. MISE EN ŒUVRE 41

Chapitre 6. Installations et configurations 42
6.1. Infrastructure 42
6.2. Automatisation CI/CD 45
6.2.1. Installation du runner 46
6.2.2. Configuration du pipeline 50
6.3. Configuration de l'orchestration : manifests Kubernetes 58
6.3.1. Déploiements 59
6.3.2. Ingress 63
6.3.3. Horizontal Pod Autoscalers 66
6.4. Application et services à déployer 69
6.4.1. Service d'authentification : Node.js / GraphQL / MySQL 70
6.4.2. Service de gestion des produits : Python / GraphQL / MySQL 75
6.4.3. Service de gestion des commandes : Node.js / GraphQL / MySQL 79
6.5. Vérification des jobs 87

Chapitre 7. Validations et perspectives 91
7.1. Validation du cluster Kubernetes 91
7.2. Validation des manifests des microservices 93
7.2.1. Notion de namespace 93
7.2.2. Validation des pods et des services 94
7.2.3. Validation de l'Ingress 95
7.3. Validation de la mise à l'échelle automatique 95
7.4. Validation d'un déploiement de bout en bout 97

7.5. Perspectives d'amélioration 103

CONCLUSION GÉNÉRALE XII

BIBLIOGRAPHIE XIII

WEBOGRAPHIE XIV

GLOSSAIRE XV

TABLE DES MATIÈRES XVII

RÉSUMÉ XXI

RÉSUMÉ

Ce mémoire présente la conception et la mise en œuvre d'un pipeline d'intégration et de déploiement continus (CI/CD) et de l'orchestration d'une application reposant sur des services indépendants. Chaque service fonctionne de manière autonome, communique avec les autres services et est testé automatiquement à chaque déploiement, ce qui garantit la fiabilité et la cohérence des fonctionnalités mises en production. Les services sont conteneurisés avec Docker, orchestrés par Kubernetes sur un cluster provisionné avec Vagrant et VirtualBox, et livrés par un pipeline GitHub Actions doté d'un mécanisme de retour arrière automatique. L'approche adoptée améliore la rapidité des mises à jour, facilite la maintenance, assure la continuité des services et réduit les erreurs liées aux interventions manuelles. Cette étude illustre l'importance de l'automatisation et de l'orchestration pour optimiser le cycle de vie des applications modernes.

Mots-clés : microservices, automatisation, conteneurisation, orchestration, CI/CD, Kubernetes, tests automatisés.

ABSTRACT

This thesis presents the design and implementation of a continuous integration and deployment (CI/CD) pipeline and of the orchestration of an application based on independent services. Each service operates autonomously, communicates with the other services and is automatically tested at every deployment, ensuring the reliability and consistency of the deployed features. The services are containerized with Docker, orchestrated by Kubernetes on a cluster provisioned with Vagrant and VirtualBox, and delivered by a GitHub Actions pipeline with an automatic rollback mechanism. This approach improves update speed, simplifies maintenance, ensures service continuity and reduces errors caused by manual interventions. The study highlights the importance of automation and orchestration in optimizing the lifecycle of modern applications.

Keywords: microservices, automation, containerization, orchestration, CI/CD, Kubernetes, automated testing.